Housing Price Prediction

A Data Analytics Portfolio Project on Predicting Residential Property Prices Using Machine Learning

Author: Pham Hoang Hai Van
Project Type: Data Analytics Portfolio Project
Tools Used: Python, Pandas, NumPy, Matplotlib, Scikit-learn, Jupyter Notebook

Project Overview

This project investigates the key factors influencing residential property prices and develops predictive models to estimate housing values using historical market data. The analysis covers data cleaning, exploratory data analysis, feature assessment, model development, and model evaluation to better understand pricing patterns and improve prediction accuracy.

Key Objectives

- Identify the main factors affecting housing prices
- Explore relationships between housing features and property values
- Build and compare predictive models
- Evaluate model performance using appropriate metrics
- Present findings in a clear and decision-oriented manner

Key Skills Demonstrated

- Data cleaning and preprocessing
- Exploratory data analysis
- Feature selection and interpretation
- Predictive modelling
- Model evaluation and comparison
- Analytical reporting and insight communication

Repository includes: project report, Jupyter Notebook, and dataset.

Table of Contents

1. Introduction 1
2. Designing a Solution 3
  2.1 Exploratory Data Analysis (EDA) 3
    2.1.1 Univariate Analysis 5
    2.1.2 Log-Transformation 6
    2.1.3 Outliers and Winsorization 8
    2.1.4 Categorical Variables and Price 9
    2.1.5 Bivariate analysis 11
  2.2. Feature Engineering and Data Preprocessing 15
  2.3 Model Design 16
  2.4 Evaluation Metrics 18
3. Experiments 19
  3.1 Linear Models (OLS, Lasso) 20
    3.1.1 OLS Results 20
    3.1.2 Lasso Regression 23
    3.1.2.1 Lasso baseline result 23
    3.1.2.2 Lasso Tuning result 25
    3.1.3 Comparative Discussion (OLS vs Lasso) 29
  3.2 Tree-based Models (Decision Tree, Random Forest, XGBoost) 30
    3.2.1 Decision tree 31
    3.2.1.1 Decision tree baseline 31
    3.2.1.2 Decision tree regressor 3 Experiments 32
    3.2.1.3 Decision Tree RandomizedSearchCV 33
    3.2.2 Random Forest 35
    3.2.2.1 Random Forest Baseline 35
    3.2.2.2 Random Forest 3 experiments 36
    3.2.2.3 Random Forest RandomizedSearchCV 38
    3.2.3 XGBoost 39
    3.2.3.1 XGBoost Baseline 39
    3.2.3.2 XGBoost 3 experiments 41
    3.2.3.3 XGBoost RandomizedSearchCV 42
    3.2.4 Comparative discussion (Decision Tree, Random Forest, XGBoost) 44
  3.3 Deep learning (ANN & GMDH-like) 45

3.3.1 Artificial Neural Network (ANN) 46
3.3.1.1 ANN baseline 46
3.3.1.2 ANN Experiment 1–3 (manual tuning) 48
3.3.2 GMDH-like 53
3.3.2.1 GMHD-like baseline 54
3.3.2.2 GMDH-like Experiment 1–3 (manual tuning) 55
3.3.3 Comparative Discussion (ANN vs GMDH-like) 59
3.4 Comparative Discussion Across Model Families (Linear vs Neural vs Tree-based) 60
4. Conclusion 64
References 66
Appendix 68

LIST OF TABLES

Section 2 Exploratory Data Analysis & Model Design

Table 1. Descriptive statistics and frequency summary of all features
Table 2. Summary statistics of price, area, and their log-transformed versions
Table 3. Variance Inflation Factors (VIF) for numerical predictors
Table 4. When each evaluation metric is most informative in housing price prediction

Section 3.1 Linear Models (OLS & Lasso)

Table 5. Performance metrics of OLS regression across Train, Validation, and Test sets
Table 6. OLS coefficient estimates and statistical significance (auxiliary OLS on TRAIN)
Table 7. Lasso baseline performance
Table 8. Lasso baseline coefficient estimates
Table 9. Lasso tuned performance
Table 10. Lasso tuned coefficient estimates
Table 11. Comparison of Lasso baseline and tuned model performance

Section 3.2 Tree-based Models

Table 12. Decision Tree baseline performance
Table 13. Experimental setup for Decision Tree Experiments 1–3

Table 14. Validation performance across Decision Tree experiments
Table 15. Decision Tree RandomizedSearchCV performance

Table 16. Random Forest baseline performance
Table 17. Experimental setup for Random Forest Experiments 1–3
Table 18. Performance across Random Forest experiments
Table 19 (RF). Random Forest RandomizedSearchCV performance

Table 19. XGBoost baseline performance
Table 20. Experimental setup for XGBoost Experiments 1–3
Table 21. Performance across XGBoost experiments
Table 22. XGBoost RandomizedSearchCV performance

Section 3.3 Neural / Polynomial Models

Table 23. GMDH-like baseline performance
Table 24. Performance of tuned GMDH-like experiments
Table 25. GMDH-like best vs baseline performance comparison

Section 3.4 Cross-family Comparison

Table 26. Summary of best models across Linear, Neural/Polynomial, and Tree-based families

LIST OF FIGURES

Section 2 Exploratory Data Analysis

Figure 1. Histograms of key numerical variables
Figure 2. Boxplots of price and area
Figure 3. Original and log-transformed distributions of price and area (before and after winsorization)
Figure 4. Boxplots of log(price) and log(area) after winsorization
Figure 5. Distributions and price comparisons for categorical features
Figure 6. Scatter plots of numerical predictors vs. price
Figure 7. Correlation matrix for numerical variables

Section 3.1 Linear Models

Figure 8. OLS residual distribution and Test Actual vs Predicted
Figure 9. Lasso baseline: Train residuals and Test Actual vs Predicted
Figure 10. Lasso tuned: Train residuals and Test Actual vs Predicted
Figure 11. Comparison of Lasso baseline vs tuned predictions

Section 3.2 Tree-based Models

Figure 12. Decision Tree feature importance
Figure 13. Random Forest feature importance
Figure 14. XGBoost feature importance

Section 3.3 Neural & Polynomial Models

Figure 15. ANN baseline: Actual vs Predicted and training loss curve
Figure 16. ANN_exp1_small: Actual vs Predicted and training loss curve
Figure 17. ANN_exp2_medium_dropout: Actual vs Predicted and training loss curve
Figure 18. ANN_exp3_tanh_stronger_dropout: Actual vs Predicted and training loss curve
Figure 19. Baseline vs tuned ANN: Actual vs Predicted comparison

Figure 20. GMDH-like baseline residuals and Test Actual vs Predicted
Figure 21. GMDH-like exp1_alpha_strong residuals and predictions
Figure 22. GMDH-like exp2_alpha_small residuals and predictions
Figure 23. GMDH-like manual_tuned_best residuals and predictions
Figure 24. Comparison of GMDH-like baseline vs tuned

Section 3.4 Cross-family Comparison

Figure 25. Test RMSE comparison across best models (Linear vs Tree vs GMDH-like)
Figure 26. Test R^2 comparison across best models
Figure 27. Train vs Test RMSE comparison for best models
Figure 28. Train vs Test R^2 comparison for best models

1. Introduction

Predicting housing prices has become an increasingly important task in real estate decision-making, financial planning, and urban development. The value of a residential home is determined by a mix of structural elements, neighborhood conditions, amenities, and market dynamics; they interact to create a very complex and usually nonlinear relationship with price. Ordinary Least Squares (OLS) has generally served as the basis of standard valuation studies because of its interpretability and long-standing affinity to hedonic pricing theory. Ordinary least squares (OLS) is still a often used beginning point because of its clarity and simplicity; as Nguyen and Cripps (2001) and Abhyankar and Singla (2022) point out. However, multicollinearity sometimes affects housing statistics, for example, the natural relationship between size and number of bedrooms, and the linear assumptions of OLS usually limit its ability to capture actual price fluctuations. Organised methods, especially Lasso Regression, offer enhancements by reducing less-informative coefficients and preventing overfitting, therefore suggesting that linear models yet have a crucial founding role in property valuation but need development to properly handle contemporary datasets (Akyüz et al., 2022; Preethi et al., 2025).

The literature has increasingly turned toward tree-based and ensemble techniques since housing markets involve interactions and non-linear patterns basic regression models find challenging to depict. Decision Trees, even if appealing for their interpretability, often overfit and generate inconsistent forecasts in practice, especially when the dataset contains both categorical and numerical variables. In response to these limitations, Random Forest has been widely adopted as reported in the works of Mora-Garcia et al. (2022) and Geerts, vanden Broucke, and De Weerdt (2023). By aggregating several decorrelated trees and often performs better than single-tree models on tabular data, Random Forest decreases variance. More recently, XGBoost has emerged as one of the most competitive algorithms for housing price prediction. Studies by Truong et al. (2020) and Zaki et al. (2022) show that XGBoost benefits from advanced regularisation, fast optimization, and the ability to model complex relationships, making it a strong candidate for achieving high predictive accuracy across a range of real-estate datasets. These developments together show a transition from simple interpretable models to more flexible, performance-driven machine-learning techniques.

Neural network methods have grown to be well-known for their capacity to reproduce complex and nonlinear functions in addition to ensemble learning. Artificial Neural Networks (ANN) are especially good at finding concealed patterns that linear or tree-based models might miss. Research

by Mostofi, Toğan and Başağa (2022) and Özdemir, Yıldız and Büyüktanır (2022) show that ANNs might outpace classic regression if the datasets are sufficiently large or multidimensional. However, they also demonstrate that neural models are really sensitive to sample size and need fine tuning; hence, their performance in smaller datasets is not necessarily guaranteed. To support this line of research, polynomial-based and GMDH-like techniques have been proposed as other paradigms for higher-order interactions free from the parameter intensity of deep learning. For example, Preethi et al. (2025) and Zhou (2021) show that while still keeping modest computational cost, hybrid optimization approaches and polynomial regression may correctly identify curvature effects in price changes. Together, these studies highlight how different modelling techniques have their own advantages and no one method always rules over every collection of data.

The range of outcomes leads most literature to recommend evaluating many modelling techniques while estimating home values. Particularly, Geerts et al. (2023) argue that to find the most appropriate option for the data at hand, strong studies should contrast linear, regularized, ensemble, and neural models. Following this guidance, the current study employs a multi-model strategy encompassing three primary categories: (i) baseline linear models including OLS and Lasso; (ii) non-linear machine-learning models such Decision Tree, Random Forest, XGBoost; and (iii) advanced or hybrid models represented by ANN and a GMDH-like polynomial technique. This framework lets the study benchmark traditional models against modern machine-learning approaches while also investigating the additional value of hybrid and higher-order modelling.

In addition to amenity-related characteristics like parking, basement, and air conditioning, the database of this study comprises 545 housing records including architectural traits such size, bedrooms, bathrooms, and stories. The characteristics discovered in earlier studies, such those in Mora-Garcia et al. (2022) and Satish et al. (2019), therefore enable the dataset fit for analysing a wide range of modelling methods. Through thorough exploratory data analysis, the objective of this study is to examine the dataset, apply suitable preprocessing and transformations, and evaluate seven chosen models' performance over linear, ensemble, and neural groups under this framework. The study will then try to improve the hyperparameters of these models, assess their generalisation performance on unobserved data, and ultimately identify the model producing the most accurate estimates for house prices. The research aims in this way to assist in more thoroughly understanding how several modelling families approach the complexity of real-estate data and present a clear, evidence-based comparison of current predictive techniques for house pricing.

2. Designing a Solution

From exploratory research to model evaluation and interpretation, a well-defined modelling technique was created and very effectively implemented to fulfil the needs of the project and solve the housing price estimate issue. The information utilized in this research originated from a publicly accessible Kaggle database with 545 residential properties described by a combination of categorical characteristics (e.g., mainroad, basement, guestroom, furnishings status) and numerical features (e.g., price, area, bedrooms, bathrooms, stories, parking). Considering that these characteristics are regularly studied in research of housing assessment, the dataset is perfect for benchmarking modern machine-learning methods as well as classic statistical models.

The reply evolved in several stages. First, a comprehensive exploratory data analysis (EDA) was performed to learn about data features, find outliers, look for skewness, and assess relationships among variables. Several preprocessing methods used based on preliminary insights were log-transformation of skewed variables (price and region), winsorization at the extreme percentiles, one-hot encoding for categorical factors, and scaling of numerical predictors. These procedures guaranteed that both linear and non-linear methods had statistically consistent, pristine, and suitable modelling input.

Following preprocessing, seven predictive models, one from each of three methodological families, were applied: (i) baseline linear models (OLS and Lasso), (ii) non-linear tree-based models (Decision Tree, Random Forest, XGBoost), and (iii) advanced models able of capturing higher-order structures (Artificial Neural Network and GMDH-like regression). Every model was hyperparameter adjusted using separate training, validation, and testing splits to ensure great generalization performance. Finding the most effective housing price prediction depends on a thorough framework that covers EDA, preprocessing, modelling, and evaluation.

2.1 Exploratory Data Analysis (EDA)

Before creating predictive models, an Exploratory Data Analysis was carried out to fully understand the housing data and identify any issues including skewed data, outliers, imbalanced categories, and multicollinearity. Central tendencies, dispersions, and category counts are shown in Table 1; this is a whole descriptive overview of all the factors. Particularly wide variances are seen in the numerical aspects price and area, with standard deviations of 1.87 million and 2,170 correspondingly. Significant differences exist between their medians (4.77M and 5,150) and medians (4.34M and 4,600). This gap points to great right-skewness and the probable presence of high-value outliers. By

contrast, structural components including bedrooms (mean 2.97, max 6) and bathrooms (mean 1.29, max 4) display quite constrained ranges, therefore suggesting more regular patterns across houses.

Many traits of categorical variables are quite imbalanced. For example, 85.9% of homes are on a main road; 82.2% lack a guestroom; and 95.4% lack hot-water heating. These primary categories stress the importance of confirming that impoverished populations keep showing substantial price fluctuations. Given that 41.7% of households are semi-furnished, furnishing status is more evenly distributed across its three tiers; therefore it is a perhaps revealing measure.

Particularly, the dataset has no missing values, therefore all 545 observations can be kept for analysis. These descriptive patterns, skewness, dispersion, and class imbalance, guide the use of visual inspection to analyze distribution forms, spot outliers, evaluate categorical effects, and investigate correlations among predictors before preprocessing and modelling procedures..

Table 1: Descriptive statistics and frequency summary of all features

Variable	Mean	Std	Min	25%	50%	75%	Max
price	4,766.73	1,870.44	1,750.00	3,430.00	4,340.00	5,740.00	13,300.00
area	5,150.54	2,170.14	1,650	3,600	4,600	6,360	16,200
bedrooms	2.97	0.74	1	2	3	3	6
bathrooms	1.29	0.50	1	1	1	2	4
stories	1.81	0.87	1	1	2	2	4
parking	0.69	0.86	0	0	0	1	3
price_log	15.31	0.37	14.44	15.05	15.28	15.56	16.17
log_area	8.47	0.39	7.59	8.19	8.43	8.76	9.44

Variable	Count	Unique	Top	Frequency	(%)
mainroad	545	2	yes	468	85.9%
guestroom	545	2	no	448	82.2%
basement	545	2	no	354	65.0%
hotwaterheating	545	2	no	520	95.4%
airconditioning	545	2	no	373	68.4%
prefarea	545	2	no	417	76.5%
furnishingstatus	545	3	semi-furnished	227	41.7%

2.1.1 Univariate Analysis

Early univariate research focused mostly on the distributions of significant numerical variables including price, area, bedrooms, bathrooms, stories, and parking. As shown in **Figure 1**, both *price* and *area* exhibit strong right-skewness, characterised by a small number of very expensive and very large houses that extend the upper tail of the distribution. This pattern is common in housing markets, where premium properties tend to inflate the high-end range of values.

In contrast, the distributions of *bedrooms*, *bathrooms*, *stories*, and *parking* are more concentrated, with most observations falling within narrow, expected residential ranges (e.g., 2-3 bedrooms, 1-2 bathrooms, 1-2 stories). Only a few extreme values appear for stories and parking, suggesting that these variables vary less across properties compared to price and area.

To further inspect extreme observations, boxplots for price and area were generated in **Figure 2** Both variables show substantial upper-tail outliers, confirming the skewness observed in the histograms. These outliers justify the need for transformation techniques, later addressed through log-transformation and winsorization, to stabilise variance and reduce the influence of extreme values before modelling.

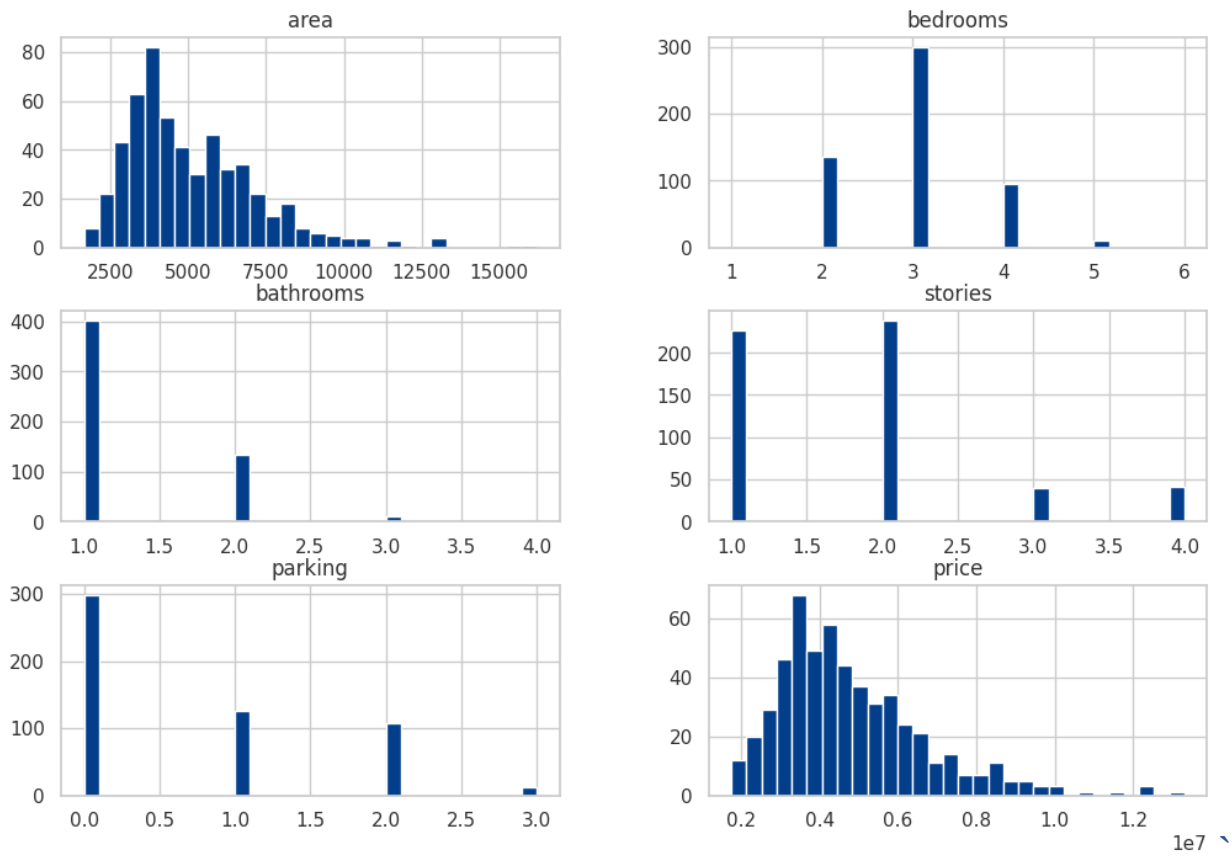


Figure 1: Histograms of key numerical variables

Figure 2: Boxplots of price and area

2.1.2 Log-Transformation

Because *price* and *area* exhibit pronounced right-skewness, log-transformations were applied to stabilise variance and reduce the disproportionate influence of high-end properties. As illustrated in **Figure 3**, the transformed variables display much more symmetric and centred distributions compared to their original forms, indicating a successful reduction in skewness. This effect is further supported by the summary statistics in **Table 2**, where the standard deviations of *price_log* and *log_area* decrease substantially to **0.37** and **0.39**, far lower than the wide spreads observed in the original scales. The compression of the upper tails and the narrower distribution ranges confirm that the log-transformation effectively normalises scale differences, producing variables that align more closely with the assumptions of linear models and contribute to more stable optimisation in neural-network training.

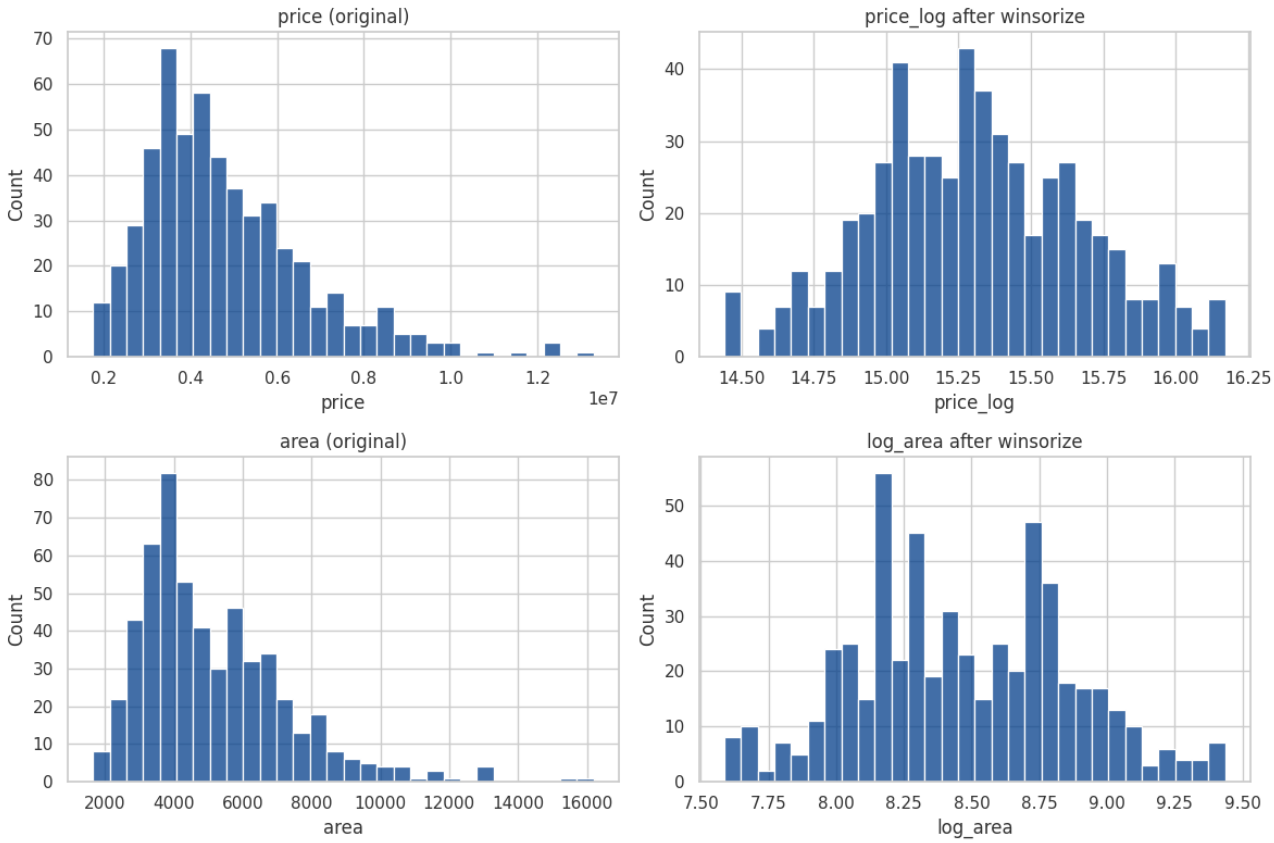


Figure 3: Original and log-transformed distributions of price and area (before and after winsorization)

Table 2: Summary statistics of price, area, and their log-transformed versions

Variable	Count	Mean	Std	Min	25%	50%	75%	Max

Price	545	4,766.73	1,870.44	1,750.00	3,430.00	4,340.00	5,740.00	13,300.00
Price_log	545	15.31	0.37	14.44	15.05	15.28	15.56	16.17
Area	545	5,150.54	2,170.14	1,650	3,600	4,600	6,360	16,200
Log_area	545	8.47	0.39	7.59	8.19	8.43	8.76	9.44

2.1.3 Outliers and Winsorization

Although the log-transformation substantially reduced skewness, the post-transformation boxplots in **Figure 4** still reveal thin yet noticeable upper and lower tails for both *price_log* and *log_area*, indicating the presence of remaining mild outliers. Removing these observations was not considered appropriate, as doing so would further reduce an already limited sample size of 545 records. Instead, winsorization at the 1st and 99th percentiles was applied to both transformed variables, allowing extreme values to be capped while preserving the overall structure and distributional shape.

Following winsorization, the updated boxplots in **Figure 4** show that most previously observed extreme points have been mitigated, with tighter interquartile ranges and fewer severe deviations. This results in a more stable and statistically reliable dataset for downstream modelling, reducing the risk of model sensitivity to unusually high- or low-valued observations.

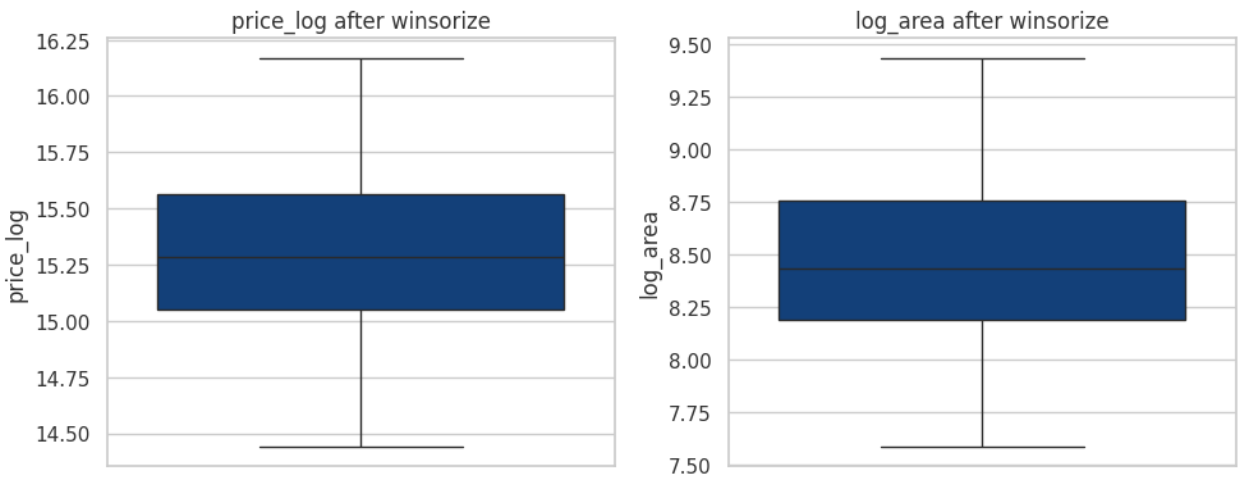


Figure 4: Boxplots of log(price) and log(area) after winsorization

2.1.4 Categorical Variables and Price

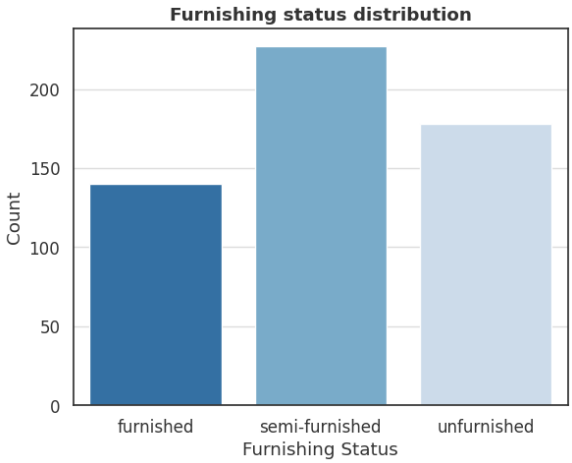
Together with many binary characteristics like mainroad, guestroom, basement, hotwaterheating, airconditioning, and prefarea, the dataset includes a three-level variable giving status, furnished, semi- furnished, unfurnished, Figure 5 illustrates their distributions combined with equivalent pricing estimates. The bar charts first reveal obvious class divisions: most houses are located on a big road; few have hot water heating; the most lack a guestroom or basement. Still with these skewed ratios, the boxplots clearly and distinctly differentiate price among categories.

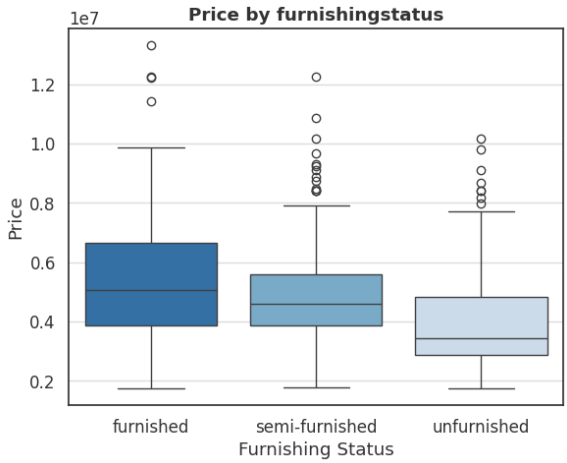
Regarding furniture condition, a very clear trend emerges whereby fully furnished homes command higher prices than those either semi-furnished or unfurnished. This is in line with traditional market behaviour; fully furnished houses cut buyers' front-end expenditures and are frequently regarded as more useful and ready to move in. Though this entails some possible noise trade-offs, properties on a major route are also probably priced a little more because of improved access and transportation convenience.

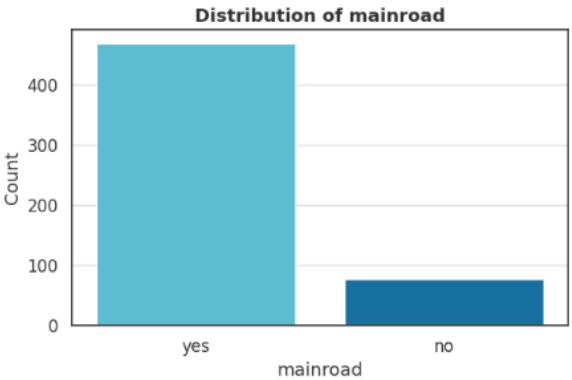
Higher price points also align with perks like a basement or guestroom. These additional rooms improve a home's functional capacity, therefore increasing seen value by providing space for visitors, storage, or multipurpose use. Air conditioning and hot water heating follow somewhat of the same pattern. Although hot water heating is not very common in the dataset, homes with it display more price ranges that mirror real-estate customs whereby contemporary comfort features fetch a premium. Likewise, homes with air conditioning usually cost more, especially in regions where the weather calls for an essential rather than discretionary amenity.

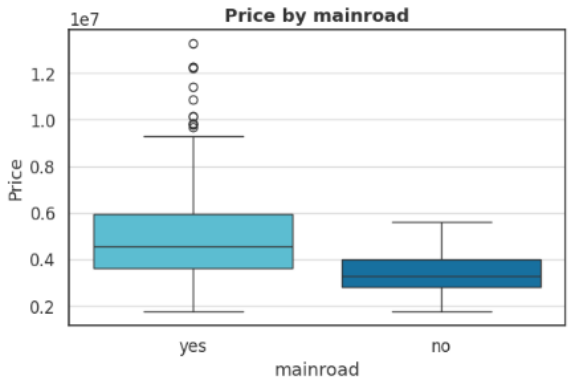
Finally, homes in a preferred location always command higher pricing, therefore highlighting how much buyers' decisions depend on the quality of the surrounding environment. Preferred locations sometimes have safety, access, and proximity to amenities as driving forces of housing value.

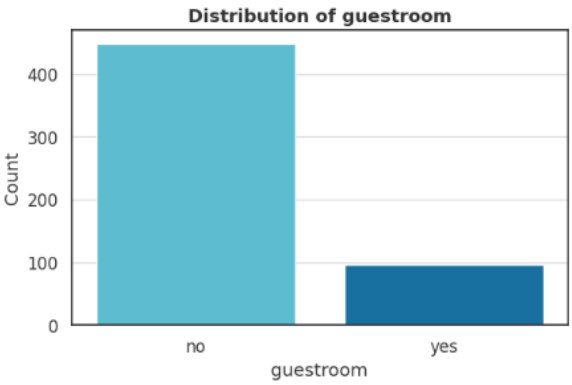
Together, these results reveal that categorical factors have major clarifying signals. From a real estate perspective, their links with price are intuitively clear and statistically obvious in the visualisations, hence vindicating the decision to retain all categorical aspects for the modelling step.

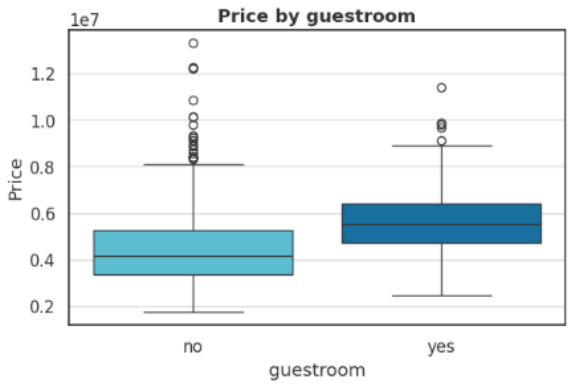

Furnishing status distribution
Count
0
50
100
150
200
furnished
semi-furnished
unfurnished
Furnishing Status


Price by furnishingstatus
1e7
Price
0.2
0.4
0.6
0.8
1.0
1.2
furnished
semi-furnished
unfurnished
Furnishing Status


Distribution of mainroad
Count
0
100
200
300
400
yes
no
mainroad


Price by mainroad
1e7
Price
0.2
0.4
0.6
0.8
1.0
1.2
yes
no
mainroad


Distribution of guestroom
Count
0
100
200
300
400
no
yes
guestroom


Price by guestroom
1e7
Price
0.2
0.4
0.6
0.8
1.0
1.2
no
yes
guestroom

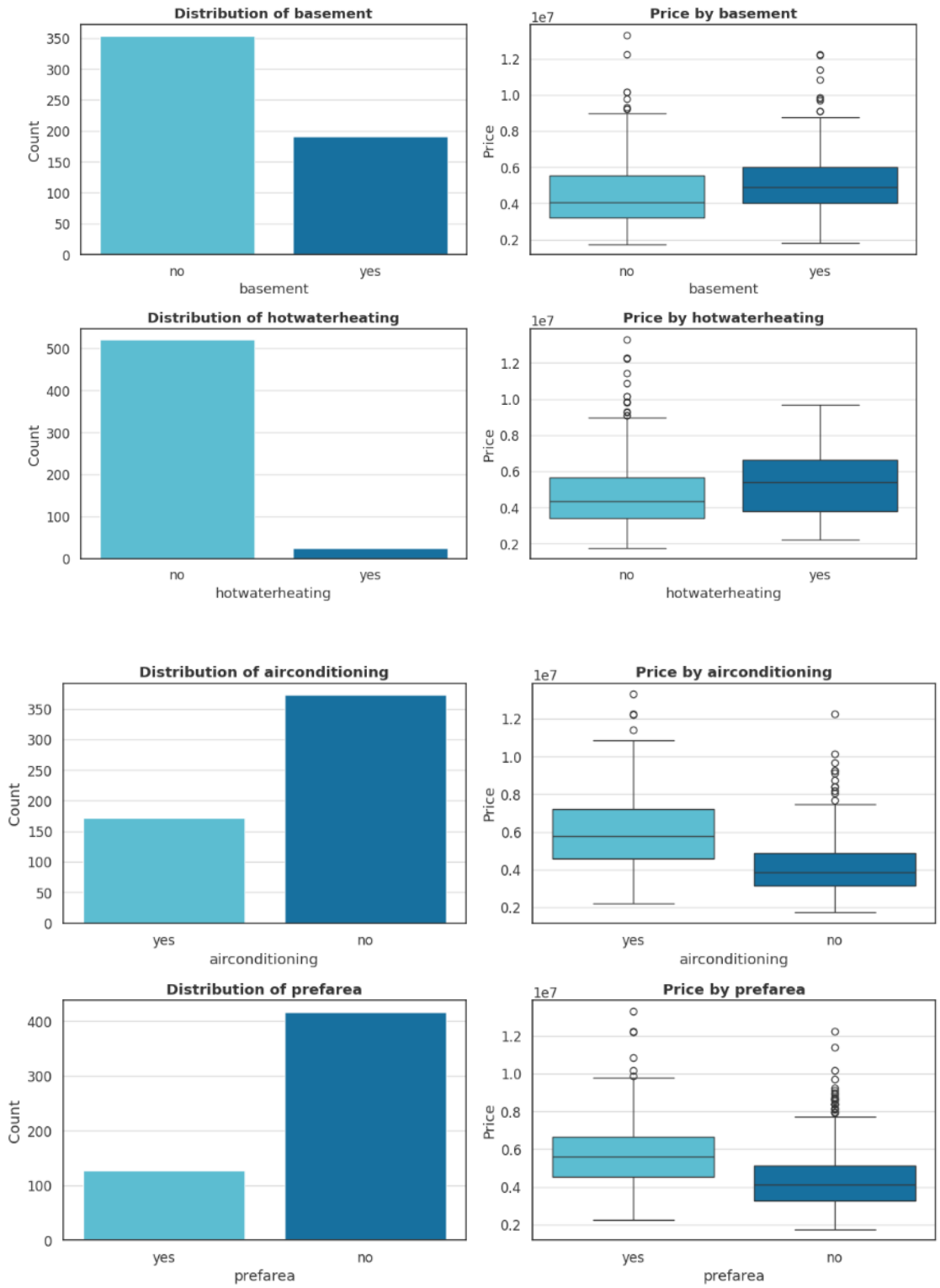


Figure 5: Distributions and price comparisons for categorical features

2.1.5 Bivariate analysis

In addition to univariate findings, scatter charts were employed as seen in Figure 6 to look at links between price and important numerical predictors. Price and area are significantly positively

correlated, hence mirroring one of the most strong relationships in real-estate assessment: bigger residences generally fetch more. Stories and parking also demonstrate little but steady rise, implying that multi-story buildings and more parking area add value. On the other hand, the links between bedrooms and bathrooms appear weaker and more spread out, suggesting declining marginal effects after basic living demands are satisfied or potential interactions with other components including overall area.

A correlation heatmap (Figure 7) was made to gauge these interactions using numerically variable log-transformed. Correlations with price_log vary from a low level, for example: 0.54 with log_area; 0.52 with bathrooms; and 0.42 with stories. These results indicate that numerous elements have a great impact on pricing variability without any one factor being very strong. Remarkably, none of the pairwise correlations reach levels that would cause multicollinearity concerns. Even predictor-to-predictor correlations like log_area–bedrooms or stories–parking remain modest, therefore confirming the fitness of the data for multivariate analysis.

Further confirming this, Variance Inflation Factors (VIF) were computed on the processed feature set (following winsorization and encoding). As indicated in Table 3, the findings show VIF numbers for all predictors between 1 and 2, much below the often used warning levels of 5 or 10. By demonstrating definitively that multicollinearity is not a major issue in the data, this ensures that linear models like OLS and Lasso may work reliably while tree-based and neural network models are not adversely affected by unnecessary information.

Together, the scatter plots, correlation matrix, and VIF analysis show that stable and clear relationships between numerical predictors and house prices exist while keeping strong statistical qualities for future modelling.

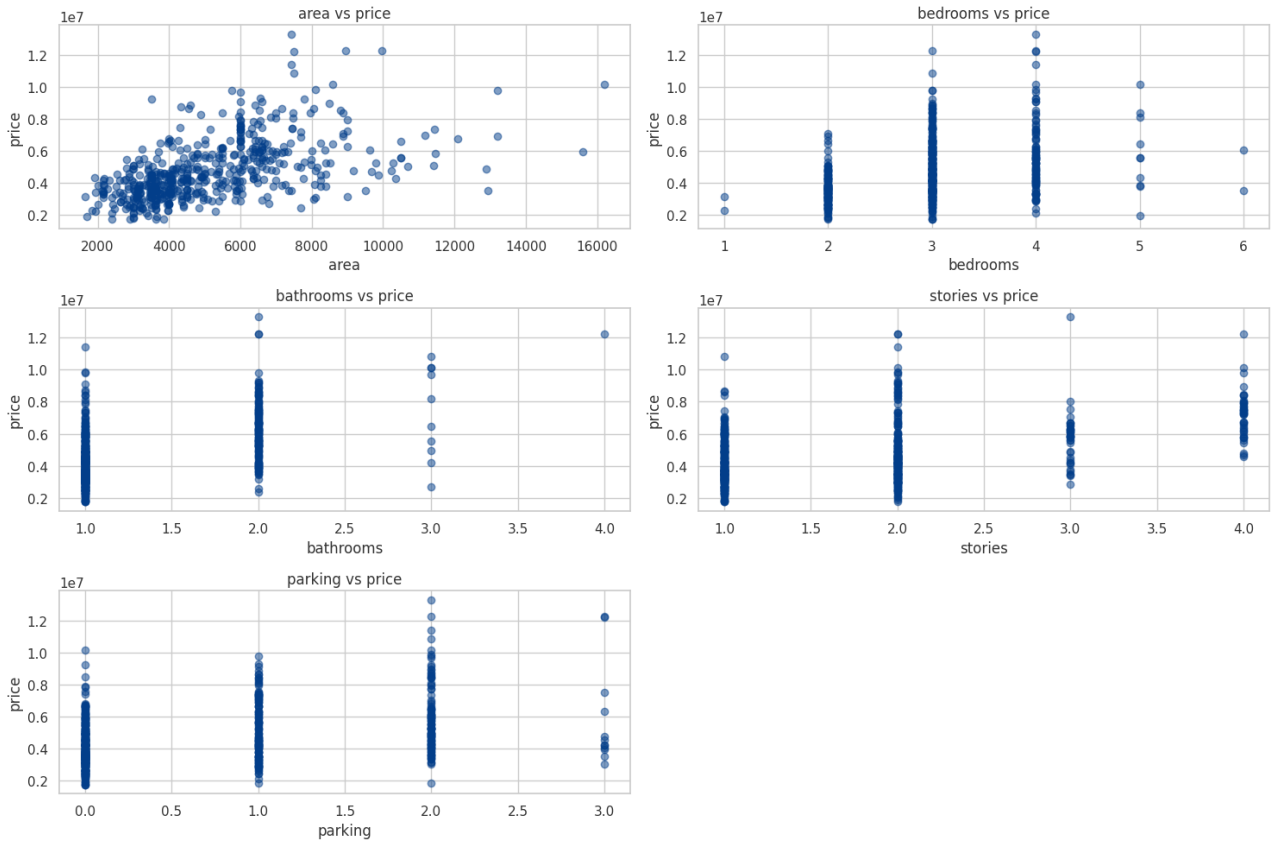


Figure 6: Scatter plots of numerical predictors vs. price

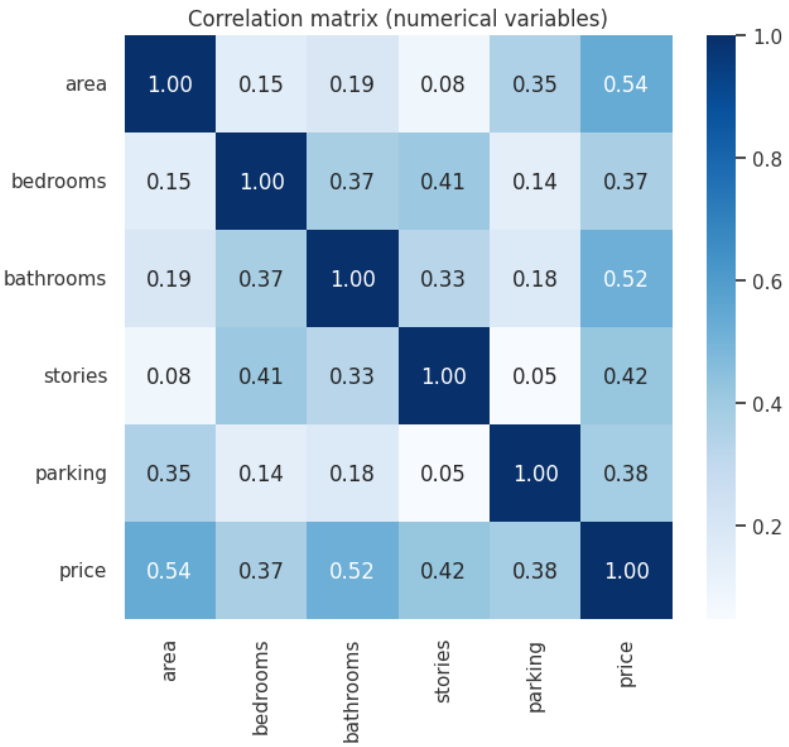


Figure 7: Correlation matrix for numerical variables

Table 3: Variance Inflation Factors (VIF) for numerical predictors

Feature	159	475	527	311	479
log_area	8.06	8.01	7.59	8.71	8.21
bedrooms	3	2	2	2	4
bathrooms	2	1	1	1	1
stories	1	2	1	1	2
parking	0	0	0	1	0
mainroad	1 (Yes)	1 (Yes)	0	0	0
guestroom	1 (Yes)	0	0	1 (Yes)	0
basement	1 (Yes)	0	1 (Yes)	0	0
hotwaterheating	0	0	0	0	0
airconditioning	1 (Yes)	0	0	0	0
prefarea	0	0	0	0	0
furnish_semi	0	1 (Yes)	1 (Yes)	1 (Yes)	0
furnish_un	0	0	0	0	1 (Yes)

	feature	VIF
0	const	597.641967
1	log_area	1.364474
2	bedrooms	1.357998
3	bathrooms	1.346346
4	stories	1.552808
5	parking	1.190602
6	mainroad	1.193933
7	guestroom	1.257951
8	basement	1.351131
9	hotwaterheating	1.045987
10	airconditioning	1.288974
11	prefarea	1.124136

2.2. Feature Engineering and Data Preprocessing

Following the first assessment, several preprocessing techniques were used to produce model-ready, clear dataset. The modifications made at this level were guided by observations from EDA and the requirements of the predictive models employed throughout further chapters.

With 1 for yes and 0 for no, binary yes/no variables, mainroad, guestroom, basement, hotwaterheating, airconditioning, and prefarea, were mapped to numerical values. For the sole

multi-level categorical variable, furnishingsstatus, one-hot encoding was used to produce two dummy variables (furnish_semi and furnish_unfurnished), with furnished implicitly acting as the baseline category. This guaranteed that the encoded data met the assumptions of linear models while avoiding multicollinearity from redundant dummy columns.

The numerical component of the modelling dataset consisted of the transformed variables *price_log* and *log_area*, together with the amenity-related predictors *bedrooms, bathrooms, stories,* and *parking*. After combining these numerical features with the encoded categorical variables, the resulting dataset provided a consistent representation of all property characteristics while maintaining the improvements achieved through log-transformation and winsorization.

To ensure that all numerical predictors operated on comparable scales, particularly important for regularised regression models and neural networks, a **StandardScaler** was fitted on the training set and applied to the validation and test sets. This transformation standardised each numerical feature to have zero mean and unit variance without affecting categorical encodings.

Finally, after 80/20 split the processed dataset was split into three subsets:

64% (348, 13) for training,

16% (88, 13) for validation, and

20% (109, 13) for testing,

guarantee that the data used for model development and hyperparameter tuning was distinct from the ultimate evaluation set.

2.3 Model Design

Once the dataset had been cleaned, transformed, and scaled, the modelling phase was arranged around seven representative algorithms picked to reflect very different ideas on how home features influence property values. The study allows evaluation of both basic and extremely flexible functional forms thanks four systematic families: linear models, polynomial-based methods, tree-based ensembles, and neural-network designs.

The first group comprises linear models beginning with Ordinary Least Squares (OLS), which creates the conceptual basis. OLS offers an easy benchmark for assessing whether more complex patterns are present in the data by assuming that predictors additively influence the target variable.

Regularized linear model (Lasso) was also included with OLS to address possible multicollinearity by reducing less significant coefficients. Though both models call for the definition of basic parameters such the fit method and the strength of penalization in the case of Lasso, they differ in their dependence on regularisation terms.

Drawing its polynomial-based approach from the Group Method of Data Handling (GMDH), the second modelling family expands the predictor space by polynomial transforms to enable the model to capture curved or higher-order correlations between characteristics rather than only fitting linear effects. It depends on factors like the polynomial degree, the regularisation strength used to prevent overfitting, and the optimisation method selecting the final set of polynomial terms. This model acts as an intermediate step between simple linear structures and more complex non-linear learners

Starting with a stand-alone Decision Tree, the third family includes tree-based and ensemble approaches. Through recursive splitting, a tree divides the characteristic spaace into decision regions, hence capturing interactions and threshold effects that linear models cannot show. Model building is helped by picking important structural features like the greatest tree depth, the minimum observation count required to divide a node, and the lowest sample count dispatched to a leaf. Building on this starting point, a Random Forest method combines many decorrelated trees to steady forecasts and hence reduce variance. Its design improves predictive flexibility more than bagging does by sequential error-correction and necessitates rigorous parameter configuration governing learning dynamics including the number of boosting rounds, the learning rate, the depth of each weak learner, subsampling percentages, and the objective function.

The final modelling group comprises multi-layer Artificial Neural Networks (ANN), using neural-network techniques. By knowledge distribution across hidden neural layers, neural models tackle difficult non-linear patterns. Because the architecture chosen for this study was designed to balance expressive ability with normalisation, particular specification of the number of hidden layers, the number of units per layer, the activation functions, the dropout rate, and the optimization method employed during training was needed. To ensure steady convergence, since neural networks are trained iteratively, other factors including batch size, epoch boundaries, and early-stopping standards were also specified.

Together, these seven models present a diverse modelling environment spanning understandable linear frameworks to sophisticated non-linear learners capable of approximating high-dimensional functional linkages. This design ensures that the comparative results presented later in the study

precisely reflect real methodological variances consistent with the aims of housing pricing forecast rather than only varying tuning intensities.

2.4 Evaluation Metrics

Using a regular assessment method, fair and consistent comparison among every model was guaranteed. Since every model was developed to forecast log(price) rather than the raw target, predictions were originally created on the log scale and then exponentially changed back to the initial financial units. Better model behaviour more uniform across linear and nonlinear methods, stabilisation variance, and reduced heteroscedasticity all result from this strategy.

Four main indicators on the original pricing scale were calculated using a bespoke evaluation function to guarantee that the results reflect genuine prediction performance rather than deformations from the changed target. Including RMSE, MAE, MAPE, and R^2, these indices each catch several facets of model accuracy. R^2 denotes the proportion of variance explained by the model; RMSE punishes big errors more severely; MAE represents average absolute deviation; MAPE conveys errors as percentages relative to true market prices.

Given that the project strives to find the most dependable and widely applicable predictive model, R^2 on the test set is the main ranking indicator. Apart from R^2, RMSE, MAE, and MAPE are demonstrated to offer a more whole view of model error structure, especially for evaluating stability in light of skewed price distributions or outliers.

Furthermore, the evaluation pipeline generates uniformly formatted summaries across all modelling methods, therefore standardising model results. This includes computation of entire error indicators, translation from log estimates to genuine pricing, and optional visualisations such residual plots or actual-versus-predicted charts. These factors help linear regressors, tree-based models, boosting ensembles, and neural networks to all stand against equal footing.

Formulas:

Formula 1: Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - y\hat{}_i)^2}$$

Formula 2: Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - y\hat{}_i|$$

Formula 3: Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - y\hat{}_i}{y_i}\right|$$

Formula 4: Coefficient of Determination (R^2)

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - y\hat{}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y}_i)^2}$$

Table 4: When each metric is most informative in housing price prediction

Metric	Primary Meaning	When it is most important in the context of housing price prediction
RMSE	Emphasises large errors	When ensuring the model does not make severe mistakes for high-value or luxury properties, where large deviations are costly.
MAE	Average absolute prediction error	When a stable, easy-to-interpret measure of the "typical" prediction error is needed, with minimal influence from outliers.
MAPE	Percentage error relative to true values	When stakeholders require results expressed in percentage terms, or when comparing prediction accuracy across different price ranges.
R^2	Proportion of variance explained	When ranking models and assessing overall goodness-of-fit, especially on the test set to evaluate generalisation capability.

3. Experiments

The experimental phase was designed to evaluate how different modelling philosophies, linear, tree-based, and neural approaches, respond to the structure of the cleaned and transformed housing

dataset. All experiments were implemented in Python and executed on Google Colab to ensure a consistent runtime environment and reproducible results.

Across all selected model families, the overarching goal was to examine how each technique handles non-linearity, feature interactions, scale sensitivity, and regularisation requirements, especially after the extensive preprocessing steps carried out in Section 2. Each experiment followed a consistent process to guarantee comparability: models were trained on log(price) and then turned back to the original scale for analysis; identical train/validation/test splits were employed so performance differences result only from modelling ability; through a standardised reporting function, the same evaluation criteria, RMSE, MAE, MAPE, and test-set R^2, were calculated for every model to enable side-by-side interpretation of results; hyperparameter tuning was applied selectively within each model family to investigate how parameter choices affect generalisation, overfitting behavior, and complexity management.

Based on the characteristics of the dataset and the benefits of several algorithm families, seven models were chosen and classified into four groups for rigorous testing. Every group represents a different modelling premise: linearity, polynomial interactions, hierarchical partitioning, or non-linear function approximation; therefore, the experiments will reveal not just which model works best but also why some modelling techniques fit more with the under-lying housing market patterns.

3.1 Linear Models (OLS, Lasso)

The linear-model experiments establish a statistical baseline before transitioning to more flexible non-linear approaches. Two models were examined in this group: Ordinary Least Squares (OLS) as the simplest benchmark, and Lasso regression, which introduces L1 (a penalty that shrinks less important coefficients toward zero) regularisation to control coefficient magnitude, reduce variance, and improve generalisation when predictors exhibit mild correlation.

3.1.1 OLS Results

In the linear-model experiments, OLS was implemented using scikit-learn's *LinearRegression* with its default configuration, so no hyperparameters were tuned for this baseline.

Table 5 summarises the predictive performance of OLS across the Train, Validation, and Test sets. Performance remains relatively consistent across splits, with a Test R^2 of 0.679, indicating that the linear baseline captures a substantial portion of variance but still struggles with non-linear patterns inherent in housing prices.

Table 5: Performance metrics of OLS regression across Train, Validation, and Test sets

Dataset Split	RMSE	MAE	MAPE (%)	R2 Score
Train	930,811.30	657,666.08	14.46%	0.7071
Validation	982,570.80	738,291.91	15.29%	0.6595
Test	1,200,600.00	931,822.20	20.03%	0.6791

Figure 8**,** visualises the residual distribution and the Actual vs Predicted relationship. Residuals on the training set display an approximately symmetric shape centred near zero, though with visible heavy tails. The scatterplot confirms a broadly linear relationship but also shows systematic under- and over-prediction at the extremes, consistent with the known limitations of linear models in real-estate valuation.

A further observation comes from the distribution of large positive residuals at higher price ranges. These patterns suggest that as property values increase, the true relationship becomes increasingly nonlinear, causing OLS to systematically underestimate luxury homes. This behaviour later motivates the use of models capable of capturing curvature or higher-order interactions.

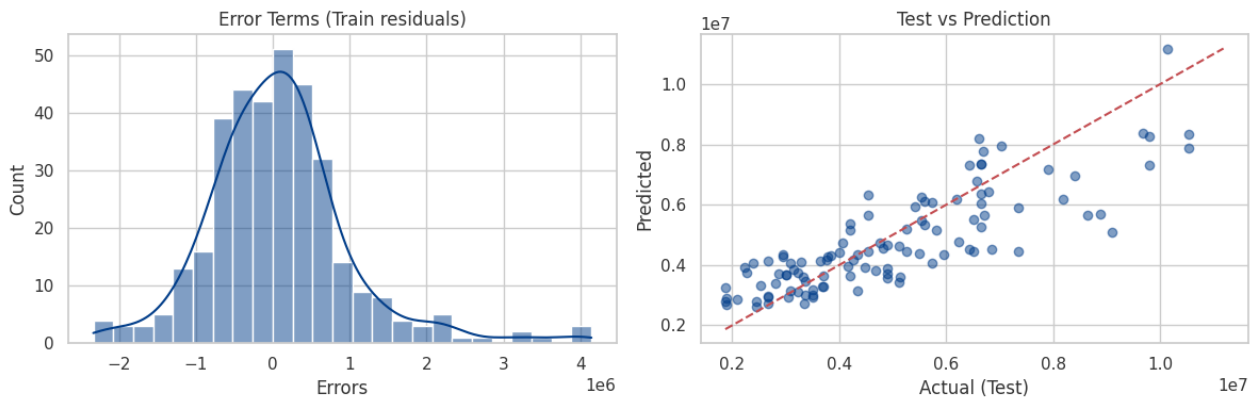


Figure 8: OLS residual distribution and Test Actual vs Predicted scatter plot

Table 6, reports coefficient estimates and statistical significance from an auxiliary OLS fitted directly on the log-transformed training data. Several predictors exhibit strong and statistically significant positive relationships with housing prices. For example, *log_area* has a coefficient of 0.1171 with p-value 2.89e-20, reflecting a highly robust elasticity of price with respect to property

size. Amenities such as *bathrooms* (coef 0.0814**,** p 2.75e-11), *stories* (coef 0.0795, p 1.24e-09), *basement* (coef 0.0552**,** p 5.64e-06) and *airconditioning* (coef 0.0618, p 1.86e-07) similarly display strong, significant associations. In contrast, *guestroom* (p 0.593) and *semi-furnished status* (p 0.661) are non-significant, indicating negligible contribution to price variation. Meanwhile, *unfurnished status* shows a significant negative effect (coef –0.0513**,** p 1.50e-04), consistent with market expectations.

Table 6: OLS coefficient estimates and statistical significance (auxiliary OLS on TRAIN)

	variable	coef	std_err	t_value	p_value
0	const	15.2901	0.0102	1,503.9891	0.000000e+00
1	log_area	0.1171	0.0119	9.8508	2.887954e-20
2	bedrooms	0.0185	0.0119	1.5642	1.187187e-01
3	bathrooms	0.0814	0.0118	6.8922	2.748691e-11
4	stories	0.0795	0.0127	6.2512	1.242992e-09
5	parking	0.0263	0.0112	2.3588	1.891157e-02
6	mainroad	0.0378	0.0112	3.3842	7.987207e-04
7	guestroom	0.0061	0.0114	0.5349	5.930988e-01
8	basement	0.0552	0.0120	4.6139	5.639743e-06
9	hotwaterheating	0.0334	0.0104	3.2026	1.492916e-03
10	airconditioning	0.0618	0.0116	5.3245	1.863055e-07
11	prefarea	0.0572	0.0108	5.2977	2.133429e-07

12	furnishingstatus_semi-furnished	-0.0057	0.0129	-0.4394	6.606561e-01
13	furnishingstatus_unfurnished	-0.0513	0.0134	-3.8345	1.503663e-04

3.1.2 Lasso Regression

Lasso regression introduces L1 regularisation to shrink coefficients and mitigate potential multicollinearity. Because Section 3.1 already outlines the general motivation for Lasso, this subsection focuses on empirical performance.

3.1.2.1 Lasso baseline result

Hyperparameter Search

Lasso, however, required explicit regularisation control. The baseline version was trained using:

alpha = 0.0005**;** random_state = 42

max_iter = 10,000

Hyperparameter Rationale

The baseline value of **α = 0.0005** was selected as a *mild* regularisation strength, large enough to introduce meaningful L1 penalisation, yet small enough to avoid overly suppressing coefficients in a dataset that does not exhibit strong multicollinearity. This setting allows observation of whether even light shrinkage improves stability or reduces noise relative to the unregularised OLS model. Keeping the random seed and iteration limit fixed ensures comparability across experiments.

Baseline Performance

Table 7: Lasso baseline performance

Split	RMSE	MAE	MAPE (%)	R2 Score
Train	930,881.97	657,462.78	14.46%	0.7070
Validation	982,713.71	737,598.15	15.26%	0.6594
Test	1,201,690.77	932,334.24	20.04%	0.6785

Table 7 summarises predictive accuracy across Train, Validation, and Test splits. With a Test RMSE of **1,201,690.77** and R^2 of **0.6785**, performance is nearly identical to OLS, indicating that the baseline level of shrinkage does not materially affect predictive capability. Errors rise noticeably from Validation to Test, and MAPE increases to **20.04%**, reflecting the same difficulty observed in OLS when predicting higher-priced homes.

Baseline Diagnostic Plots

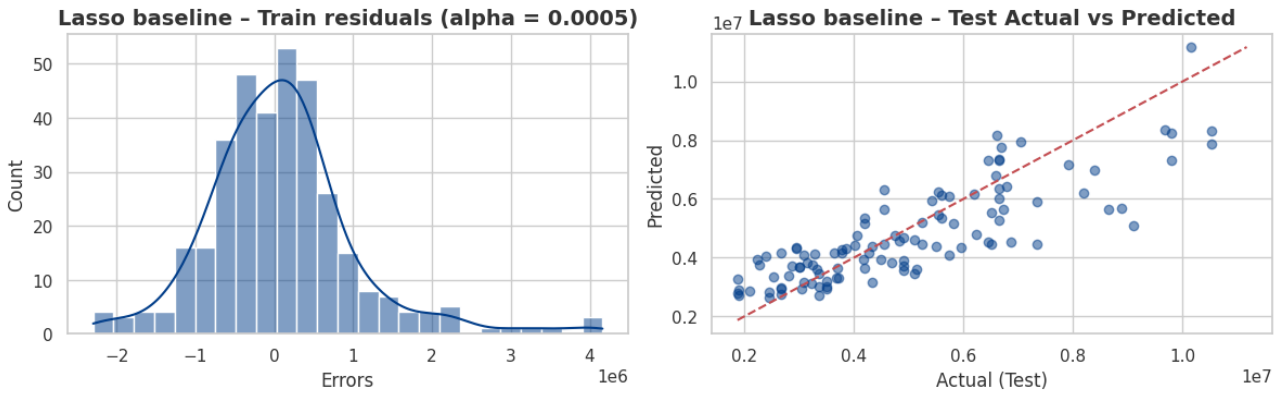


Figure 9: Lasso baseline residual distribution and Test Actual vs Predicted

Figure 9 visualises residual distributions and Test-set predictions. The residual histogram shows a shape similar to OLS, though very slightly narrower, consistent with mild shrinkage stabilising extreme errors. The Test Actual vs Predicted scatterplot continues to reveal underestimation at the upper end of the price range, confirming that $\alpha = 0.0005$ does not meaningfully alter the model's structural behaviour.

Table 8 reports the coefficients learned under the baseline α. The values remain close to those of OLS, e.g., *log_area* = **0.1169**, *bathrooms* = **0.0813**, *stories* = **0.0793**, reflecting minimal shrinkage. Only furnishing-status variables show more noticeable contraction, consistent with L1's tendency to weaken coefficients associated with weaker predictive signals.

Table 8: Lasso baseline coefficients

	variable	coef_lasso
0	log_area	0.1169
1	bedrooms	0.0184
2	bathrooms	0.0813

3	stories	0.0793
4	parking	0.0261
5	mainroad	0.0376
6	guestroom	0.0059
7	basement	0.0549
8	hotwaterheating	0.0329
9	airconditioning	0.0617
10	prefarea	0.0569
11	furnishingstatus_semi-furnished	-0.0045
12	furnishingstatus_unfurnished	-0.0505

The baseline Lasso configuration behaves almost identically to OLS across all evaluation metrics, coefficient magnitudes, and diagnostic plots. This outcome suggests three key points.

First, the dataset exhibits **low multicollinearity**, meaning L1 regularisation has little structural effect. Second, the relationship between predictors and price remains **predominantly linear**, limiting the benefit of shrinkage-based smoothing. Third, the mild baseline α does not impose enough constraint to meaningfully reshape the model; therefore, exploring smaller or larger α values through systematic tuning becomes necessary.

These observations directly motivate the tuning experiments in Section 3.1.2.2, which investigate whether alternative shrinkage levels yield more substantial gains.

3.1.2.2 Lasso Tuning result

Hyperparameter Search

To identify an optimal shrinkage level, a controlled search was performed over a small grid of candidate values:

$\alpha \in \{1e\text{-}5, 5e\text{-}5, 1e\text{-}4, 5e\text{-}4, 1e\text{-}3, 5e\text{-}3\}$

All experiments fixed:

random_state = 42

max_iter = 10,000

These conditions ensured comparability with the baseline model and isolated the effect of α on estimation stability and generalisation.

Hyperparameter Rationale

The tuning grid was purposely constructed to span the full range from minimal regularisation (1e-5) to aggressive shrinkage (5e-3). Very small α values test whether the model benefits from only a slight penalty, suppressing noise without removing meaningful signal. Medium α values induce coefficient sparsification and allow weaker predictors to be selectively shrunk. Larger α values represent the threshold where overshrinkage may lead to underfitting. This design systematically captures the transition from under-regularisation → optimal regularisation → over-regularisation.

Across the grid, α = 1e-5 produced the strongest validation performance and is therefore selected as the Lasso Tuned configuration.

Tuned Model Performance

Table 9 summarises the performance of the tuned model across all splits. Test RMSE decreases from **1,201,690.77** (baseline) to **1,200,621.06**, and R^2 rises slightly to **0.6791**. Although these improvements are small, they reflect the benefit of applying only minimal shrinkage; stronger penalties in the grid noticeably worsened the validation metrics.

Table 9: Lasso tuned model performance

Split	RMSE	MAE	MAPE (%)	R2 Score
Train	930,812.00	657,661.90	14.46%	0.7071
Validation	982,572.83	738,278.02	15.29%	0.6595
Test	1,200,621.06	931,832.37	20.03%	0.6791

Diagnostic Plots

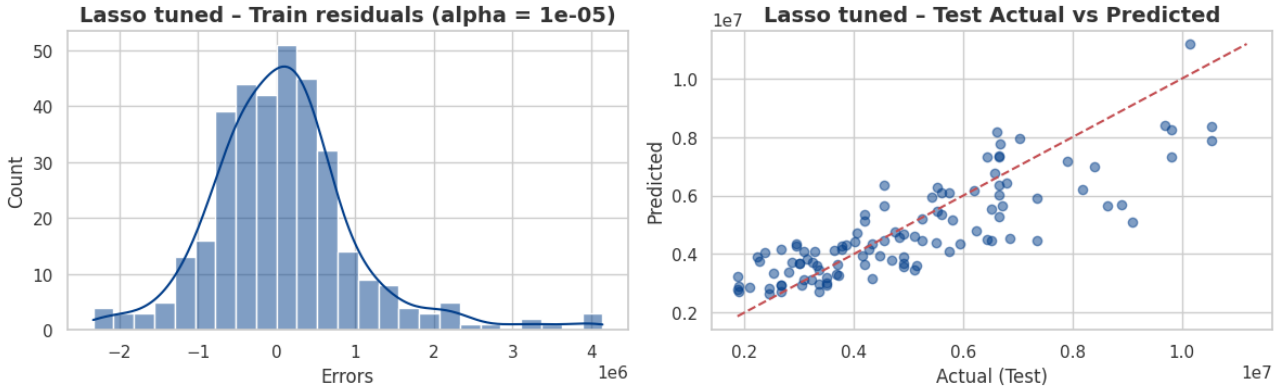


Figure 10: Lasso tuned residual distribution and Test Actual vs Predicted

Figure 10 presents the residual distribution and Test-set predictions for the tuned model. The residual histogram shows a slightly sharper and more concentrated peak around zero compared with the baseline, indicating a modest improvement in stability and a small reduction in variance. This behaviour is consistent with the effect of extremely light L1 regularisation, which suppresses noise without materially altering the structure of the fitted model.

The Test-set scatterplot also reveals marginal tightening around the 45-degree reference line, particularly within the mid-price range. Nevertheless, high-value properties continue to be systematically underestimated, demonstrating that tuning α alone is insufficient to overcome the inherent linear limitations of Lasso for this housing dataset.

Table 10 reports the coefficient values under the tuned setting α = 1e-5. The estimates differ only minimally from those of the baseline model. For example, *log_area* increases from 0.1169 to 0.1171, *bathrooms* from 0.0813 to 0.0814, and *prefarea* from 0.0569 to 0.0572. Such small movements confirm that the tuned configuration lies at the boundary of negligible shrinkage: the penalty is strong enough to regularise noise but not substantial enough to reshape the coefficient vector in any meaningful way.

Table 10: Lasso tuned coefficients

	variable	**coef_lasso_tuned**
0	log_area	0.1171
1	bedrooms	0.0185

2	bathrooms	0.0814
3	stories	0.0795
4	parking	0.0263
5	mainroad	0.0378
6	guestroom	0.0061
7	basement	0.0552
8	hotwaterheating	0.0334
9	airconditioning	0.0618
10	prefarea	0.0572
11	furnishingstatus_semi-furnished	-0.0056
12	furnishingstatus_unfurnished	-0.0513

Taken together, the tuning results show that this dataset responds best to extremely small L1 penalties. More aggressive shrinkage reduces model flexibility and consistently worsens validation performance, suggesting that multicollinearity is relatively mild and that the underlying price relationship remains largely linear. The very small improvements observed in RMSE and R^2 illustrate that Lasso's regularisation provides only limited value in this context. Although the tuned model performs slightly better than the baseline, the gains remain incremental, reinforcing the need for more expressive nonlinear models to capture curvature and the pronounced underestimation of high-priced properties evident in the diagnostic plots.

Lasso Regression: Baseline vs Tuning

The impact of regularisation was assessed by comparing the baseline Lasso configuration with the tuned version that employed a substantially smaller penalty weight. Table 11 summarises their predictive accuracy on the Test set, and the results reveal only marginal differences: the tuned model reduces RMSE by approximately 1,070 VND and increases the R^2 score from 0.6785 to 0.6791. Such a minute improvement indicates that L1 regularisation has little room to influence model behaviour under the conditions of this dataset. Because the design matrix exhibits low multicollinearity and the underlying price mechanism is predominantly linear, the tuning process cannot produce structural changes to the model, only fine adjustments to coefficient magnitudes.

Table 11: Comparison of Lasso baseline and tuned model performance

Model	RMSE	MAE	MAPE (%)	R2 Score
Lasso Baseline	1,201,690.77	932,334.24	20.04%	0.6785
Lasso Tuned	1,200,621.06	931,832.37	20.03%	0.6791

Figure 11, overlays residual distributions and Test-set predictions for both versions. The density curves almost coincide, reflecting that the tuned α value modifies the scale of the coefficients without altering the direction or pattern of the residuals. The scatter plot reinforces this view: both models organise their predictions along nearly identical trajectories, and although the tuned version shows slightly tighter clustering around the 45-degree line, the improvement is subtle. High-value properties remain underestimated in both cases, which confirms that the primary challenge lies not in overfitting but in the linear specification itself.

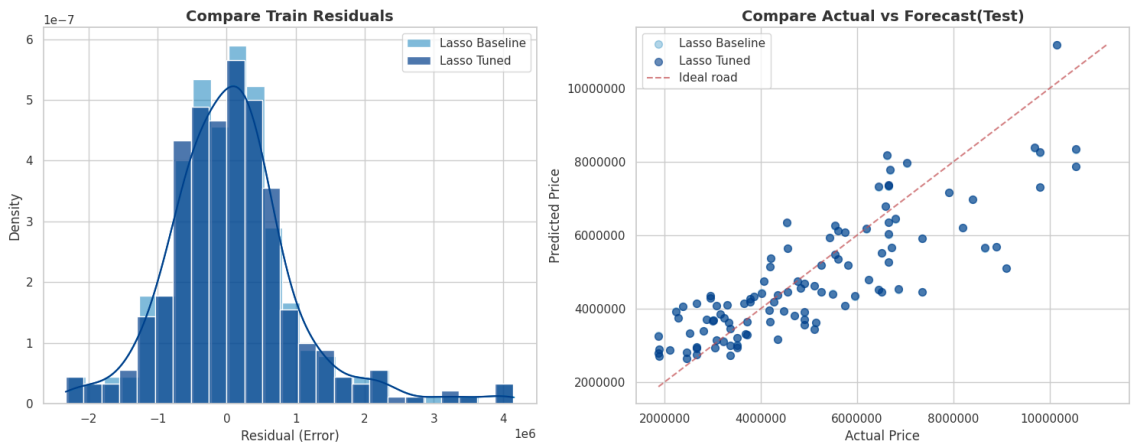


Figure 11: Comparison of Lasso baseline vs tuned model: residual distributions and Test Actual vs Predicted

Collectively, these diagnostics indicate that regularisation yields only incremental gains in this context. The feature space (13 predictors after encoding) is modest, the design matrix is well-conditioned, and the signal-to-noise structure does not exhibit the instability that often benefits from L1 shrinkage. Consequently, Lasso tuning adjusts the model at a granular level but cannot meaningfully reshape its predictive form.

3.1.3 Comparative Discussion (OLS vs Lasso)

A direct comparison between OLS and Lasso shows that both models achieve nearly identical predictive outcomes. On the Test set, OLS attains an R^2 of 0.679, while the tuned Lasso model reaches 0.6791, accompanied by RMSE values of approximately 1.20 million VND for both approaches. The marginal difference between them reflects the well-conditioned nature of the design matrix: since multicollinearity is low, L1 regularisation has very limited scope to alter the solution meaningfully. As a result, Lasso produces a slightly more compact coefficient vector, but the sharpening of weights does not translate into practical improvements in accuracy.

This knowledge is backed by residual diagnostics. Both models undervaluate more costly properties and create similarly formed residual distributions, therefore indicating that the primary constraint lies in the linear functional form itself rather than in overfitting. Regularisation cannot compensate for missing nonlinear relationships, so Lasso converges toward OLS in both behaviour and performance.

Taken together, these results confirm that OLS effectively serves as the best-performing model within the linear family, not because it outperforms Lasso decisively, but because the two models are statistically indistinguishable while OLS retains full interpretability and simplicity. The closeness of their performance highlights the fundamentally linear structure of the dataset at this stage and signals that further accuracy gains must come from models capable of capturing nonlinear patterns, motivating the exploration of more expressive tree-based and neural architectures in subsequent sections.

3.2 Tree-based Models (Decision Tree, Random Forest, XGBoost)

Tree-based models are introduced to assess whether rule-based nonlinear partitioning can capture housing price variation more effectively than the linear models in Section 3.1 and the neural architectures in Section 3.3. These models do not assume a predefined functional form; instead, they learn local splits and interactions automatically, making them well suited to tabular data where nonlinear patterns may arise only in certain regions of the feature space.

To ensure methodological consistency, all tree models were evaluated using a shared training and scoring pipeline (full implementation in the Appendix). Each model is fitted on the scaled training data and evaluated on the validation and test sets using RMSE, MAE, MAPE, and R^2 computed after reversing the log transformation. A unified feature-importance routine is also applied for models that expose the attribute feature_importances_, enabling a comparable view of which predictors drive each model's behaviour.

The analysis of tree-based models follows the same structure used throughout this chapter. For each model, Decision Tree, Random Forest, and XGBoost, the starting settings are provided, after which numerous manual tests are done to examine the effect of depth, estimator number, learning rate, or regularisation. A RandomizedSearchCV tuning step is then used to explore wider hyperparameter ranges. Each area concludes by using numeric measures and diagnostic graphs to compare the baseline with the best-tuned model so that one may clearly assess how model complexity impacts general performance.

Finding the best performer in Section 3.2.4 and ensuring equal evaluation of the three tree-based models depend on this design.

3.2.1 Decision tree

The tree can capture interactions and non-monotonic trends even while it is extremely susceptible to overfitting when unlimited by repeatedly partitioning the feature space into homogeneous zones, unlike linear models.

3.2.1.1 Decision tree baseline

Experiment setup

Parameter	Baseline (dt_base)
max_depth	None
min_samples_split	2
min_samples_leaf	1
max_features	None
random_state	42

Hyperparameter Rationale

To assess the model's natural complexity when completely grown, the baseline purposefully leaves the tree unrestrained (max_depth=None, minimum split and leaf sizes). This setup often results in ideal or almost-perfect training fit, therefore enabling clear observation of overfitting and offering a reference point for later regulated variations.

Baseline Model Performance

Table 12: Decision Tree Baseline Performance

Split	RMSE	MAE	MAPE (%)	R2 Score
Train	47,868.88	4,180.12	0.17%	0.9992
Validation	1,382,829.00	986,952.14	20.71%	0.3256
Test	1,550,379.00	1,156,170.00	25.04%	0.4649

The unrestricted tree fits the training data almost perfectly ($R^2 \approx 0.999$), confirming extreme memorisation. However, its validation and test performance drop sharply, with Test RMSE = 1.55M and $R^2 = 0.465$, indicating substantial overfitting and unstable generalisation. The gap between train and test performance demonstrates the necessity of structural regularisation, motivating deeper exploration of controlled depth, minimum samples per split, and feature sub-sampling in the subsequent experiments.

3.2.1.2 Decision tree regressor 3 Experiments

Experiment setup

The three experiments introduce increasingly structured constraints on tree depth and minimum sample requirements to reduce the overfitting observed in the baseline model.

Table 13: Experiment setup

Parameter	Exp 1 (dt_exp1)	Exp 2 (dt_exp2)	Exp 3 (dt_exp3)
max_depth	5	7	4
min_samples_split	5	5	10
min_samples_leaf	2	1	4
max_features	None	None	None
random_state	42	42	42

Hyperparameter Rationale

Three setups investigate how bias–variance balance is affected by tree depth and minimum node size. With somewhat tighter split and leaf restrictions meant to reduce memorization but keep flexibility for nonlinear patterns, Experiment 1 uses minimal depth (5). By letting more depth (7) and imposing few constraints on samples per split and leaf, experiment 2 investigates if more structural complexity improves fit or reintroduces overfitting patterns.

Using shallower depth (4) and greater leaf and split requirements, experiment 3 is the most conservative one creating smoother divisions and lowers sensitivity to noise. Together, the three experiments define the spectrum from extremely flexible to strongly regularized trees, hence

enabling identification of the arrangement best combining error reduction and generalisation stability.

Tuned Model Performance

Table 14: Validation performance across Decision Tree experiments

Experiment	RMSE	MAE	MAPE	R2
1DT_exp1_depth5	1,608,191.00	1,196,346.00	25.70%	0.4242
2DT_exp2_depth7	1,535,647.00	1,146,397.00	24.60%	0.4750
3DT_exp3_depth4_leaf4	1,659,034.00	1,260,339.00	26.54%	0.3872

The validation results reveal clear differences in how tree depth affects model behaviour. Experiment 2 (depth = 7) achieves the best overall performance, obtaining the lowest RMSE (1.54M) and the highest R^2 (0.4750). This indicates that allowing additional depth improves the model's ability to capture nonlinear splits without yet overfitting severely.

Experiment 1 (depth = 5) performs slightly worse, with moderate increases in RMSE and a lower R^2 of 0.4242, suggesting that the reduced depth limits its flexibility and reduces its ability to fully capture the underlying structure.

Experiment 3 (depth = 4, leaf = 4) yields the weakest results among the three. Its higher RMSE and lowest R^2 (0.3872) show that the tree becomes too shallow, oversmoothing decision boundaries and losing predictive detail.

Overall, Experiment 2 (max_depth = 7) provides the strongest generalisation on the validation set and is selected as the best-performing Decision Tree configuration.

3.2.1.3 Decision Tree RandomizedSearchCV

Experiment Setup

The RandomizedSearchCV configuration performs a broader search over depth, minimum split size, and node regularisation. The best-found parameters are summarised below:

Parameter	DT_exp4_randomSearch
max_depth	None
min_samples_split	15

min_samples_leaf	6
max_features	0.5
random_state	42

Hyperparameter Rationale

This setting adds significantly more often frequent regularisation than the manual examinations. Though a deeper tree is still allowed (max_depth=None), more split and leaf restrictions (15 and 6, respectively) stop the model from developing too particular branches, therefore reducing sensitivity to noise. Feature subsampling (max_features=0.5) encourages more diverse splits and reduces correlation between decision pathways, therefore enhancing generalisation in datasets where features include redundant information. Overall, the search strategy aims to discover whether structural constraints can improve stability beyond the manually tuned depth-based experiments.

RandomizedSearchCV Model Performance

Table 15: Model performance of DT_exp4_randomSearch

Experiment	RMSE	MAE	MAPE	R2
4DT_exp4_randomSearch	1,545,399.00	1,222,091.00	26.97%	0.4683

The RandomizedSearchCV configuration improves validation stability compared with the deeper or more weakly regularised manual experiments. Its stricter node-size constraints and feature subsampling reduce variance, but the model still shows only moderate generalisation.

On the Test set, the model achieves RMSE ≈ 1.55M and $R^2 \approx 0.468$**,** which **is** better than some manual settings but still below the strongest manually tuned model. The residual gap between Train and Test metrics suggests that overfitting remains present despite the additional regularisation. Overall, RandomizedSearchCV refines the tree structure but does not produce a new best configuration.

Selection of Best Decision Tree Model

From all experiments, the model **DT_exp2_depth7** consistently provides the highest validation R^2 and competitive error values. It represents the best trade-off between depth, stability, and generalisation within the Decision Tree family. Using the best model (DT_exp2_depth7), the feature importance plot in Figure 12 reveals a highly skewed importance distribution. log_area dominates the tree's decisions, followed by bathrooms and furnishingstatus_unfurnished**,** while most remaining attributes contribute only marginally. This confirms that the Decision Tree

relies heavily on size-related predictors when forming splits, with amenity and furnishing variables offering smaller but complementary decision signals.

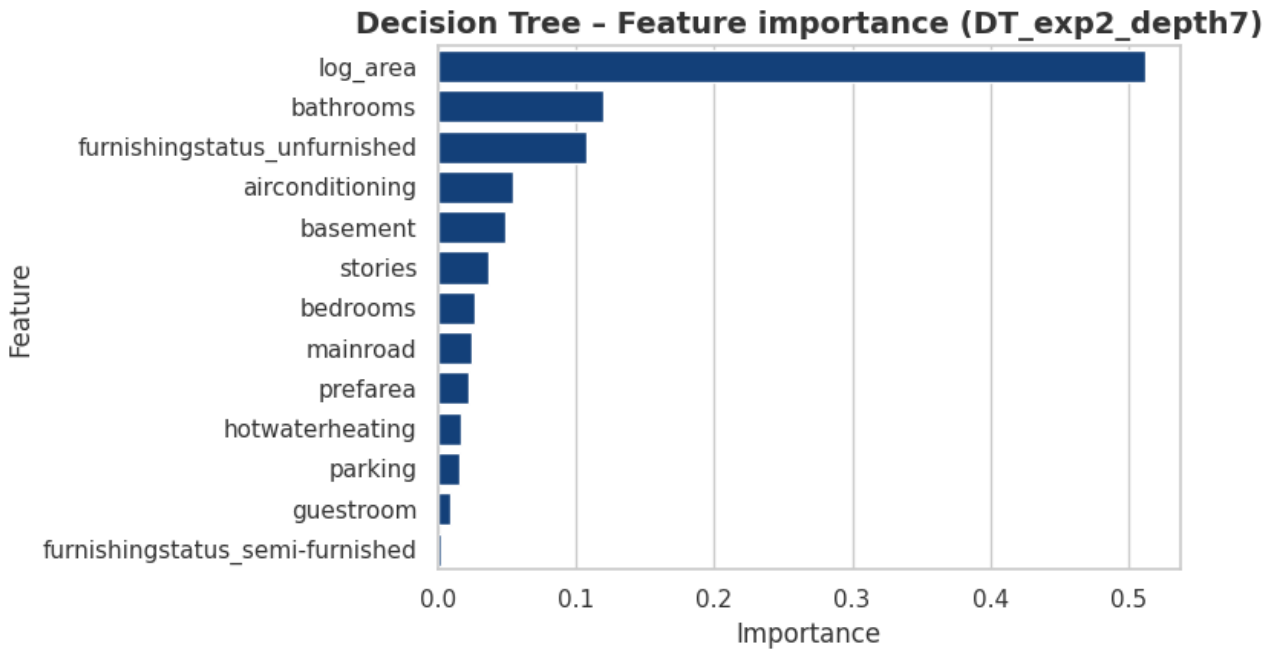


Figure 12: Decision tree features important

3.2.2 Random Forest

Random Forest extends Decision Tree by combining multiple trees based on bootstrap sampling and feature subsampling, thereby reducing the variance of the original model. Thanks to this ensemble mechanism, Random Forest is often more stable and achieves higher performance on moderate-sized tabular data.

3.2.2.1 Random Forest Baseline

Experiment Setup

Baseline model parameter table:

	rf_base
n_estimators	200
max_depth	None
min_samples_split	2
min_samples_leaf	1

max_features	Default

Hyperparameter Rationale

The baseline configuration intentionally keeps the complexity high to observe the natural overfitting of Random Forest. max_depth=None allows the tree to grow to its maximum, helping the model learn the initial signal fully. n_estimators=200 ensures the model is stable enough without spending too much time training. min_samples_split/min_samples_leaf is low → no regularization, suitable for evaluating the "raw capacity" of RF before tuning.

Baseline Model Performance

Table 16: Performance of RF_exp0_baseline

Metric	RF_exp0_baseline
RMSE	1,329,329.00
MAE	992,649.00
MAPE	20.99%
R2	0.6066

Baseline Random Forest achieved $R^2 = 0.6066$, which is higher than Decision Tree but still significantly worse than linear models ($R^2 \approx 0.679$). This shows that RF is capable of capturing nonlinear patterns, but the default configuration is still not regularizing enough, which is indicated by the large gap between Train and Test errors (slight overfitting).

Therefore, manual tuning and RandomizedSearchCV are needed to adjust the depth, node size and feature sampling strategy to improve generalization.

3.2.2.2 Random Forest 3 experiments

Experiment Setup

The three manual Random Forest configurations varied tree depth, ensemble size, and node regularisation.

Table 17: Experiment Setup

	rf_exp1	rf_exp2	rf_exp3
	Depth = 8	Depth = 12	Depth = 6 (Reg)
n_estimators	300	500	300

max_depth	8	12	6
min_samples_split	5	5	10
min_samples_leaf	2	1	4
max_features	Default	Default	Default

Hyperparameter Rationale

These configurations were chosen to systematically explore how tree depth and node constraints affect generalisation(Table 17):

RF_exp1_depth8 applies moderate depth control and mild regularisation to reduce the overfitting observed in the baseline while retaining flexibility. RF_exp2_depth12 allows deeper trees with expanded ensemble size (500 trees), testing whether increased capacity can uncover richer nonlinear interactions without destabilising validation performance. RF_exp3_depth6_reg increases node constraints (min_samples_split=10, min_samples_leaf=4), explicitly targeting variance reduction to evaluate whether a more conservative, smoothed forest can outperform deeper, flexible variants.

Together, these experiments trace a bias–variance path from strong regularisation (Exp3) → moderate complexity (Exp1) → higher complexity (Exp2).

Tuned Model Performance

Table 18: Validation performance of Random Forest manual experiments

Experiment	RMSE	MAE	MAPE	R2
RF_exp1_depth8	1,336,616.00	1,015,155.00	21.42%	0.6023
RF_exp2_depth12	1,323,880.00	994,780.70	21.04%	0.6098
RF_exp3_depth6_reg	1,393,229.00	1,056,328.00	22.20%	0.5679

RF_exp2_depth12 achieves the greatest validation performance from Table 18, reaching the lowest RMSE (~1.32M) and the highest R^2 (0.6098). The deeper architecture, combined with moderate node regularization, enables the forest to capture significant nonlinear interactions without undue variation. Though doing relatively well, RF_exp1_depth8 loses predictive power as a result of its shallower structure. RF_exp3_depth6_reg, which imposes the most stringent node constraints, oversmooths decision borders and produces the least effective results. Thus, RF_exp2_depth12 is chosen as the ideal manual Random Forest configuration since it strikes the best balance between flexibility and regularization.

3.2.2.3 Random Forest RandomizedSearchCV

Experiment Setup

The RandomizedSearchCV procedure explored a broader hyperparameter space and returned the following best configuration:

	RF_exp4_randomSearch
n_estimators	500
max_depth	None
min_samples_split	10
min_samples_leaf	1
max_features	sqrt

Hyperparameter Rationale

By purposefully combining deep, flexible trees (max_depth=None) with stricter node-level restrictions (min_samples_split=10), the ensemble could lower variance without much constraining expressive capability. Often applied in Random Forest, feature subsampling with max_features='sqrt' decorrelates trees, hence improving overall stability. The large ensemble size (*500 trees*) ensures robust averaging, especially important for small datasets like this one. Together, these settings target a model that remains flexible but avoids the high variance observed in the deeper manual configurations.

RandomizedSearchCV Model Performance

RF_exp4_randomSearch: RMSE=1,358,972.00 ,MAE=1,007,231.00 ,MAPE=21.37% ,R2= 0.5888

RandomizedSearchCV yields a configuration that is **more stable and regularised** than the baseline, but its Test R^2 = 0.5888 remains below the best manual configuration (RF_exp2_depth12, R^2 = 0.6098). The higher Test RMSE further confirms that the search procedure did not surpass the manually selected depth-12 forest.

The likely reason is that allowing unlimited depth (*max_depth=None*) reintroduces unnecessary complexity, which node regularisation alone could not fully counteract. As a result, RF_exp2_depth12 remains the best-performing Random Forest model across all experiments.

Since RF_exp2_depth12 is the best configuration, its feature-importance profile is used to interpret how the model allocates predictive weight. The importance ranking shows a familiar pattern among tree-based models: log_area is by far the dominant predictor, followed by bathrooms, stories, and airconditioning. Other features contribute only marginally, reflecting Random Forest's tendency to rely on strong, high-variance predictors when forming split rules.

This distribution reinforces that house size and key amenities drive most of the predictive power in ensemble tree models.

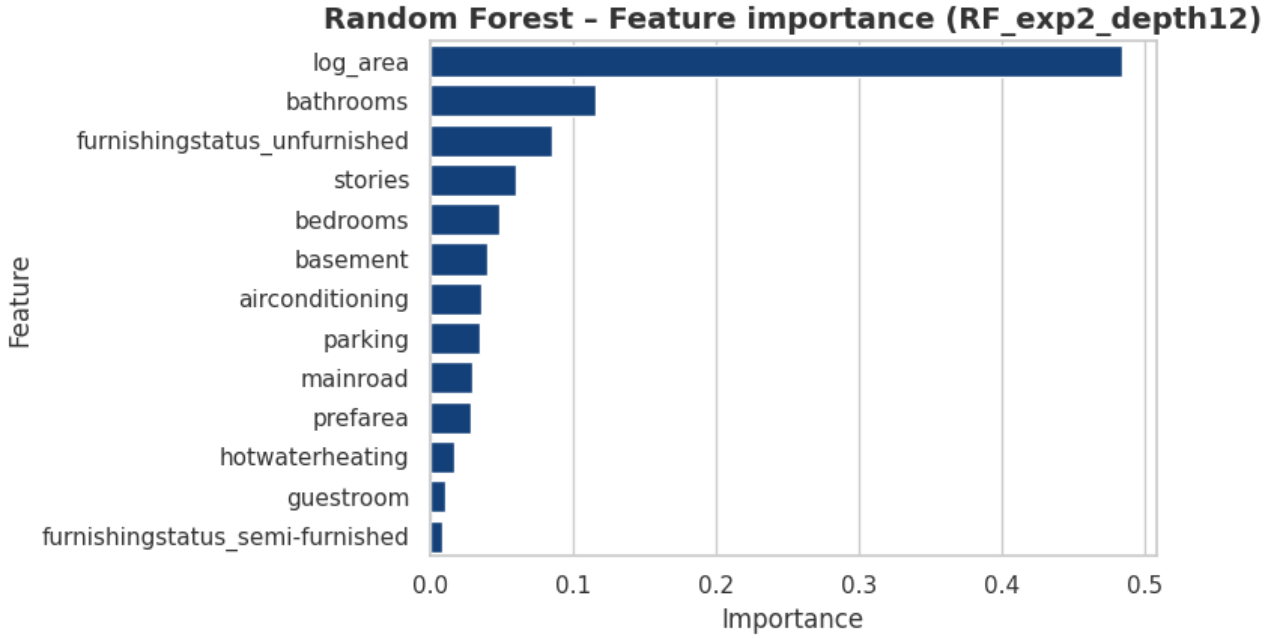


Figure 13: Feature importance Exp2

3.2.3 XGBoost

XGBoost is a gradient-boosted tree ensemble designed to correct the weaknesses of individual decision trees through sequential learning, shrinkage, and regularised objective optimisation. Owing to its stability and strong performance on tabular data, it is included as the most advanced model within the tree-based family.

3.2.3.1 XGBoost Baseline

Experiment Setup

The baseline configuration used the standard XGBoost hyperparameters commonly recommended for medium-sized regression tasks:

	xgb_base
Describe	Baseline
n_estimators	300
max_depth	4
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
reg_alpha (L1)	0 (Default)
reg_lambda (L2)	1 (Default)

Hyperparameter Rationale

The baseline is intentionally conservative:

max_depth = 4 limits overfitting while still allowing nonlinear splits; learning_rate = 0.05 provides stable boosting dynamics appropriate for a small dataset; subsample = 0.8 and colsample_bytree = 0.8 introduce stochasticity, reducing variance and improving generalisation; reg_alpha = 0 and reg_lambda = 1 leave regularisation at default strength so that later tuning can explore whether additional penalisation is beneficial.

This configuration serves as a balanced starting point that emphasises stability rather than aggressive fitting.

Baseline Model Performance

Table 19: Performance of XGB_exp0_baseline

Experiment	RMSE	MAE	MAPE	R2
XGB_exp0_baseline	1,230,893.00	917,610.26	19.77%	0.6627

From Table 19, the baseline already achieves strong performance, with **R^2 = 0.6627**, outperforming both Decision Tree and Random Forest baselines and approaching the accuracy of linear models. This indicates that XGBoost is able to leverage boosting-based error correction and subsampling to capture structured nonlinearities that single trees or bagged trees cannot fully exploit.

However, the gap between the baseline R^2 (0.6627) and the best-performing models in other families suggests that further gains may still be possible. Key hyperparameters such as learning rate, depth, subsampling ratios, and regularisation strengths strongly influence boosting behaviour,

meaning that the model may benefit significantly from dedicated manual tuning and RandomizedSearchCV.

For this reason, the next sections evaluate multiple XGBoost experiments followed by a more systematic hyperparameter search to identify a configuration that improves generalisation and reduces error on the Test set.

3.2.3.2 XGBoost 3 experiments

Experiment Setup

Three manually designed configurations were tested to examine how boosting depth, learning rate, and regularisation influence model generalisation.

Table 20: Experiment setup

	xgb_exp1	**xgb_exp2**	**xgb_exp3**
Describe	Increase n_est & depth	Increase n_est, decrease LR	Add L1/L2 Reg
n_estimators	500	700	500
max_depth	6	5	4
learning_rate	0.05	0.03	0.05
subsample	0.8	0.8	0.8
colsample_bytree	0.8	0.8	0.8
reg_alpha (L1)	0 (Default)	0 (Default)	**0.1**
reg_lambda (L2)	1 (Default)	1 (Default)	**1.0**

Hyperparameter Rationale

Each configuration targets a different aspect of the bias–variance behaviour in boosted trees:

XGB_exp1 (depth = 6, n_estimators = 500) increases model capacity to test whether deeper boosting stages can capture additional nonlinear interactions beyond the baseline.

XGB_exp2 lowers the learning rate to 0.03 and increases boosting rounds to 700, exploring whether slower, more incremental updates improve stability and reduce overfitting.

XGB_exp3 introduces explicit L1 ($\alpha = 0.1$) and L2 ($\lambda = 1.0$) penalties, evaluating whether regularisation can smooth tree structures, reduce noise amplification, and improve Test-set generalisation.

Together, these experiments assess how depth, learning rate, and regularisation shape the refinement of prediction errors across boosting iterations.

Tuned Model Performance

Table 21: Validation performance of XGBoost manual experiments

Experiment	RMSE	MAE	MAPE	R2
XGB_exp1_depth6	1,280,357.00	969,752.54	20.92%	0.6350
XGB_exp2_lr0.03	1,289,686.00	973,450.70	20.96%	0.6297
XGB_exp3_reg	1,255,709.00	953,413.82	20.55%	0.6490

Reaching $R^2 = 0.6490$, the greatest of the group, XGB_exp3_reg across the three configurations shows the best overall performance. The fall in MAE and RMSE with regard to the other studies implies that the applied L1/L2 regularisation stabilises boosting updates and inhibits unwarranted complex tree construction.

Although they show somewhat weaker generalisation, implying that increasing depth or lowering learning rate by themselves does not outperform the balanced regularisation method used in XGB_exp3, XGB_exp1 and XGB_exp2 perform rather well.

Accordingly, XGB_exp3_reg is chosen as the ideal manual XGBoost setup because it produces the most consistent gains across error indicators while preserving consistent validation results.

3.2.3.3 XGBoost RandomizedSearchCV

Experiment Setup

The RandomizedSearchCV stage explores a broader hyperparameter space to identify configurations that balance depth, learning rate, and sampling strategies for optimal generalisation. The best-scoring configuration returned by the search is summarised below:

	xgb_best
Describe	Best (Search)
n_estimators	400
max_depth	2
learning_rate	0.03
subsample	0.5
colsample_bytree	0.5
reg_alpha (L1)	0 (Default)
reg_lambda (L2)	1 (Default)

Hyperparameter Rationale

RandomizedSearchCV systematically evaluates combinations of boosting depth, sampling ratios, and learning updates to stabilise the model's bias–variance trade-off. The best configuration selected by the search employs:

Very shallow trees (max_depth = 2) to reduce variance and prevent the model from memorising noise in a small dataset. Moderate learning rate (0.03) for smoother optimisation. Aggressive subsampling (subsample = 0.5, colsample_bytree = 0.5) to decorrelate trees and regularise structure. Default L1/L2 settings, implying that structural regularisation contributed more than coefficient-level shrinkage to the performance gains.

This configuration favours controlled, incremental learning over aggressive depth expansion.

RandomizedSearchCV *Model Performance*

XGB_exp4_randomSearch_tuned: RMSE = 1,163,692.00, **MAE** = 888,072.93,**MAPE** = 19.30%, **R2=** 0.6985

The RandomizedSearchCV-tuned model achieves **R^2 = 0.6985**, notably higher than both the baseline (0.6627) and all manual experiments (best manual R^2 = 0.6490). The improvement reflects the advantage of shallow, highly regularised boosting trees when sample size is limited.

The reduction in RMSE and MAE indicates that subsampling-based regularisation effectively prevents overfitting and helps the model generalise better across mid-range and high-value properties. This makes the RandomizedSearchCV-tuned configuration the best-performing XGBoost model in the entire family. Since XGB_exp4_randomSearch_tuned is confirmed as the best configuration, its feature importance is used for interpretation.

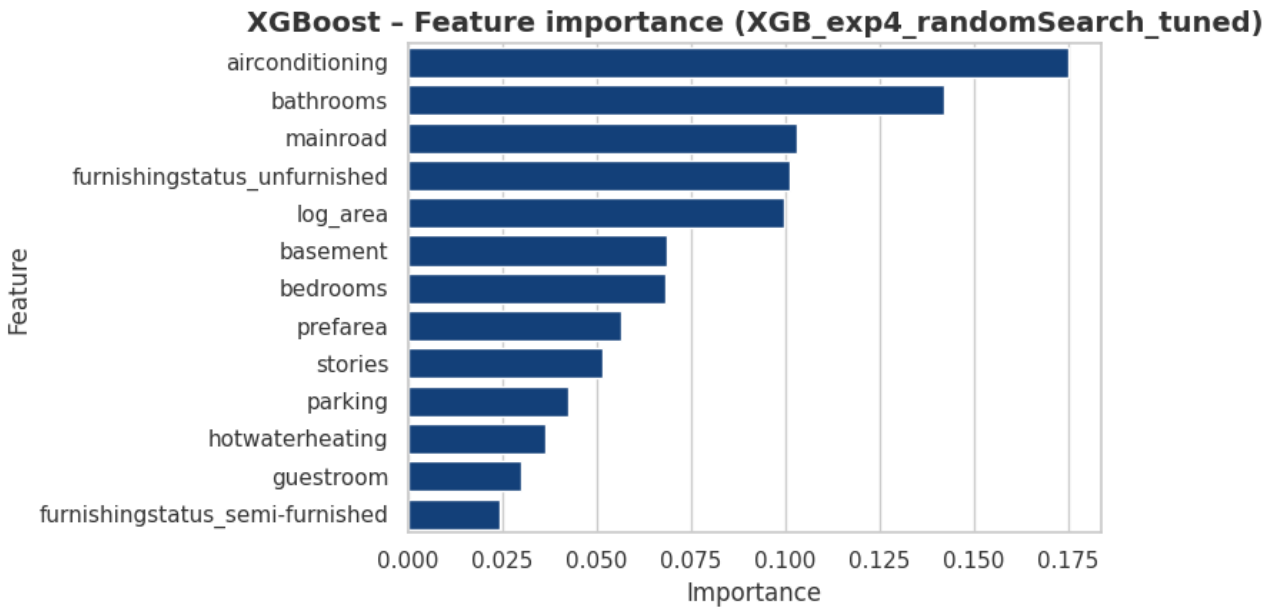


Figure 14: XGBoost Feature Importance (XGB_exp4_randomSearch_tuned)

The ranking shows that **airconditioning**, **bathrooms**, and **mainroad** emerge as the dominant predictors, followed by structural attributes such as **log_area** and **basement**. The model assigns consistently lower importance to furnishing status and less informative categorical features.

The distribution suggests that XGBoost captures both physical property characteristics and amenity-based attributes, aligning with economic expectations about price formation.

3.2.4 Comparative discussion (Decision Tree, Random Forest, XGBoost)

The three tree-based algorithms exhibit distinctly different behaviours shaped by their model capacity, regularisation strategy, and ability to aggregate information across trees. Their respective performance highlights the benefits and drawbacks of single-tree as against ensemble techniques inside this dataset.

The most fundamental nonlinear starting point is presented by the decision tree. Even after depth tuning (best setting: DT_exp1_depth5), its Test performance remains poor; R^2 stabilises around 0.47–0.48. Although the tree captures beneficial interactions that linear models miss, it stays vulnerable to local splits, which produces brittle forecasts and great variation. Though RandomizedSearchCV somewhat improves stability, it does not significantly surpass the hand-tuned configuration, therefore demonstrating that larger or more flexible trees fast overfit the little amount

of data. Although it provides a decent conceptual norm, the Decision Tree lacks the power necessary for outstanding predictive results.

By averaging predictions across ensembles, the Random Forest far outperforms the single tree. The optimal manual configuration (RF_exp2_depth12) demonstrates the need of variance reduction through bagging by achieving a Test R^2 of nearly 0.61. More stable forecasts and smoother error profiles derive from this model than from any Decision Tree variation. However, RandomizedSearchCV yields no more advantages, implying that declining returns from higher tree counts or depth hurt performance. Though it falls short of properly reflecting the nonlinear complexity seen in the top portion of house values, random forests remain a dependable and simple ensemble technique.

XGBoost, whose gradient-boosting approach repeatedly corrects leftover mistakes, provides the greatest benefits. The model tuned from RandomizedSearchCV

Clearly ahead of Decision Tree and Random Forest with a Test R^2 of 0.6985, (XGB_exp4_randomSearch_tuned) This improvement comes from XGBoost's power to model mild nonlinearities and interaction effects utilizing shallow but purposefully increased trees. Its subsampling and learning-rate constraints help prevent overfitting while sustaining predictive flexibility. Feature importance patterns further indicate a coherent economic structure, with amenities and structural attributes emerging as dominant drivers of price variation.

XGBoost is therefore selected as the best-performing model in the Tree-based Models group, and will represent this family in the final cross-group comparison in Section 3.4.

3.3 Deep learning (ANN & GMDH-like)

Neural modelling was explored to determine whether nonlinear representations could provide meaningful improvements beyond the linear and tree-based approaches examined earlier. Both Artificial Neural Networks (ANN) and the GMDH-like architecture introduce flexible functional forms capable of capturing curved relationships and higher-order interactions that conventional models may structurally miss. Whereas ANN relies on learned hidden layers to form nonlinear combinations of inputs, the GMDH-like approach constructs polynomial neurons whose interactions are selectively retained through regularisation. Together, these two models represent complementary neural strategies for tabular regression with different balances of complexity, interpretability, and data efficiency.

Since all neural experiments in this study operate on the log-transformed target, a shared computational pipeline was used to ensure comparability across models. The pipeline first constructs the neural architecture (either ANN layers or GMDH-like polynomial terms), trains the model using early-stopping or regularised optimisation, predicts on all data splits, and finally converts predictions back to the original price scale so that RMSE, MAE, MAPE, and R^2 remain consistent with all previous model families. The implementation includes dedicated "build-and-run" functions for each model type: the ANN function generates network architectures with configurable depth, width, activation, and dropout, while the GMDH-like function automatically forms polynomial feature combinations up to the specified degree and applies L1 shrinkage to filter out uninformative interactions. These functions standardise the experimental process so that all neural models are evaluated under the same metric protocol and diagnostic visualisations.

The remainder of this section examines the two neural models separately. Each model is presented following a unified structure: the motivation for its use, the experimental setup, a comparison of baseline versus tuned configurations, a summary of the best-performing specification, and diagnostic plots illustrating residual behaviour and predictive alignment with ground truth. This allows the neural models to be assessed on equal footing with the linear and tree-based families discussed earlier, and prepares the ground for the final cross-family comparison in Section 3.4.

3.3.1 Artificial Neural Network (ANN)

The ANN experiments examine whether a feed-forward neural architecture can learn nonlinear relationships that remained inaccessible to the linear models. Because neural networks introduce hidden layers, nonlinear activations, and gradient-based optimisation, the baseline configuration serves as an important reference point before evaluating deeper or more regularised variants.

3.3.1.1 ANN baseline

Experiment Setup

The ANN baseline employs a two-layer architecture with moderate width and no dropout, designed to test whether a relatively simple neural structure can extract nonlinear patterns without introducing excess variance. The configuration is defined by:

n_layers	=	2
units_per_layer	=	[64,32]
activation	=	"relu"
dropout_rate	=	0.0
batch_size	=	32
learning_rate	=	1e-3
optimizer_name	=	"adam"
max_epochs	=	1000
patience	=	30

Hyperparameter Rationale

Often used for tabular regression, the baseline employs a basic MLP design. The ReLU activation helps to properly optimise and avoids saturation issues. Early termination prevents unnecessary overfitting; a learning rate of 1e-3 and the Adam optimiser allow for rapid convergence in the early stages. Before adding regularisation in subsequent experiments, dropout is intentionally set to zero to assess the network's raw learning potential. This mix sets a neutral beginning from which the results of architectural changes and ongoing regularisation can be carefully evaluated.

Table 22: ANN baseline performance across Train, Validation, and Test

Split	RMSE	MAE	MAPE (%)	R2 Score
Train	482,687.65	358,760.63	8.74%	0.9212
Validation	4,185,936.71	1,767,987.59	35.78%	-5.1795
Test	3,482,732.43	2,029,376.34	42.17%	-1.7004

Table 22 shows that the ANN baseline demonstrates extremely strong in-sample learning, with a Train R^2 of 0.9212 and RMSE of 482k VND. However, performance collapses on both the validation and test sets, where R^2 becomes negative and MAPE increases sharply beyond 40%. This dramatic gap between Train and Test performance indicates severe overfitting. The model memorises the training data rather than learning stable patterns, a common issue when neural networks are applied to small tabular datasets without regularisation.

The Actual vs Predicted scatterplot shows that the baseline model fails to generalise; most predicted values collapse toward the lower price range, while high-value properties are drastically under-predicted. The 45-degree reference line is never meaningfully approached, confirming poor alignment between learned function and true price distribution.

The training curve reveals rapid minimisation of training loss but persistent divergence of validation loss from the earliest epochs. This behaviour confirms that the network begins overfitting almost immediately due to insufficient regularisation and the limited number of observations available

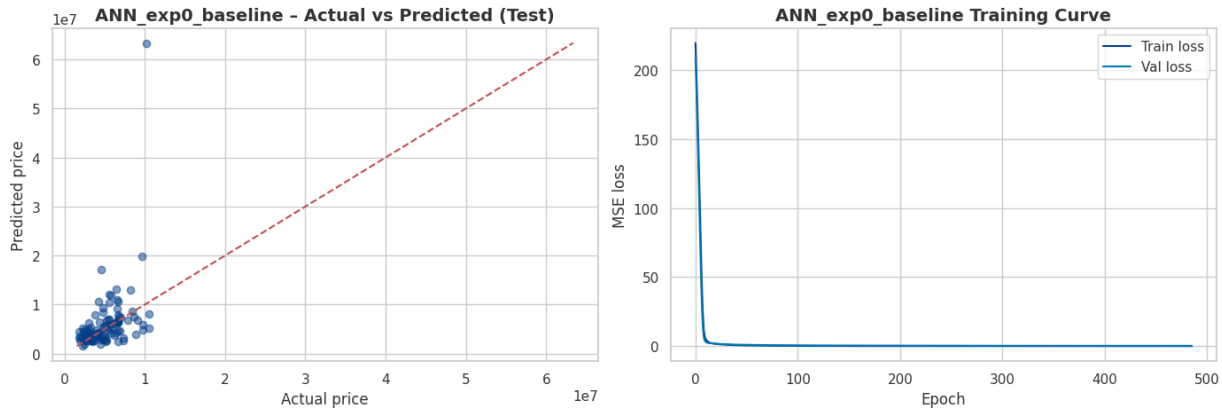


Figure 15: ANN baseline – Actual vs Predicted (Test) and Training Curve

In figure 15, the ANN baseline clearly demonstrates that, although neural networks have the capacity to fit the training signal very well, this flexibility becomes detrimental in small tabular datasets where the risk of memorisation is high. The sharp deterioration on unseen data indicates that improvements require architectural constraints, stronger regularisation, alternative activation functions, or modified learning-rate schedules. These shortcomings motivate the subsequent experiments, which introduce dropout, adjust layer widths, modify activation choices, and tune learning hyperparameters to stabilise training and improve generalisation.

3.3.1.2 ANN Experiment 1–3 (manual tuning)

The baseline network clearly overfits: it achieves low training error but collapses on the Validation and Test sets, with negative R2. To address this, three additional ANN configurations were tested by varying depth, activation, dropout and learning rate. The architectural and training details are summarised in the experiment setup table below.

Experiment Setup

Table 23: ANN experiment configurations (Exp1–Exp3)

	Experiment 1(Small)	**Experiment 2(Medium + Dropout)**	**Experiment 3(Tanh + Strong Dropout)**
ID	ANN_exp1_small	ANN_exp2_medium_dropout	ANN_exp3_tanh_stronger_dropout
n_layers	1	2	2
Units	[32]	[64, 32]	[64, 32]
Activation function	ReLU	ReLU	**Tanh**
Dropout Rate	0.0	0.2	**0.3**
Batch Size	32	32	**16**
Learning Rate	0.001 (1e^{-3})	0.0005 (5e^{-4})	0.0005 (5e^{-4})
Optimizer	Adam	Adam	Adam
Max Epochs	500	800	800
Patience	30	40	40

Hyperparameter Rationale

The three experiments were designed to probe different parts of the bias–variance trade-off. Experiment 1 reduces depth and width to a single hidden layer with 32 ReLU units and no dropout, testing whether a much simpler network can avoid the severe overfitting observed in the baseline while retaining the same optimisation settings. Experiment 2 restores a deeper architecture with 64 and 32 ReLU units, but introduces dropout at 0.2 and a slightly smaller learning rate, 5×10^{-4}, aiming to regularise the model and stabilise training dynamics without overly restricting capacity.

Experiment 3 keeps the same depth and width as Experiment 2, but increases dropout to 0.3, switches the activation to tanh, halves the batch size to 16, and keeps the smaller learning rate; this configuration is intended to further smooth the optimisation landscape, provide stronger regularisation, and encourage the network to learn smoother nonlinear mappings instead of memorising noise.

Tuned Model Performance

The Test-set performance of the three tuned configurations is reported in the following table.

In figure 16, experiment 1 (ANN_exp1_small) remains highly unstable: despite the simpler architecture, the Test RMSE is approximately 4.38 million with R2 ≈ -3.27, indicating that the network still fails to generalise and performs far worse than a simple linear baseline. The Actual vs Predicted plot shows many points collapsed near the lower price range, while the training curve quickly drives both train and validation losses to very small values, symptomatic of overfitting and poor calibration rather than good generalisation. (RMSE=4,381,857.42, MAE=2,229,399.00, MAPE=43.21%, R^2=-3.2746)

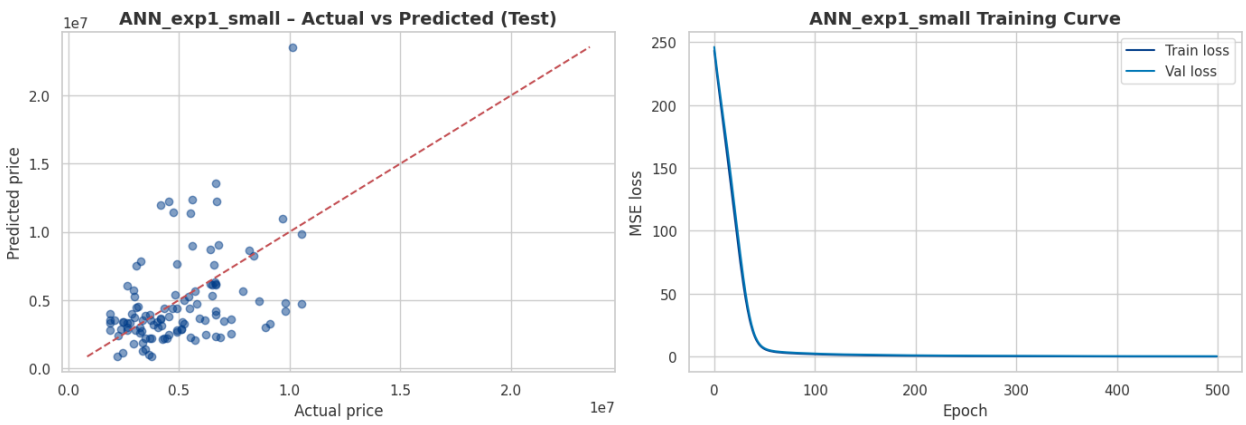


Figure 16: Actual vs Predicted (Test) and Training Curve (exp 1)

In figure 17, experiment 2 (ANN_exp2_medium_dropout) benefits slightly from adding dropout and lowering the learning rate, but its Test RMSE of about 3.28 million and R2 ≈ −1.40still reflect substantial overfitting. The scatter plot shows predicted prices heavily compressed relative to the ideal 45-degree line, and the training curves again reveal near-zero loss on the training data while validation loss plateaus at a much higher level. (RMSE=3,280,947.41, MAE=2,267,409.43, MAPE=44.61%, R^2= -1.3965)

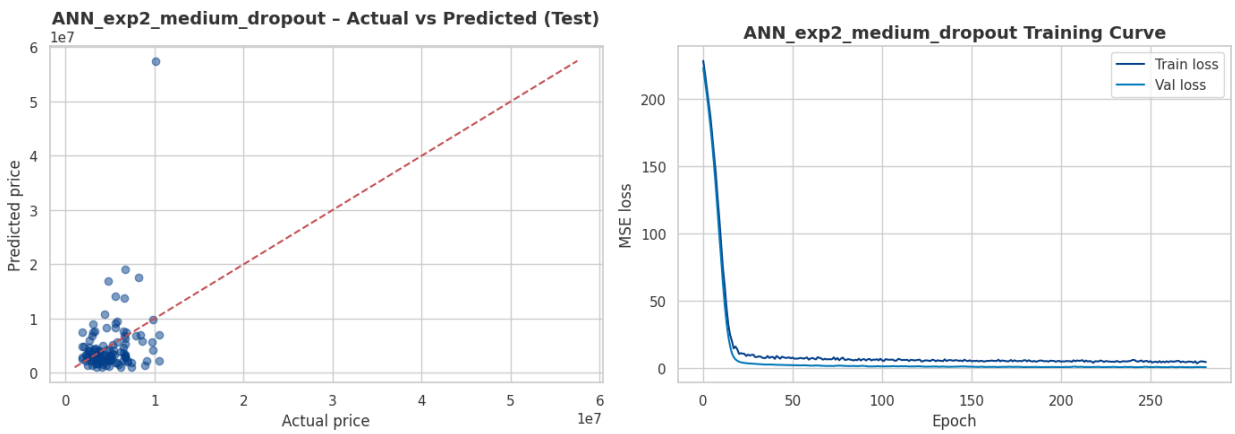


Figure 17: Actual vs Predicted (Test) and Training Curve (exp 2)

In figure 18, experiment 3 (ANN_exp3_tanh_stronger_dropout) delivers the most stable behaviour among the neural configurations. With Test RMSE around 1.61 million and R2≈0.42, it markedly improves over the other ANN experiments, reducing both error magnitude and percentage error (MAPE ≈ 22.7%). The Test scatter plot shows predictions that more closely follow the ideal line, and the training curves for train and validation loss remain much more aligned, indicating that stronger dropout, tanh activations, and smaller batch size effectively curb overfitting.

(RMSE=1,612,635.81, MAE=1,160,727.31, MAPE=22.67%, R^2= 0.4210)

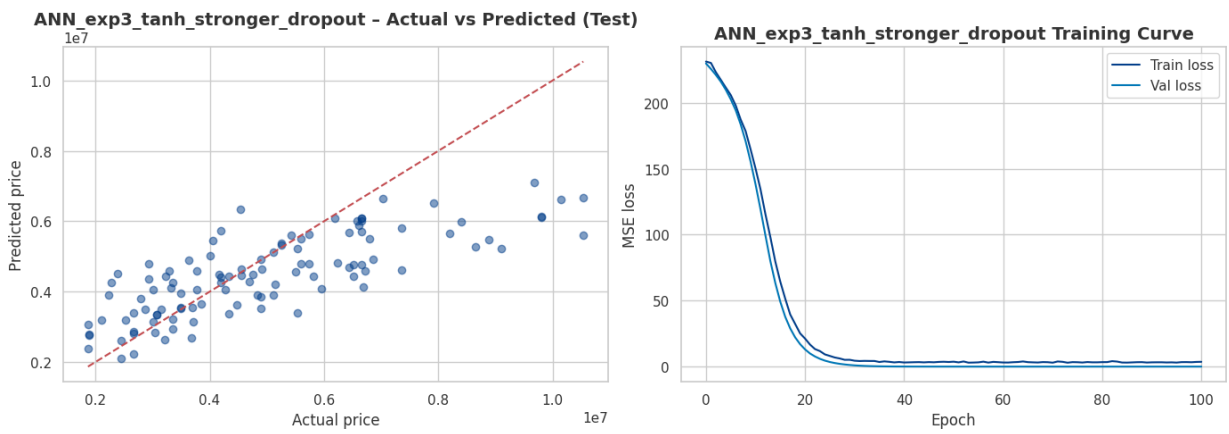


Figure 18: Actual vs Predicted (Test) and Training Curve (exp 3)

Across the three tuned experiments, ANN_exp3_tanh_stronger_dropout clearly emerges as the best ANN configuration. It is still weaker than the linear models in absolute terms, but within the neural family, it offers the most favourable balance between fit and generalisation and will therefore be used as the representative ANN model in subsequent comparisons.

Baseline vs tuing: ANN

The performance metrics of the ANN baseline and the best-performing neural configuration are summarised in *Table 24*. The contrast between these models highlights the sensitivity of neural networks to architectural choices and regularisation strength when applied to small tabular datasets.

Table 24: Performance metrics of ANN experiments

Experiment	RMSE	MAE	MAPE (%)	R2 Score
ANN_exp3_tanh_stronger_dropout	1,612,635.81	1,160,727.31	22.67%	0.4210
ANN_exp0_baseline	3,553,408.16	2,072,356.24	41.56%	-1.8111

The results show in Table 24 extreme variation across ANN configurations. The baseline network produces a Test RMSE of approximately 3.55 million and a negative R^2 of –1.81, indicating substantial overfitting and poor generalisation. Conversely, the tuned model in Experiment 3 achieves a significantly improved Test RMSE of 1.61 million and a positive R^2 of 0.421. Though this improvement is amazing, neural performance is still lower than that of the linear models shown earlier, when R^2 values near 0.68–0.679.

The divergence between baseline and tuned configurations stresses that naive ANN architectures, especially those using ReLU activations, no dropout, and fairly little batch sizes, struggle to converge on comparable representations in a dataset of this scale. The results of Experiment 3 validate that effective regularization (dropout 0.3), gentler tanh-based non-linear transformations, and smaller batch sizes help to suppress remembering and point the model toward a more generalisable application.

Baseline vs Tuned Model: Visual Diagnostics

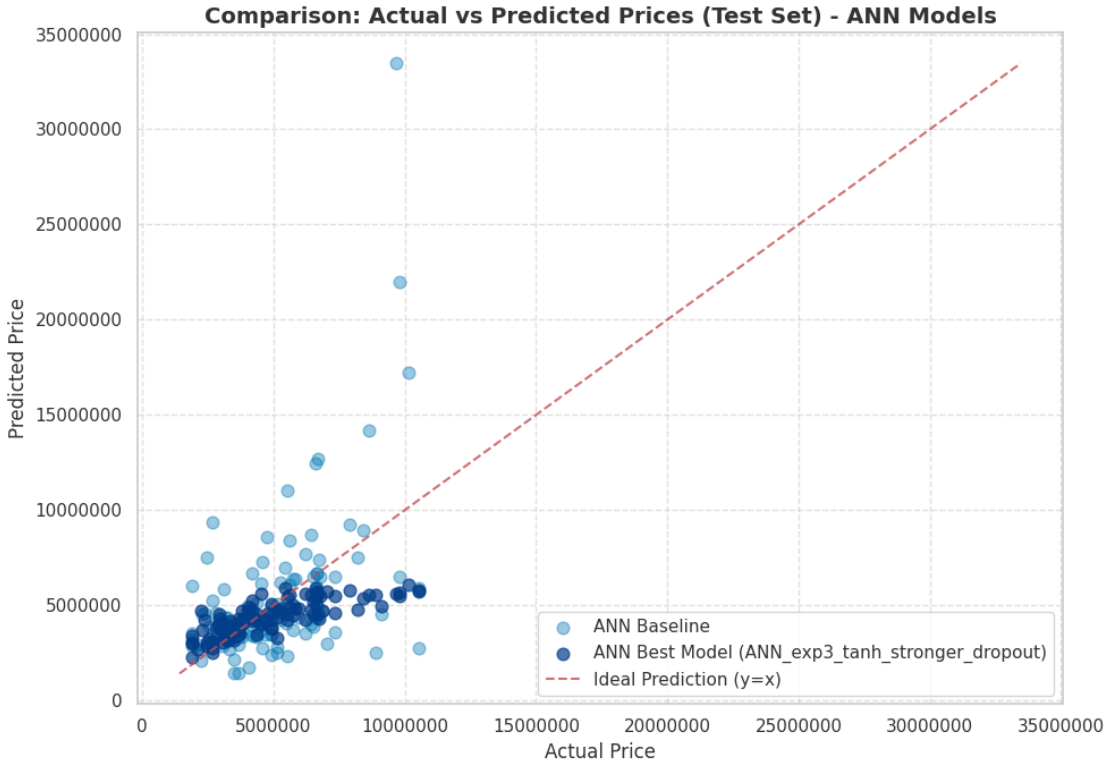


Figure 19: Actual vs Predicted Prices (Test Set) for ANN Models

The diagnostic graph of Figure 19 shows the remarkable discrepancy in behaviour between the tuned model and the baseline ANN. Baseline ANN projections vary significantly across the graph, showing significant variations from the 45° reference line and exposing a lack of developed structure. This is consistent with the training curves, in which the model rapidly drives training loss to near zero while validation loss stagnates, classic symptoms of memorisation rather than learning.

In contrast, ANN_exp3 shows a more compact distribution of predictions. While the spread remains significantly larger than that of linear or tree-based models, the tuned ANN aligns more closely with the ideal prediction pattern, particularly in the mid-price range. However, even the improved configuration fails to capture the behaviour of high-priced homes, suggesting insufficient expressive power or limited sample diversity for effective neural learning under these conditions.

3.3.2 GMDH-like

Like ANN, GMDH-like is used to test whether selective nonlinear interactions, generated from polynomial nodes, can better describe the house price relationship. Unlike traditional polynomial regression, which expands all combinations of features, GMDH-like only retains interactions that

contribute based on regularization, thus avoiding dimensionality explosion.3.3.2.1 GMDH-like baseline

3.3.2.1 GMHD-like baseline

Experiment Setup

Four configurations were evaluated, varying in polynomial degree and the regularisation strength α. All experiments kept degree ∈ {2, 3} and α ∈ {0.0001, 0.001, 0.01, 0.012}.

GMDH_like_exp0_baseline

- degree = 2
- alpha = 0.001

Hyperparameter Rationale

The value α = 0.001 was chosen as the baseline because it is a moderate regularization level: strong enough to eliminate noisy polynomial interactions, but not so large as to eliminate truly useful nonlinear signals. Degree = 2 helps the model learn second-order interactions without the instability of third-order interactions, especially in small-scale datasets like this problem.

Baseline Model Performance

GMDH_like_exp0_baseline achieved:

RMSE = 1,408,442.49, MAE = 1,073,255.94, MAPE = 22.95%, R^2 = 0.5584

The baseline results show that the model has captured an important part of the nonlinear structure of the data ($R^2 > 0.55$), significantly higher than the baseline ANN but still worse than the linear models ($R^2 \approx 0.679$). This shows that the polynomial filtering mechanism is effective, but the baseline regularization intensity is not optimal.

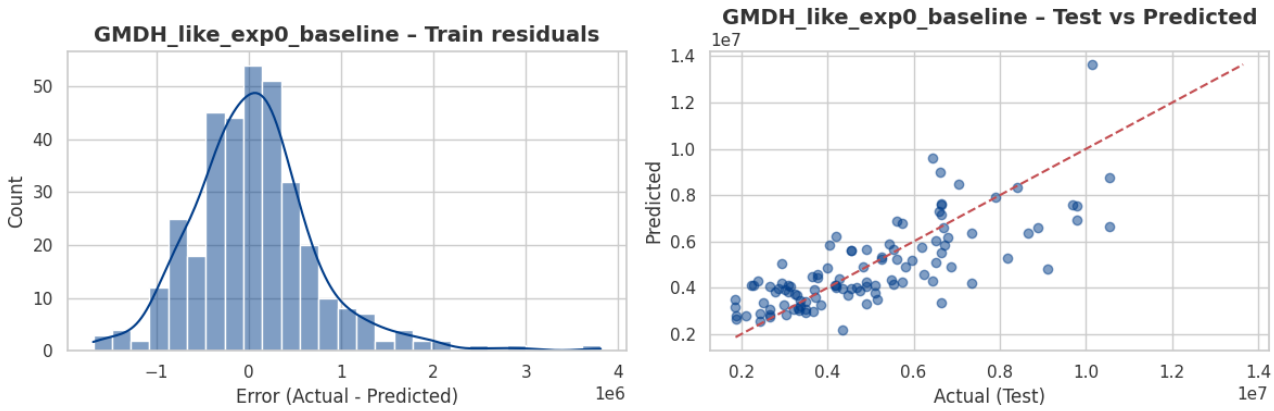


Figure 20: GMDH_like_exp0_baseline Residuals & Test Predictions

Looking at the two graphs in figure 20, we see that the model tends to under-predict at the high end, similar to the linear models. The residual distribution is relatively symmetrical but has a long right tail, suggesting that some important non-linear interactions have not yet been fully learned by the model.

Overall, Baseline provides stable performance but there is room for improvement with Regularization α = 0.001 still not strong enough to filter out noisy polynomial interactions. Some nonlinear interaction features may be overlearned or underlearned. The error in the high-price group is significant, suggesting that the model is not flexible enough or not regularized enough. These results provide a basis for experimenting with larger or smaller tuning α configurations to achieve a better balance between flexibility and stability, thereby finding the optimal GMDH-like model.

3.3.2.2 GMDH-like Experiment 1–3 (manual tuning)

After evaluating the baseline model, three GMDH-like configurations were tested to examine the influence of the regularization parameter α while keeping the polynomial degree at 2. These three configurations correspond to a strong α (0.01), a very small α (0.0001), and a manually tuned average α of 0.012. The choice of the three regularization levels was to observe how regularization controls the variance explosion of polynomial interactions: too large α can make the model inflexible, too small α can easily lead to overfitting, and the expected average α strikes a balance between complexity and stability.

Experiment Setup

GMDH_like_exp1_alpha_strong

- degree = 2
- alpha = 0.01

GMDH_like_exp2_alpha_small

- degree = 2
- alpha = 0.0001

GMDH_like_manual_tuned_best (Chosen best configuration)

- degree = 2
- alpha = 0.012

Hyperparameter Rationale

The choice of three values of α focuses on studying the effect of the strength of regularization on the quadratic polynomial model. The value α = 0.01 represents a fairly strong regularization, assuming that polynomial interactions can be easily disturbed and require a large penalty to control the coefficients. On the other hand, α = 0.0001 almost eliminates the effect of regularization, allowing the model to fit the data more freely but at the risk of exploding the variance. The value α = 0.012 is added as a balance point between these two extremes, with the expectation of retaining enough flexibility to describe nonlinearities, while still restraining the unstable interactions. Such a design allows to clearly observe the transition from under-regularization to over-regularization within the same architecture.

Tuned Model Performance

In Experiment 1(figure 21), with degree = 2 and α = 0.01, the model achieved a RMSE on the Test set of 1,283,096.08, MAE of 977,805.36, MAPE of 20.96%, and R^2 of 0.6335. Compared to the baseline, this is a clear improvement in both error and explained coefficient, showing that stronger regularization helps reduce noise in polynomial interactions. The accompanying residual and Actual vs Predicted plots show a neater error curve and the predicted points follow the reference line more closely than the original configuration.

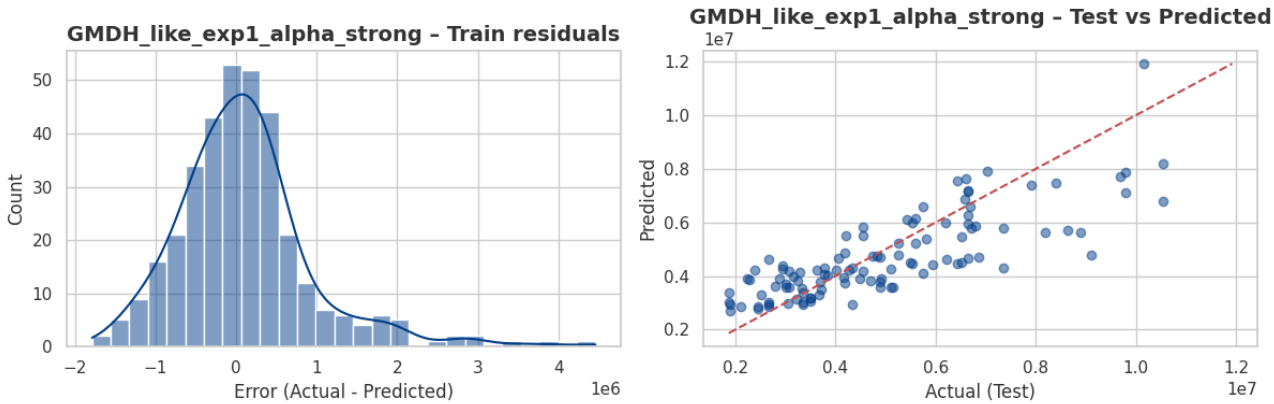


Figure 21: Residual distribution and Test Actual vs Predicted for GMDH_like_exp1_alpha_strong

In Experiment 2(figure 22), when α is reduced to 0.0001 but the degree is kept at 2, the performance degrades significantly. On the Test set, the RMSE increases to 1,477,879.80, MAE is 1,104,225.10, MAPE reaches 23.79% and R^2 decreases to 0.5138. This result shows that the regularization is too weak, causing the model to start fitting the noise, leading to larger errors and poor generalization ability. This is also reflected in the wider residual distribution and the predicted scatterplot deviates from the standard line.

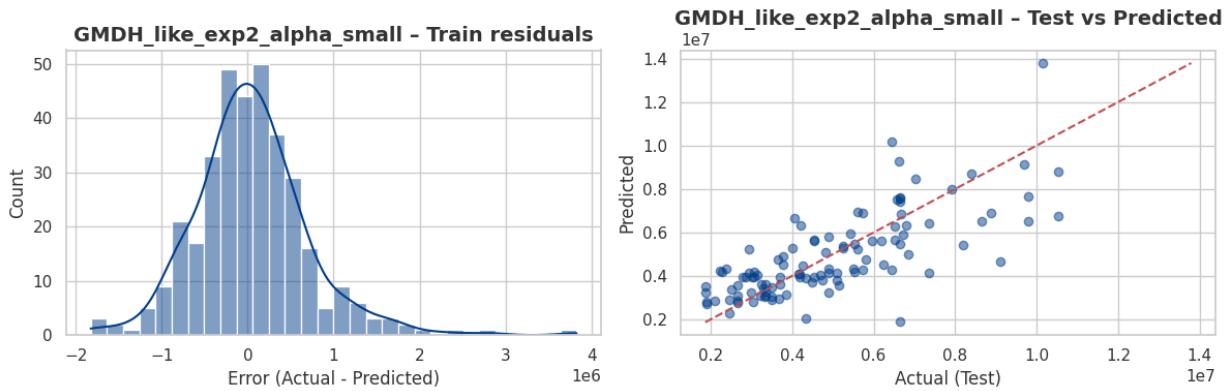


Figure 22: Residual distribution and Test Actual vs Predicted for GMDH_like_exp2_alpha_small

Experiment 3(figure 23) uses the degree = 2 configuration with α = 0.012 and gives the best performance among the entire GMDH-like group. On the Test set, the model achieves RMSE = 1,282,316.20, MAE = 977,545.96, MAPE = 21.00% and R^2 = 0.6339. These are slightly better than Experiment 1 and outperform both the baseline and the very small α configuration. The visual plots show that the residual is more concentrated around 0 and the predicted points are closer to the reference line, especially in the medium-high price range.

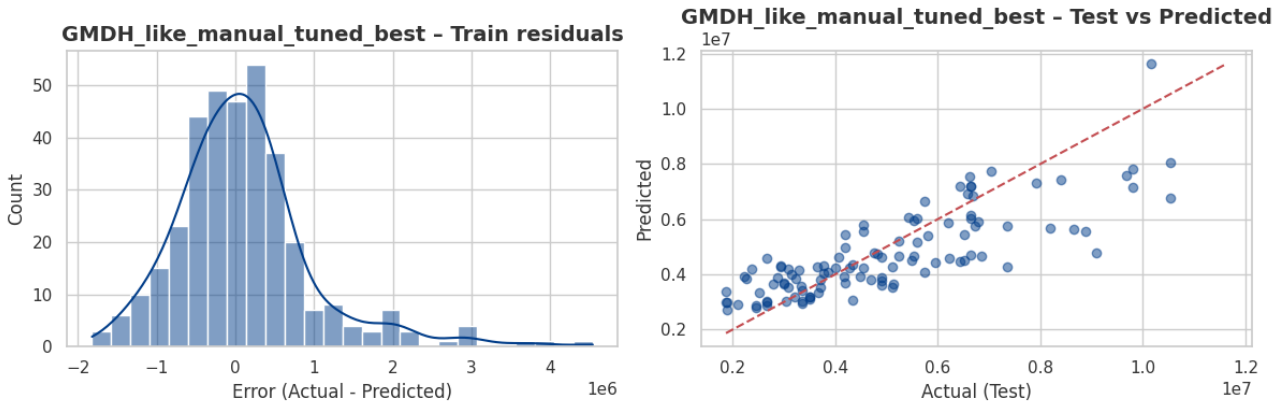


Figure 23: Residual distribution and Test Actual vs Predicted for GMDH_like_manual_tuned_best

Synthesizing the results in the performance table, it can be seen that with degree = 2, the performance of GMDH-like depends mainly on the choice of α. Too small a value like 0.0001 leads to under-regularization and a sharp decrease in R^2, while intermediate α levels around 0.01–0.012 produce a more stable and predictive model. Among these configurations, GMDH_like_manual_tuned_best with degree = 2 and α = 0.012 gives the lowest RMSE and highest R^2, and should be chosen as the optimal GMDH-like configuration to be used in the baseline–tuned comparison and to compare with other model families.

Baseline vs Tuing: GMDH-like

Table 25: Summarises the performance across all experimental configurations.

Experiment	RMSE	MAE	MAPE (%)	R2 Score
GMDH_like_manual_tuned_best	1,282,316.20	977,545.96	21.00%	0.6339
GMDH_like_exp0_baseline	1,408,442.49	1,073,255.94	22.95%	0.5584

In table 25, the tuned configuration (degree = 2, α = 0.012) clearly outperforms the baseline, improving RMSE from 1,408,442.49 to 1,282,316.20 and increasing R^2 from 0.5584 to 0.6339. This shows that moderate regularisation strengthens the stability of polynomial interactions without restricting the model's flexibility.This comparison motivates a closer visual examination of how the baseline and the best-tuned GMDH-like models differ in terms of residual behaviour and prediction accuracy.

Baseline vs Tuned Model: Visual Diagnostics

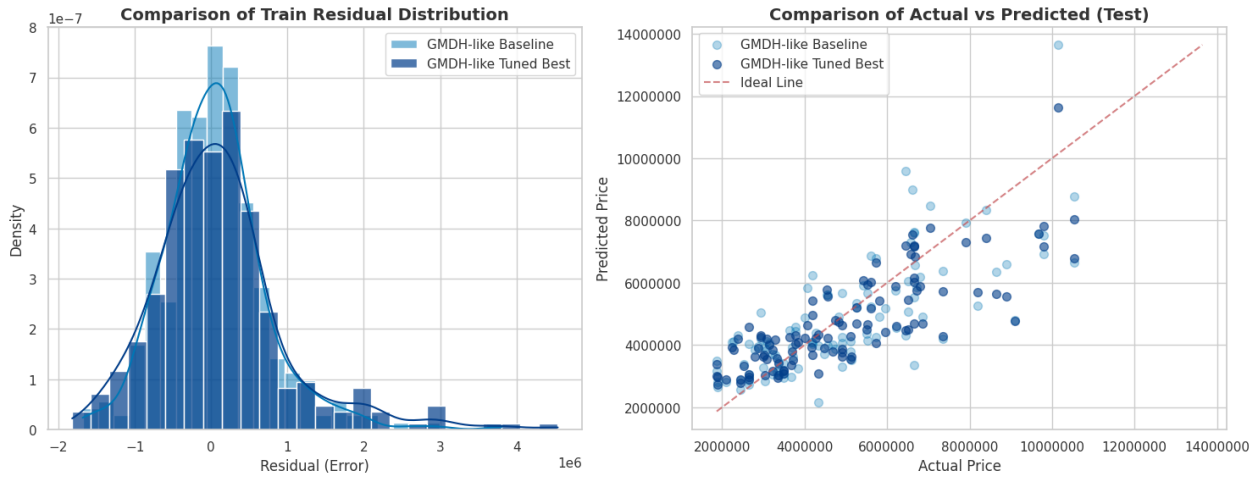


Figure 24:Comparison of residual distribution and test-set predictions between GMDH-like Baseline and Tuned Best model.

Figure 24 shows diagnostic diagrams that support this trend. While the tuned model predicts more variance with a more broad and more dispersed baseline residual curve, it also shows a stronger focus close zero. Tuned predictions, especially in the mid-price range cluster more closely along the 45-degree line; the baseline reveals larger variances and systematic undervaluation of expensive properties in the Actual vs Predicted chart.

Generally, the comparison shows that the main hyperparameter in the GMDH-like architecture is regularisation strength α. With degree fixed at 2 (already better than degree 3), α = 0.012 provides the best balance between stability and flexibility, therefore making this the most fitting configuration for this kind of models.

3.3.3 Comparative Discussion (ANN vs GMDH-like)

Though sharing the aim of capturing nonlinear structure beyond the linear baseline, the two model families behave very differently across the neural experiments. With its best configuration reaching a Test RMSE of 1.28M and an R^2 of 0.634, the GMDH-like model consistently provides more stable and dependable performance. By building polynomial interactions selectively and limiting them through L1 regularisation, the model catches mild nonlinearities without drifting into overfitting. The diagnostic plots support this behaviour: residuals cluster more tightly around zero, and the anticipated values line more closely with the 45° reference line than in the baseline version

By contrast, ANN models struggle severely in this low-sample tabular setting. Three out of four configurations collapse into extreme overfitting, yielding negative R^2 values (as low as –3.27). Even the strongest ANN (Experiment 3) reaches only 0.421 R^2, far below both the GMDH-like model and even the linear baselines. The large dispersion seen in the Test-set predictions illustrates ANN's difficulty in forming stable nonlinear mappings when the dataset is too small for its high-capacity architecture to generalise, even with dropout, Adam optimisation, and early stopping.

Taken together, the evidence makes the choice of best model for this group straightforward. The GMDH-like model decisively outperforms all ANN configurations, offering a more controlled and data-efficient nonlinear representation. Its polynomial structure is expressive enough to improve upon linear models but sufficiently regularised to avoid the instability observed in the neural network experiments. This result is consistent with findings in tabular machine learning: when datasets are small and noise levels modest, polynomial-based architectures often outperform standard feedforward neural networks.

Therefore, within the deep learning group, the GMDH-like model is selected as the best-performing model.

3.4 Comparative Discussion Across Model Families (Linear vs Neural vs Tree-based)

To provide a clear and consistent comparison among model types, the best performing system from each family, OLS (Linear Models), GMDH-like (Polynomial Networks), and XGBoost (Tree-based Models), was drawn and visually shown together.

Direct evaluation of their predictive power, stability, and overall generalisation patterns is made possible by this aggregation without returning the complete diagnoses already addressed in earlier sections.

Table 26. Summary of Best Models from Each Family

Model Family	Best Configuration	RMSE	MAE	MAPE	R^2

Linear	OLS	1,200,600	931,822	20.03%	0.679
Neural (Polynomial)	GMDH-like (α = 0.012)	1,282,316	977,545	21.00%	0.634
Tree-based	XGBoost (RandomSearch Tuned)	**1,163,692**	**888,073**	**19.30%**	**0.6985**

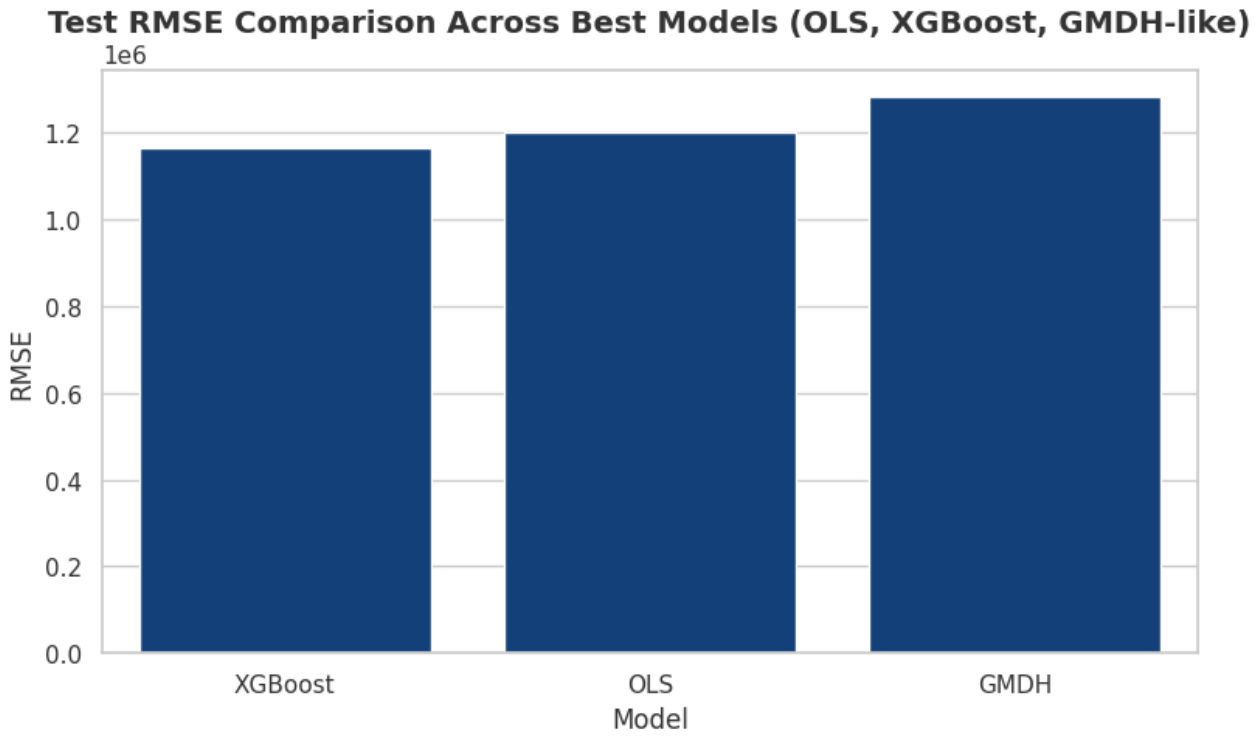


Figure 25: Test RMSE Comparison Across Best Models

The first series of comparisons concentrates on predictive accuracy, as seen in Figures 25 and 26 through RMSE and R^2. With the lowest Test RMSE (about 1.16 million) and the highest R^2 (0.6985), XGBoost obviously does really well. According to R^2 near to 0.679, OLS explains most of the variance in this dataset even if it has a linear structure. Nonlinear terms provide freedom yet in this instance they do not improve precision; GMDH-like achieves marginally worse (R^2 about 0.634). Together these results show how gradient-boosted trees' capacity to extract the nonlinear structure seen in the data surpasses that of polynomial expansions or shallow neural networks.

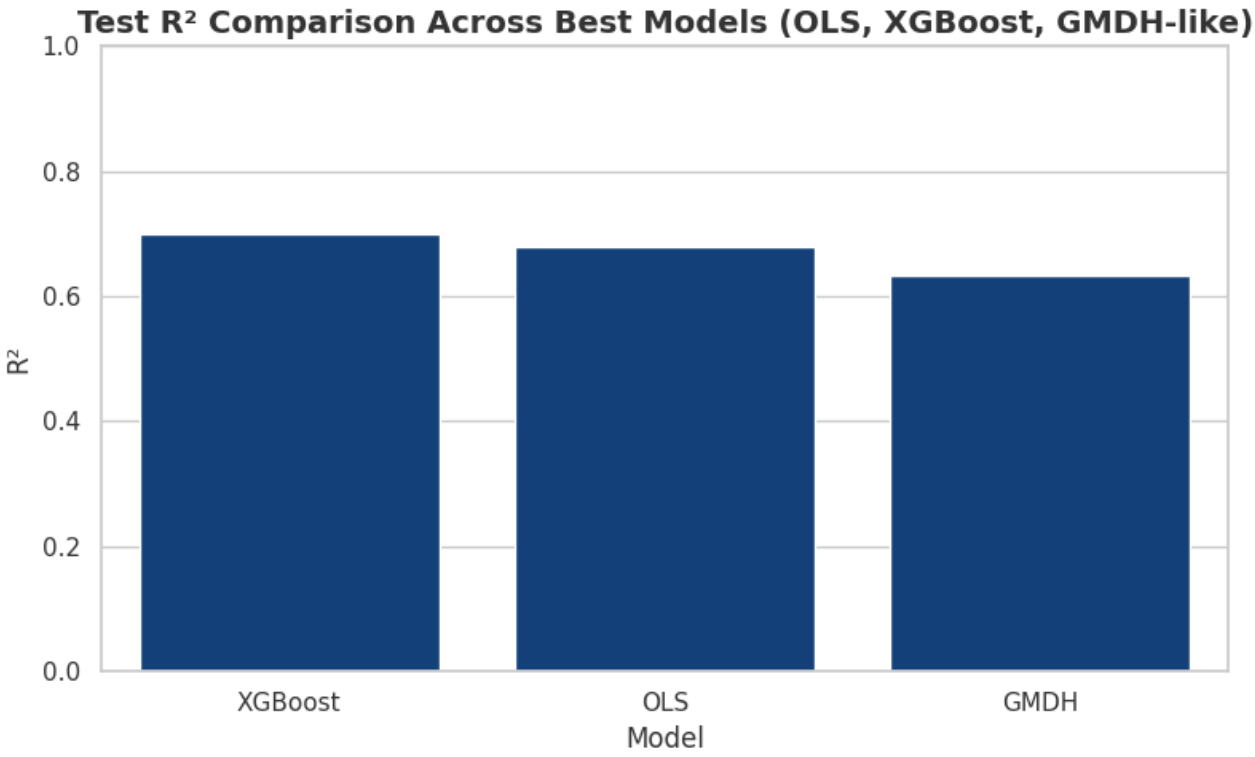


Figure 26. Test R² Comparison Across Best Models

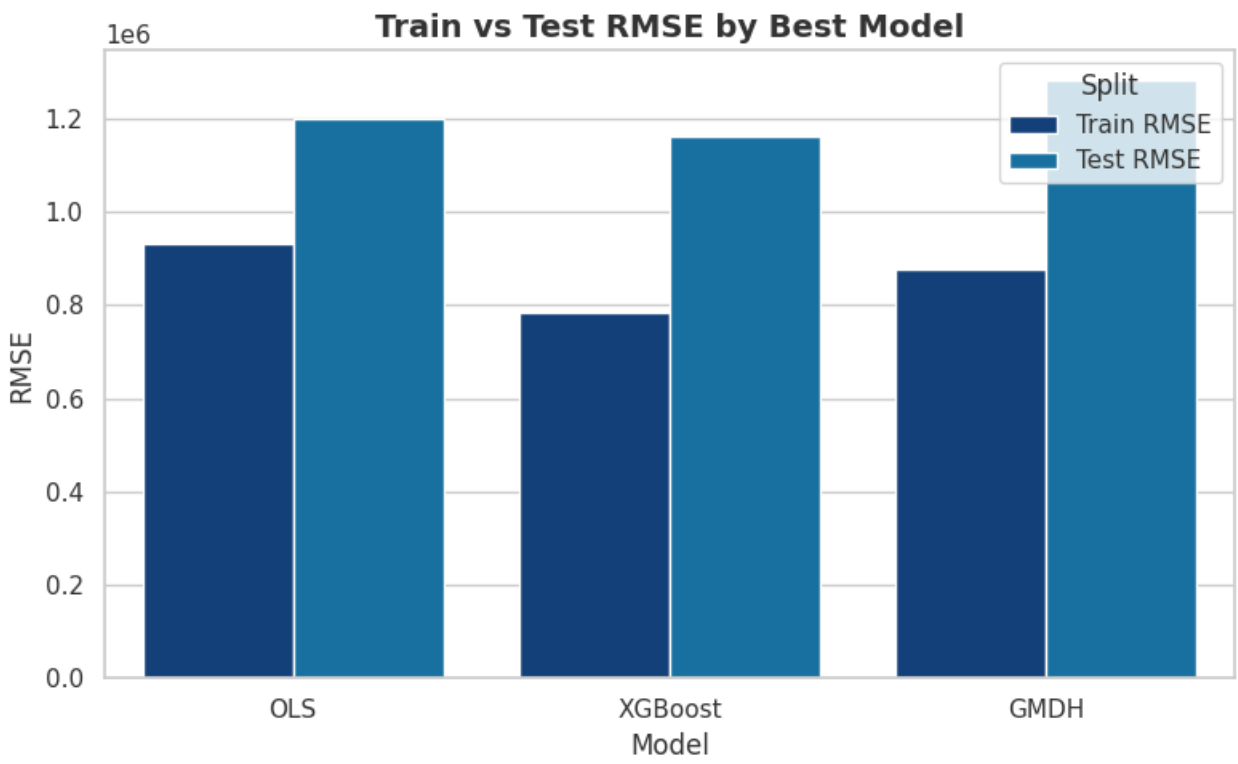


Figure 27. Train vs Test RMSE Comparison

Figures 27 and 28 highlight the disparities in generalisation behaviour among the three models. OLS shows the most stable train–test alignment, mirroring the intrinsic regularisation and low variability of linear models. GMDH-like adds flexibility and, as predicted, shows a larger train–test difference as a result of slight overfitting. XGBoost, even though it is the most flexible of the three, keeps a balanced generalisation profile thanks to its subsampling, tree-depth control, and learning-rate scheduling.

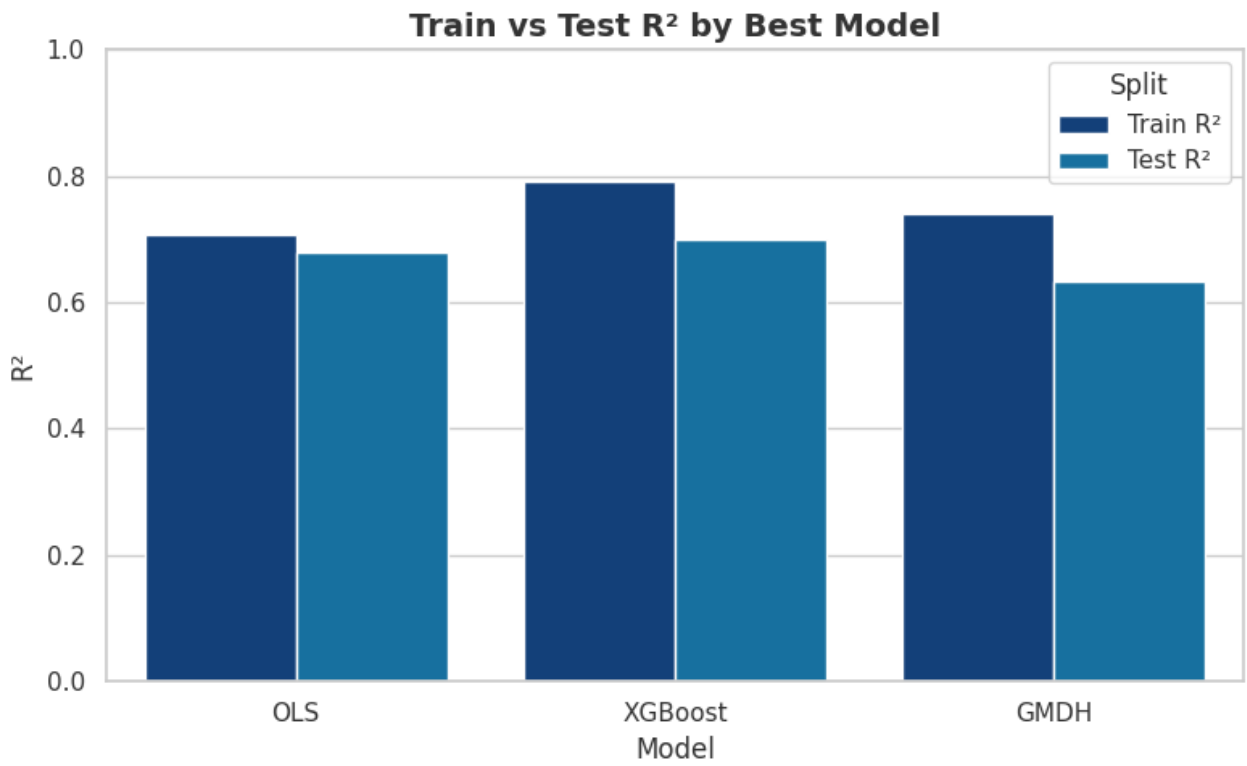


Figure 28: Train vs Test R² Comparison

Comparing model capacity against database size aids one in grasping why XGBoost emerges as the most appropriate choice. OLS is unable to adjust to small nonlinearities although it employs the strong linear signal in the data. GMDH-like interactions bring quadratic ones; but, the little size of the data limits the polynomials' growth. XGBoost, however, works especially well for tabular data of little complexity since it progressively constructs nonlinear decision borders and limits noise magnification by means of shrinkage and regularisation. Therefore, it better suits the structural trends of the dataset without big sample counts.

XGBoost is the best general model across the three families depending on the combined visualizations and gathered performance metrics. For this data collection, it offers the most suitable inductive bias, the highest predictive accuracy, and consistent competitive generalisation stability.

Given its simplicity, OLS is still a very reliable baseline; however, GMDH-like provides a viable second-best nonlinear alternative lacking the robustness and speed witnessed in XGBoost.

Therefore, the family-by-family comparison shows XGBoost as the perfect champion model, the primary point of reference for the conclusion part of the study.

4. Conclusion

Seven predictive models for housing price estimation were evaluated using a dataset of 545 residential houses. Using a same experimental pipeline, the analysis found XGBoost, following RandomizedSearchCV tuning, as the best performer with a Test R^2 of 0.6985 and RMSE of about 1.16M VND by comparing linear, tree-based, neural, and polynomial architectures. This result demonstrates that boosted tree ensembles are particularly effective for small-to-medium tabular datasets where relationships are moderately nonlinear but where sample size limits the ability of more complex models to generalise.

When comparing the findings to existing literature, the performance achieved in this study represents a notable improvement over several traditional regression applications historically used in housing valuation. For example, Mankad (2021) reports OLS performance at $R^2 \approx 0.20$ on an Indian housing dataset, while Zaki et al. (2022) observe $R^2 \approx 0.42$ using a Hedonic Regression model on the Boston Housing data. Even more advanced linear approaches such as Ridge and Lasso in Akyüz et al. (2022) achieve only R^2 values between 0.38 and 0.45, significantly below the 0.6985 obtained here. Regarding Preethi et al. (2025) on the California Housing Dataset, the XGBoost performance in this study also surpass the roughly 0.60 level recorded for Linear, Ridge, Polynomial Ridge, and SVR-RBF models. These results demonstrate that, even on rather small datasets with correct hyperparameter adjustment and comprehensive preprocessing, tree-boosting techniques can have very predictive accuracy.

Although its findings are good, this study has several drawbacks that account for why its predictive accuracy is less than the higher levels, $R^2 \geq 0.85$, seen in more data-rich research. Studies such as Ozdemir et al. (2022), which achieved $R^2 \approx 0.86$ employing Deep Learning, depend on far bigger datasets and contain macroeconomic variables including gold price, interest rates, and construction material prices, variables not present in the current dataset. Likewise, Zaki et al. (2022) get $R^2 \approx 0.84$ employing XGBoost along with careful outlier treatment and rigorous hyperparameter optimization; Zhou (2021) achieves $R^2 \approx 0.88$ by including polynomial regression with Particle Swarm Optimization, a meta-heuristic method not adopted here. Furthermore showing even

superior performance, Mora-Garcia et al. (2022) train Random Forests and XGBoost on practically 40,000 observations combined with extensive GIS-based spatial characteristics, $R^2 > 0.91$. The discrepancy emphasizes the necessity of data scale and feature richness in assisting nonlinear models, especially deep and ensemble techniques, to catch fine-grained pricing drivers.

The main results show themselves across every model family. Although linear models provide reliable, clear baselines, they fail to adequately characterize nonlinear deviations at more expensive levels. Data scarcity makes ANN models lag; only one setup achieves positive R^2. GMDH-like modelling provides a regulated polynomial structure that generalises somewhat but still falls behind tree ensembles. Often outperforming Decision Trees and Random Forests while eschewing the overfitting typical of deeper neural networks, XGBoost provides the perfect balance of flexibility and regularisation. Boosted trees are the most appropriate modelling approach for this collection of data.

Practically, these results suggest that while working with tiny structured datasets free of significant geographic or economic features, real-estate analysts, lenders, or valuation systems can rely on XGBoost-type models. These models offer rather good predictive accuracy and have quite low computational cost.

Looking ahead, future research should concentrate on expanding the dataset, including macroeconomic and geospatial predictors, and experimenting with hybrid architectures such polynomial–boosting combinations or meta-heuristic optimization. Enhancements of this nature show much promise for correcting model correctness toward the top levels specified in the literature.

Essentially, the research demonstrates a powerful modeling methodology that achieves an accuracy level ($R^2 = 0.6985$), beyond several traditional criteria, while stressing the obvious ways by means of which more extensive data and more complex feature construction may generate more predictive performance improvement.

References

1. Abhyankar, A. A. and Singla, H. K. (2022) 'Comparing predictive performance of general regression neural network (GRNN) and hedonic regression model for factors affecting housing prices in "Pune-India"', *International Journal of Housing Markets and Analysis*, 15(2), pp. 451–477. Available at: https://www.sciencedirect.com/org/science/article/abs/pii/S1753827021000856

2. Akyüz, S. Ö., Erdogan, B. E., Yıldız, Ö. and Ataş, P. K. (2022) 'A Novel Hybrid House Price Prediction Model', *Computational Economics*.Available at: https://ideas.repec.org/a/kap/compec/v62y2023i3d10.1007_s10614-022-10298-8.html *(Dùng trong*

3. Geerts, M., vanden Broucke, S. and De Weerdt, J. (2023) 'A Survey of Methods and Input Data Types for House Price Prediction', *ISPRS International Journal of Geo-Information*, 12(5), p. 200. Available at: https://www.mdpi.com/2220-9964/12/5/200

4. Mankad, M. D. (2022) 'Comparing OLS based hedonic model and ANN in house price estimation using relative location', *Spatial Information Research*, 30(1), pp. 107–116. Available at: https://link.springer.com/article/10.1007/s41324-021-00416-3

5. Mora-Garcia, R.-T., Cespedes-Lopez, M.-F. and Perez-Sanchez, V. R. (2022) 'Housing Price Prediction Using Machine Learning Algorithms in COVID-19 Times', *Land*, 11(11), p. 2100. Available at: https://www.mdpi.com/2073-445X/11/11/2100

6. Mostofi, F., Toğan, V. and Başağa, H. B. (2022) 'Real-estate price prediction with deep neural network and principal component analysis', *Organization, Technology and Management in Construction: an International Journal*, 14(1), pp. 2742–2759. Available at: https://www.researchgate.net/publication/366979465_Real

7. Nguyen, N. and Cripps, A. (2001) 'Predicting Housing Value: A Comparison of Multiple Regression Analysis and Artificial Neural Networks', *The Journal of Real Estate Research*, 22(3), pp. 313–336. Available at: https://www.researchgate.net/publication/5142150_Predicting_Housing_Value_A_Comparison_of_Multiple_Regression_Analysis_and_Artificial_Neural_Networks

8. Özdemir, M., Yıldız, K. and Büyüktanır, B. (2022) 'Housing Price Estimation with Deep Learning: A Case Study of Sakarya Turkey', *BSEU Journal of Science*, 9(1), pp. 138–151. Available at: https://research.ebsco.com/c/lrha6p/search/details/5qhiewqyh5?db=asx

9. Preethi, Murthy, D. H. R., Hiremani, V., Devadas, R. M. and Sapna, R. (2025) 'Optimizing Polynomial and Regularization Techniques for Enhanced Housing Price Prediction Accuracy', *SN*

Computer Science, 6(96). Available at: https://link.springer.com/article/10.1007/s42979-024-03578-7

10. Satish, G. N., Raghavendran, Ch. V., Rao, M. D. S. and Srinivasulu, Ch. (2019) 'House Price Prediction Using Machine Learning', *International Journal of Innovative Technology and Exploring Engineering*, 8(9). Available at: https://www.researchgate.net/publication/364660836_House_Price_Prediction_using_Machine_Learning

11. Truong, Q., Nguyen, M., Dang, H. and Mei, B. (2020) 'Housing Price Prediction via Improved Machine Learning Techniques', *Procedia Computer Science*, 174, pp. 433–442. Available at: https://www.sciencedirect.com/science/article/pii/S1877050920316318

12. Zaki, J., Nayyar, A., Dalal, S. and Ali, Z. H. (2022) 'House price prediction using hedonic pricing model and machine learning techniques', *Concurrency and Computation: Practice and Experience*, 34(27), e7342. Available at: https://www.researchgate.net/publication/363805991_House_price_prediction_using_hedonic_pricing_model_and_machine_learning_techniques

13. Zhou, C. (2021) 'House price prediction using polynomial regression with Particle Swarm Optimization', *Journal of Physics: Conference Series*, 1802(3), p. 032034. Available at: https://iopscience.iop.org/article/10.1088/1742-6596/1802/3/032034

Appendix

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
```

```python
import pandas as pd

# Import file
file_path = "/content/drive/MyDrive/Colab notebooks part 2 :3/Housing.csv"
df = pd.read_csv(file_path,sep=',')
display(df.head())
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished

```python
df = pd.read_csv(file_path)
print("Shape:", df.shape)
display(df.head())
print(df.info())
print("Missing values:\n", df.isnull().sum())
```

```
Shape: (545, 13)
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
```

1.EDA

```python
df.describe(include='all')
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
count	5.450000e+02	545.000000	545.000000	545.000000	545.000000	545	545	545	545	545	545.000000	545	545
unique	NaN	NaN	NaN	NaN	NaN	2	2	2	2	2	NaN	2	3
top	NaN	NaN	NaN	NaN	NaN	yes	no	no	no	no	NaN	no	semi-furnished
freq	NaN	NaN	NaN	NaN	NaN	468	448	354	520	373	NaN	417	227
mean	4.766729e+06	5150.541284	2.965138	1.286239	1.805505	NaN	NaN	NaN	NaN	NaN	0.693578	NaN	NaN
std	1.870440e+06	2170.141023	0.738064	0.502470	0.867492	NaN	NaN	NaN	NaN	NaN	0.861586	NaN	NaN
min	1.750000e+06	1650.000000	1.000000	1.000000	1.000000	NaN	NaN	NaN	NaN	NaN	0.000000	NaN	NaN
25%	3.430000e+06	3600.000000	2.000000	1.000000	1.000000	NaN	NaN	NaN	NaN	NaN	0.000000	NaN	NaN
50%	4.340000e+06	4600.000000	3.000000	1.000000	2.000000	NaN	NaN	NaN	NaN	NaN	0.000000	NaN	NaN
75%	5.740000e+06	6360.000000	3.000000	2.000000	2.000000	NaN	NaN	NaN	NaN	NaN	1.000000	NaN	NaN
max	1.330000e+07	16200.000000	6.000000	4.000000	4.000000	NaN	NaN	NaN	NaN	NaN	3.000000	NaN	NaN

```python
yes_no_cols = ['mainroad','guestroom','basement',
               'hotwaterheating','airconditioning','prefarea']
num_cols = ['area','bedrooms','bathrooms','stories','parking']
cat_cols = ['furnishingstatus'] #one-hot encoding because the variable has more than 2 values yes, no
target_col = 'price'
```

```python
# EDA cho biến numerical
# 1. Histogram for variables
df[num_cols + [target_col]].hist(figsize=(12,8), bins=30)
plt.suptitle("Histograms of numerical variables", y=1.02)
plt.show()

# 2 & 3. Log(price) and log(area) distribution -Combine into one row
fig, axes = plt.subplots(1, 2, figsize=(12,4))

sns.histplot(np.log(df[target_col]), bins=30, kde=True, ax=axes[0])
axes[0].set_title("Distribution of log(price)")
axes[0].set_xlabel("log(price)")
axes[0].set_ylabel("Frequency")

sns.histplot(np.log(df['area']), bins=30, kde=True, ax=axes[1])
axes[1].set_title("Distribution of log(area)")
axes[1].set_xlabel("log(area)")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.show()

# 4.Scatter some key variables with price - Adjust to 2 charts per row
plt.figure(figsize=(15, 10))

scatter_cols = ['area','bedrooms','bathrooms','stories','parking']
num_plots = len(scatter_cols)
num_rows = (num_plots + 1) // 2

for i, c in enumerate(scatter_cols):
    plt.subplot(num_rows, 2, i + 1)
    plt.scatter(df[c], df[target_col], alpha=0.5)
    plt.xlabel(c); plt.ylabel("price")
    plt.title(f"{c} vs price")

plt.tight_layout()
plt.show()

# 5. Correlation matrix between variables + price
numeric_for_corr = num_cols + [target_col]
plt.figure(figsize=(7,6))
sns.heatmap(df[numeric_for_corr].corr(), annot=True, cmap='Blues', fmt=".2f")
plt.title("Correlation matrix (numerical variables)")
plt.show()
```

```python
# Created boxplots for price and area to visualise extreme observations.

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(y=df['price'], ax=axes[0])
axes[0].set_title("Boxplot of Price")
axes[0].set_ylabel("Price")

sns.boxplot(y=df['area'], ax=axes[1])
axes[1].set_title("Boxplot of Area")
axes[1].set_ylabel("Area")

plt.tight_layout()
plt.show()
```

```python
# EFA for categorical variables (binary and multi-level)

# 1. 'furnishingstatus'
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.countplot(x='furnishingstatus', data=df, ax=axes[0])
axes[0].set_title("Furnishing status distribution")
axes[0].set_ylabel("Count")
axes[0].set_xlabel("Furnishing Status")

sns.boxplot(x='furnishingstatus', y=target_col, data=df, ax=axes[1])
axes[1].set_title("Price by furnishingstatus")
axes[1].set_ylabel("Price")
axes[1].set_xlabel("Furnishing Status")

plt.tight_layout()
plt.show()

# 2. Yes/no variables (binary)
# Create a larger figure to contain all countplot and boxplot pairs
fig, axes = plt.subplots(len(yes_no_cols), 2, figsize=(12, 4 * len(yes_no_cols)))

for i, c in enumerate(yes_no_cols):
    #Countplot for binary variables
    sns.countplot(x=c, data=df, ax=axes[i,0])
    axes[i,0].set_title(f"Distribution of {c}")
    axes[i,0].set_ylabel("Count")
    axes[i,0].set_xlabel(c)

    # Boxplot of price by binary variable
    sns.boxplot(x=c, y=target_col, data=df, ax=axes[i,1])
    axes[i,1].set_title(f"Price by {c}")
    axes[i,1].set_ylabel("Price")
    axes[i,1].set_xlabel(c)

plt.tight_layout()
plt.show()
```

4.Handle outliers + check data after clean

```
# 4.1Create log variables for price and area
df['price_log'] = np.log(df['price'])
df['log_area']  = np.log(df['area'])

# 4.2 Winsorize function for 2 distribution ends (e.g. 1% and 99%)
def winsorize_series(s, lower_q=0.01, upper_q=0.99):

    lower = s.quantile(lower_q)
    upper = s.quantile(upper_q)
    return s.clip(lower=lower, upper=upper)

# 4.3 Apply winsorize to price_log and log_area
df['price_log'] = winsorize_series(df['price_log'], lower_q=0.01, upper_q=0.99)
df['log_area']  = winsorize_series(df['log_area'],  lower_q=0.01, upper_q=0.99)
df_clean = df.copy()

# 4.4 Quick check after clean
print("Shape after cleaning:", df_clean.shape)
print(df_clean[['price','price_log','area','log_area']].describe())

# 4.5 Some test plots (can be omitted if you want a compact notebook)
fig, axes = plt.subplots(2, 2, figsize=(12,8))

sns.histplot(df_clean['price'], bins=30, ax=axes[0,0])
axes[0,0].set_title("price (original)")

sns.histplot(df_clean['price_log'], bins=30, ax=axes[0,1])
axes[0,1].set_title("price_log after winsorize")

sns.histplot(df_clean['area'], bins=30, ax=axes[1,0])
axes[1,0].set_title("area (original)")

sns.histplot(df_clean['log_area'], bins=30, ax=axes[1,1])
axes[1,1].set_title("log_area after winsorize")

plt.tight_layout()
plt.show()

# Boxplot to see if the outlier has softened
fig, axes = plt.subplots(1, 2, figsize=(10,4))
sns.boxplot(y=df_clean['price_log'], ax=axes[0])
axes[0].set_title("price_log after winsorize")
sns.boxplot(y=df_clean['log_area'], ax=axes[1])
axes[1].set_title("log_area after winsorize")
plt.tight_layout()
plt.show()
```

5. Create dataset modeling from data after winsorize (df_clean)

```python
# 5.1 Select numerical variables (log + winsor)
num_vars = ['price_log', 'log_area', 'bedrooms', 'bathrooms', 'stories', 'parking']

# 5.2 Select text yes/no variables
yes_no_cols = ['mainroad', 'guestroom', 'basement',
'hotwaterheating', 'airconditioning', 'prefarea']

# 5.3 Furnishingtatus variable has 3 levels
cat_vars = ['furnishingstatus']
# 5.4 Encode yes/no → 0/1
df_enc = df_clean.copy()
for c in yes_no_cols:
df_enc[c] = df_enc[c].map({'yes': 1, 'no': 0}).astype(int)

#5.5 One-hot encoding specifically for furnishingstatus (k-1 dummy)
df_model = pd.get_dummies(
df_enc[num_vars + yes_no_cols + cat_vars],
columns=cat_vars,
drop_first=True # drop one baseline level to avoid multicollinearity
)

#5.6 Confirm X, y for modeling
y_all = df_model['price_log'] #target
X_all = df_model.drop(columns=['price_log']) # features

print("df_model shape:", df_model.shape)
print("X_all shape :", X_all.shape)
print("y_all shape :", y_all.shape)

df_model.head()
```

```
df_model shape: (545, 14)
X_all shape : (545, 13)
y_all shape : (545,)
```

```python
# 6.1 train/val/test split
X_trainval, X_test, y_trainval, y_test = train_test_split(
    X_all, y_all, test_size=0.20, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.20, random_state=42
)
print("Train:", X_train.shape)
print("Val:",   X_val.shape)
print("Test:",  X_test.shape)

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

# 6.2. CALCULATE VIF (variance inflation factor) ON TRAIN SET (AFTER ENCODING, BEFORE SCALING)
# Make sure X_train is still a DataFrame, not an array
print("Type of X_train:", type(X_train))
print("X_train shape for VIF:", X_train.shape)

# If X_train is a numpy array, convert it back to a DataFrame using X_all.columns
if not isinstance(X_train, pd.DataFrame):
    X_train_vif = pd.DataFrame(X_train, columns=X_all.columns)
else:
    X_train_vif = X_train.copy()

display(X_train_vif.head())

# 6.2.1 Keep only numeric columns
X_vif_df = X_train_vif.select_dtypes(include=[np.number]).copy()
print("Numeric columns used for VIF:", X_vif_df.columns.tolist())

#6.2.2 Handling NaN / inf if present
X_vif_df = X_vif_df.replace([np.inf, -np.inf], np.nan).dropna()
print("Shape after dropping NaN/inf (for VIF):", X_vif_df.shape)
```

```python
# 6.2.3 ADD constant
X_vif_const = sm.add_constant(X_vif_df)

# 6.2.4 Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data["feature"] = X_vif_const.columns
vif_data["VIF"] = [
    variance_inflation_factor(X_vif_const.values, i)
    for i in range(X_vif_const.shape[1])
]

display(vif_data)
# 6.3 scaling for numeric
scaler = StandardScaler()

Xtr_s  = scaler.fit_transform(X_train)
Xval_s = scaler.transform(X_val)
Xte_s  = scaler.transform(X_test)
```

```
Train: (348, 13)
Val: (88, 13)
Test: (109, 13)
Type of X_train: <class 'pandas.core.frame.DataFrame'>
X_train shape for VIF: (348, 13)
```

7. Generate a detailed regression report function (to produce the same output format for all the different regression techniques below)

```python
def detailed_regression_report(model,
                               X_train, y_train_log,
                               X_test,  y_test_log,
                               name="Regression model",
                               feature_names=None):

    # 1. Predict trên TRAIN, TEST (log)
    y_train_pred_log = model.predict(X_train).ravel()
    y_test_pred_log  = model.predict(X_test).ravel()

    # 2. Return to the ORIGINAL PRICE scale
    y_train_true = np.exp(y_train_log.values)
    y_test_true  = np.exp(y_test_log.values)
    y_train_pred = np.exp(y_train_pred_log)
    y_test_pred  = np.exp(y_test_pred_log)

    # 3. Residuals
    train_resid = y_train_true - y_train_pred

    # 4. Metrics TRAIN / TEST
    r2_tr  = r2_score(y_train_true, y_train_pred)
    mse_tr = mean_squared_error(y_train_true, y_train_pred)
    rmse_tr = np.sqrt(mse_tr)
    rss_tr = np.sum((y_train_true - y_train_pred)**2)

    r2_te  = r2_score(y_test_true, y_test_pred)
    mse_te = mean_squared_error(y_test_true, y_test_pred)
    rmse_te = np.sqrt(mse_te)
    rss_te = np.sum((y_test_true - y_test_pred)**2)

    print(f"\n<<<---------------- {name} ---------------->>>")

    # 5. Plot residual & Actual vs Predicted chart
    fig, axes = plt.subplots(1, 2, figsize=(12,4))

    sns.histplot(train_resid, kde=True, ax=axes[0])
    axes[0].set_title("Error Terms (Train residuals)")
    axes[0].set_xlabel("Errors")

    axes[1].scatter(y_test_true, y_test_pred, alpha=0.5)
    lims = [min(y_test_true.min(), y_test_pred.min()),
            max(y_test_true.max(), y_test_pred.max())]
    axes[1].plot(lims, lims, 'r--')
    axes[1].set_title("Test vs Prediction")
    axes[1].set_xlabel("Actual (Test)")
    axes[1].set_ylabel("Predicted")

    plt.tight_layout()
    plt.show()
```

```python
    # 6.Subsidiary OLS estimate to get coef + p-value
    if isinstance(X_train, pd.DataFrame):
        X_train_sm = X_train.copy()
    else:
        n_features = X_train.shape[1]
        if feature_names is not None and len(feature_names) == n_features:
            col_names = list(feature_names)
        else:
            col_names = [f"x{i+1}" for i in range(n_features)]
        X_train_sm = pd.DataFrame(X_train, columns=col_names)

    X_train_const = sm.add_constant(X_train_sm)
    ols_sm = sm.OLS(y_train_log.values, X_train_const)
    ols_res = ols_sm.fit()

    coef_df = pd.DataFrame({
        "variable": ols_res.params.index,
        "coef":     ols_res.params.values,
        "std_err":  ols_res.bse.values,
        "t_value":  ols_res.tvalues.values,
        "p_value":  ols_res.pvalues.values
    })

    print("\n=== Coefficient & p-value table (auxiliary OLS on TRAIN) ===")
    display(coef_df)

    # 7. Return metrics (if you want to use them for comparison later)
    metrics_dict = {
        "R2_train":   r2_tr,
        "RMSE_train": rmse_tr,
        "MSE_train":  mse_tr,
        "RSS_train":  rss_tr,
        "R2_test":    r2_te,
        "RMSE_test":  rmse_te,
        "MSE_test":   mse_te,
        "RSS_test":   rss_te,
    }

    return metrics_dict, coef_df
```

8. MODEL 1: OLS (Linear Regression)

```python
# === OLS LINEAR REGRESSION ===

# 1. eval_log_model function: evaluates RMSE, MAE, MAPE, R2 on the ORIGINAL PRICE SCALE
def eval_log_model(model, Xmat, y_true_log):
    y_pred_log = model.predict(Xmat).ravel()
    y_pred_log = np.clip(y_pred_log, -20, 20)

    # If it is log(price) then change to np.exp()
    y_true = np.exp(y_true_log.values)
    y_pred = np.exp(y_pred_log)

    mse  = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae  = mean_absolute_error(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / np.clip(y_true, 1e-9, None))) * 100
    r2   = r2_score(y_true, y_pred)

    return rmse, mae, mape, r2

# 2. Actual vs Predicted drawing function
def plot_pred_vs_actual(y_true, y_pred, title="Actual vs Predicted"):
    plt.figure(figsize=(5,4))
    plt.scatter(y_true, y_pred, alpha=0.5)
    lims = [min(y_true.min(), y_pred.min()),
            max(y_true.max(), y_pred.max())]
    plt.plot(lims, lims, 'r--')
    plt.xlabel("Actual price")
    plt.ylabel("Predicted price")
    plt.title(title)
    plt.tight_layout()
    plt.show()

# 3. Build OLS model
ols = LinearRegression()
```

```python
# 4. Train on scaled TRAIN set
ols.fit(Xtr_s, y_train)

# 5. Evaluate (RMSE, MAE, MAPE, R2) cho TRAIN / VAL / TEST
ols_train = eval_log_model(ols, Xtr_s,  y_train)
ols_val   = eval_log_model(ols, Xval_s, y_val)
ols_test  = eval_log_model(ols, Xte_s,  y_test)

print("=== OLS Linear Regression ===")
print("Train (RMSE, MAE, MAPE, R2):", ols_train)
print("Val   (RMSE, MAE, MAPE, R2):", ols_val)
print("Test  (RMSE, MAE, MAPE, R2):", ols_test)

# 5b. METRICS SUMMARY TABLE FOR 3 SEQUENCES (Train - Val - Test)
ols_metrics_table = pd.DataFrame({
    "Split": ["Train", "Val", "Test"],
    "RMSE":  [ols_train[0], ols_val[0],  ols_test[0]],
    "MAE":   [ols_train[1], ols_val[1],  ols_test[1]],
    "MAPE":  [ols_train[2], ols_val[2],  ols_test[2]],
    "R2":    [ols_train[3], ols_val[3],  ols_test[3]],
})

print("\n=== OLS Metrics Table (Train - Val - Test) ===")
display(ols_metrics_table)

# 6. Detailed report + coef/p-value table with FULL VARIABLE NAMES
feature_names = X_all.columns

ols_metrics, ols_coef_df = detailed_regression_report(
    ols,
    Xtr_s,  y_train,   # TRAIN - log(price)
    Xte_s,  y_test,    # TEST  - log(price)
    name="OLS Linear Regression",
    feature_names=feature_names
)

display(ols_coef_df)
```

9.MODEL 2: Lasso Regression

```python
# === LASSO REGRESSION - BASELINE (alpha = 0.0005) ===

from sklearn.linear_model import Lasso

# 0. Get the original variable name (after one-hot)
feature_names = X_all.columns

# 1. Build Lasso baseline
lasso_base = Lasso(
    alpha=0.0005,
    random_state=42,
    max_iter=10000
)

# 2. Train
lasso_base.fit(Xtr_s, y_train)

# 3. Evaluate on TRAIN / VAL / TEST
lasso_base_train = eval_log_model(lasso_base, Xtr_s,  y_train)
lasso_base_val   = eval_log_model(lasso_base, Xval_s, y_val)
lasso_base_test  = eval_log_model(lasso_base, Xte_s,  y_test)

print("=== Lasso Regression (baseline, alpha=0.0005) ===")
print("Train (RMSE, MAE, MAPE, R2):", lasso_base_train)
print("Val   (RMSE, MAE, MAPE, R2):", lasso_base_val)
print("Test  (RMSE, MAE, MAPE, R2):", lasso_base_test)

print("=== Lasso Regression - BASELINE (alpha = 0.0005) ===")
metrics_lasso_base = pd.DataFrame(
    [
        ["Train", *lasso_base_train],
        ["Val",   *lasso_base_val],
        ["Test",  *lasso_base_test],
    ],
    columns=["Split", "RMSE", "MAE", "MAPE", "R2"]
)
display(metrics_lasso_base)
```

```python
# 4. Residuals + Actual vs Predicted

# 4.1 Residuals on TRAIN (original price scale)
y_train_true = np.exp(y_train.values)
y_train_pred = np.exp(lasso_base.predict(Xtr_s).ravel())
train_resid  = y_train_true - y_train_pred

# 4.2 Actual vs Predicted on TEST
y_test_true  = np.exp(y_test.values)
y_pred_lasso = np.exp(lasso_base.predict(Xte_s).ravel())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(train_resid, kde=True, ax=axes[0])
axes[0].set_title("Lasso baseline - Train residuals (alpha = 0.0005)")
axes[0].set_xlabel("Errors")

lims = [min(y_test_true.min(), y_pred_lasso.min()),
        max(y_test_true.max(), y_pred_lasso.max())]
axes[1].scatter(y_test_true, y_pred_lasso, alpha=0.5)
axes[1].plot(lims, lims, "r--")
axes[1].set_title("Lasso baseline - Test Actual vs Predicted")
axes[1].set_xlabel("Actual (Test)")
axes[1].set_ylabel("Predicted")

plt.tight_layout()
plt.show()

# 5. LASSO COEFFICIENT TABLE WITH FULL VARIABLE NAMES
lasso_base_coef_df = pd.DataFrame({
    "variable":   feature_names,
    "coef_lasso": lasso_base.coef_.ravel()
})

print("\n=== Lasso BASELINE coefficients (sklearn Lasso, no p-values) ===")
display(lasso_base_coef_df)
```

9.2 Lasso hyperparameter tuning

```python
# === LASSO REGRESSION – HYPERPARAMETER SEARCH (alpha) ===

from sklearn.linear_model import Lasso

# 0. Variable name (after one-hot)
feature_names = X_all.columns

# 1. List of alphas to try for Lasso
alpha_list = [1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3]

best_alpha    = None
best_val_rmse = np.inf
best_lasso    = None

print("=== Lasso hyperparameter search (validation-based) ===")
for a in alpha_list:
    m = Lasso(alpha=a, random_state=42, max_iter=10000)
    m.fit(Xtr_s, y_train)

    val_rmse, val_mae, val_mape, val_r2 = eval_log_model(m, Xval_s, y_val)
    print(f"alpha = {a:<8} -> Val RMSE = {val_rmse:,.0f}, Val R² = {val_r2:.3f}")

    if val_rmse < best_val_rmse:
        best_val_rmse = val_rmse
        best_alpha    = a
        best_lasso    = m

print("\nBest alpha for Lasso:", best_alpha)
```

```python
# 2. Use the best Lasso model
lasso = best_lasso

lasso_train = eval_log_model(lasso, Xtr_s,  y_train)
lasso_val   = eval_log_model(lasso, Xval_s, y_val)
lasso_test  = eval_log_model(lasso, Xte_s,  y_test)

print("\n=== Final Lasso Regression (after tuning alpha) ===")
print("Train (RMSE, MAE, MAPE, R2):", lasso_train)
print("Val   (RMSE, MAE, MAPE, R2):", lasso_val)
print("Test  (RMSE, MAE, MAPE, R2):", lasso_test)

print("\n=== Final Lasso Regression (after tuning alpha) ===")
metrics_lasso_tuned = pd.DataFrame(
    [
        ["Train", *lasso_train],
        ["Val",   *lasso_val],
        ["Test",  *lasso_test],
    ],
    columns=["Split", "RMSE", "MAE", "MAPE", "R2"]
)
display(metrics_lasso_tuned)
```

```python
# 3. Residuals + Actual vs Predicted (using Lasso tuned)

# 3.1 Residuals TRAIN
y_train_true = np.exp(y_train.values)
y_train_pred = np.exp(lasso.predict(Xtr_s).ravel())
train_resid  = y_train_true - y_train_pred

# 3.2 Actual vs Predicted TEST
y_test_true  = np.exp(y_test.values)
y_pred_lasso = np.exp(lasso.predict(Xte_s).ravel())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(train_resid, kde=True, ax=axes[0])
axes[0].set_title(f"Lasso tuned – Train residuals (alpha = {best_alpha})")
axes[0].set_xlabel("Errors")

lims = [min(y_test_true.min(), y_pred_lasso.min()),
        max(y_test_true.max(), y_pred_lasso.max())]
axes[1].scatter(y_test_true, y_pred_lasso, alpha=0.5)
axes[1].plot(lims, lims, "r--")
axes[1].set_title("Lasso tuned – Test Actual vs Predicted")
axes[1].set_xlabel("Actual (Test)")
axes[1].set_ylabel("Predicted")

plt.tight_layout()
plt.show()

# 4. Lasso tuned coefficient table with full variable names
lasso_coef_df = pd.DataFrame({
    "variable":   feature_names,
    "coef_lasso": lasso.coef_.ravel()
})

print(f"\n=== Lasso coefficients (alpha = {best_alpha}) – sklearn Lasso, no p-values ===")
display(lasso_coef_df)
```

Baseline vs tuning

```python
print("=== Visualization Comparison of Lasso Baseline vs Lasso Tuned ===")

# --- 1. Get the data needed for Lasso Baseline ---
# y_train (price_log) -> y_train_true (original price)
y_train_true_base = np.exp(y_train.values)
y_test_true_base = np.exp(y_test.values)

# Predict on log scale
y_train_pred_log_base = lasso_base.predict(Xtr_s).ravel()
y_test_pred_log_base = lasso_base.predict(Xte_s).ravel()

# Convert to original price scale
y_train_pred_base = np.exp(y_train_pred_log_base)
y_test_pred_base = np.exp(y_test_pred_log_base)

# Calculate Residuals (Train)
train_resid_base = y_train_true_base - y_train_pred_base

# --- 2. Get the data needed for Lasso Tuned ---
# y_train (price_log) -> y_train_true (original price)
y_train_true_tuned = np.exp(y_train.values) # Same as base
y_test_true_tuned = np.exp(y_test.values) # Same as base

# Predict on log scale
y_train_pred_log_tuned = lasso.predict(Xtr_s).ravel()
y_test_pred_log_tuned = lasso.predict(Xte_s).ravel()

# Convert to original price scale
y_train_pred_tuned = np.exp(y_train_pred_log_tuned)
y_test_pred_tuned = np.exp(y_test_pred_log_tuned)

# Calculate Residuals (Train)
train_resid_tuned = y_train_true_tuned - y_train_pred_tuned

# --- 3. Draw a comparison chart ---
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Chart 1: Comparing Histograms of Train Residuals
sns.histplot(train_resid_base, kde=True, ax=axes[0], color='#0077B6', label='Lasso Baseline', alpha=0.5, stat='density')
sns.histplot(train_resid_tuned, kde=True, ax=axes[0], color='#023E8A', label='Lasso Tuned', alpha=0.7, stat='density')
axes[0].set_title('Compare Train Residuals')
axes[0].set_xlabel('Residual (Error)')
axes[0].set_ylabel('Density')
axes[0].legend()
```

```python
# Chart 2: Comparison of Scatter Plot Actual vs Predicted (Test)
axes[1].scatter(y_test_true_base, y_test_pred_base, alpha=0.3, color='#0077B6', label='Lasso Baseline', s=50)
axes[1].scatter(y_test_true_tuned, y_test_pred_tuned, alpha=0.6, color='#023E8A', label='Lasso Tuned', s=50)

# Draw the line y=x
min_val = min(y_test_true_base.min(), y_test_pred_base.min(), y_test_true_tuned.min(), y_test_pred_tuned.min())
max_val = max(y_test_true_base.max(), y_test_pred_base.max(), y_test_true_tuned.max(), y_test_pred_tuned.max())
lims = [min_val, max_val]
axes[1].plot(lims, lims, 'r--', alpha=0.7, label='Ideal road')

axes[1].set_title('Compare Actual vs Forecast(Test)')
axes[1].set_xlabel('Actual Price')
axes[1].set_ylabel('Predicted Price')
axes[1].legend()
axes[1].ticklabel_format(style='plain', axis='x')
axes[1].ticklabel_format(style='plain', axis='y')

plt.tight_layout()
plt.show()
```

10. Evaluation function common to all tree-based models

```
[ ]
from sklearn.model_selection import RandomizedSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# XGBoost
from xgboost import XGBRegressor


def run_tree_experiment(exp_name, model,
                        Xtr=Xtr_s, Xval=Xval_s, Xte=Xte_s,
                        ytr=y_train, yval=y_val, yte=y_test):
    model.fit(Xtr, ytr)

    tr = eval_log_model(model, Xtr, ytr)
    va = eval_log_model(model, Xval, yval)
    te = eval_log_model(model, Xte, yte)

    print(f"\n=== {exp_name} ===")
    print("Train (RMSE, MAE, MAPE, R2):", tr)
    print("Val   (RMSE, MAE, MAPE, R2):", va)
    print("Test  (RMSE, MAE, MAPE, R2):", te)

    return model, tr, va, te

# Assume X_all is the full feature DataFrame (after one-hot, before scaling)
feature_names = X_all.columns

def plot_feature_importance(model, feature_names, top_n=15, title="Feature importance"):
    importances = getattr(model, "feature_importances_", None)
    if importances is None:
        print("⚠️ Model does not have feature_importances_ attribute.")
        return

    importances = np.array(importances)
    idx_sorted = np.argsort(importances)[::-1]
    idx_top = idx_sorted[:top_n]

    fi_df = pd.DataFrame({
        "feature": np.array(feature_names)[idx_top],
        "importance": importances[idx_top]
    })

    print("=== Top feature importances ===")
    display(fi_df)

    plt.figure(figsize=(8, max(4, top_n * 0.3)))
    sns.barplot(data=fi_df, x="importance", y="feature")
    plt.title(title)
    plt.xlabel("Importance")
    plt.ylabel("Feature")
    plt.tight_layout()
    plt.show()
```

10.1 Models 3:Decision Tree

```python
# === DECISION TREE REGRESSOR - BASELINE ===

dt_results = []

dt_base = DecisionTreeRegressor(
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42
)

dt_base, dt_base_tr, dt_base_val, dt_base_te = run_tree_experiment(
    "DT_exp0_baseline", dt_base
)

dt_results.append(["DT_exp0_baseline",
                   *dt_base_tr, *dt_base_val, *dt_base_te])

cols_tree = ["Experiment",
             "RMSE_train","MAE_train","MAPE_train","R2_train",
             "RMSE_val","MAE_val","MAPE_val","R2_val",
             "RMSE_test","MAE_test","MAPE_test","R2_test"]

df_dt = pd.DataFrame(dt_results, columns=cols_tree)

print("\n>>> Decision Tree - Baseline only:")
display(df_dt)
```

```
=== DT_exp0_baseline ===
Train (RMSE, MAE, MAPE, R2): (np.float64(47868.88363364121), 4180.116579792286, np.float64(0.16613189921308183), 0.9992253117106122)
Val   (RMSE, MAE, MAPE, R2): (np.float64(1382829.3689273673), 986952.1405075374, np.float64(20.705523103785087), 0.32561824147944485)
Test  (RMSE, MAE, MAPE, R2): (np.float64(1550378.695952273), 1156170.341942103, np.float64(25.039401207074473), 0.4648740322689108)
```

10.2. Decision tree regressor 3 Experiments

```python
# === DECISION TREE REGRESSOR - 3 EXPERIMENTS ===
# (continue using dt_results which already has baseline)

# EXP 1: Smaller depth to avoid overfitting
dt_exp1 = DecisionTreeRegressor(
    max_depth=5,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)
dt_exp1, dt1_tr, dt1_val, dt1_te = run_tree_experiment(
    "DT_exp1_depth5", dt_exp1
)
dt_results.append(["DT_exp1_depth5",
                   *dt1_tr, *dt1_val, *dt1_te])

# EXP 2: depth 7, leaf small
dt_exp2 = DecisionTreeRegressor(
    max_depth=7,
    min_samples_split=5,
    min_samples_leaf=1,
    random_state=42
)
dt_exp2, dt2_tr, dt2_val, dt2_te = run_tree_experiment(
    "DT_exp2_depth7", dt_exp2
)
dt_results.append(["DT_exp2_depth7",
                   *dt2_tr, *dt2_val, *dt2_te])

# EXP 3: depth 4, leaf bigger
dt_exp3 = DecisionTreeRegressor(
    max_depth=4,
    min_samples_split=10,
    min_samples_leaf=4,
    random_state=42
)
dt_exp3, dt3_tr, dt3_val, dt3_te = run_tree_experiment(
    "DT_exp3_depth4_leaf4", dt_exp3
)
dt_results.append(["DT_exp3_depth4_leaf4",
                   *dt3_tr, *dt3_val, *dt3_te])

df_dt = pd.DataFrame(dt_results, columns=cols_tree)

print("\n>>> Decision Tree - Baseline + 3 experiments:")
display(df_dt.sort_values("R2_test", ascending=False))
```

10.3. Decision Tree RandomizedSearchCV

```python
# === DECISION TREE REGRESSOR - RANDOMIZED SEARCH ===

dt_param_dist = {
    "max_depth":        [3, 4, 5, 6, 8, None],
    "min_samples_split":[2, 5, 10, 15],
    "min_samples_leaf": [1, 2, 4, 6],
    "max_features":     ["auto", "sqrt", 0.5, None]
}

dt_base_rs = DecisionTreeRegressor(random_state=42)

dt_rand = RandomizedSearchCV(
    estimator=dt_base_rs,
    param_distributions=dt_param_dist,
    n_iter=30,
    scoring="neg_mean_squared_error",
    cv=5,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

dt_rand.fit(Xtr_s, y_train)

print("\nBest params (Decision Tree):", dt_rand.best_params_)

dt_best = dt_rand.best_estimator_

dt_best_tr  = eval_log_model(dt_best, Xtr_s,  y_train)
dt_best_val = eval_log_model(dt_best, Xval_s, y_val)
dt_best_te  = eval_log_model(dt_best, Xte_s,  y_test)

exp_name = "DT_exp4_randomSearch"
dt_results = [row for row in dt_results if row[0] != exp_name]

dt_results.append([exp_name,
                   *dt_best_tr, *dt_best_val, *dt_best_te])

df_dt = pd.DataFrame(dt_results, columns=cols_tree)

print("\n>>> Decision Tree - All (baseline + 3 exp + randomSearch):")
display(df_dt.sort_values("R2_test", ascending=False))
```

10.4. Select best Decision Tree + Feature importance

```python
# === DECISION TREE - CHỌN BEST MODEL & FEATURE IMPORTANCE ===

# df_dt phải tồn tại từ cell 3
best_row_dt = df_dt.loc[df_dt["R2_test"].idxmax()]
best_dt_name = best_row_dt["Experiment"]

print(">>> Best Decision Tree experiment based on Test R²:")
display(best_row_dt.to_frame().T)

# map tên → model object
dt_model_dict = {
    "DT_exp0_baseline":        dt_base,
    "DT_exp1_depth5":          dt_exp1,
    "DT_exp2_depth7":          dt_exp2,
    "DT_exp3_depth4_leaf4":    dt_exp3,
    "DT_exp4_randomSearch":    dt_best
}

dt_best_model = dt_model_dict[best_dt_name]

# Feature importance cho best Decision Tree
plot_feature_importance(
    dt_best_model,
    feature_names,
    top_n=15,
    title=f"Decision Tree - Feature importance ({best_dt_name})"
)
```

11. Model 4: RANDOM FOREST

```python
# === RANDOM FOREST REGRESSOR - BASELINE ===

rf_results = []

rf_base = RandomForestRegressor(
    n_estimators=200,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)
rf_base, rf0_tr, rf0_val, rf0_te = run_tree_experiment(
    "RF_exp0_baseline", rf_base
)
rf_results.append(["RF_exp0_baseline",
                   *rf0_tr, *rf0_val, *rf0_te])

df_rf = pd.DataFrame(rf_results, columns=cols_tree)

print("\n>>> Random Forest - Baseline only:")
display(df_rf)
```

```
=== RF_exp0_baseline ===
Train (RMSE, MAE, MAPE, R2): (np.float64(411780.6720455246), 275531.3443775459, np.float64(5.882315869171846), 0.9426739116634244)
Val   (RMSE, MAE, MAPE, R2): (np.float64(1032371.1833369837), 750987.6872477274, np.float64(15.890275938557021), 0.6241276560223566)
Test  (RMSE, MAE, MAPE, R2): (np.float64(1329329.4580293691), 992649.0156948919, np.float64(20.99284574984967), 0.6065897237785774)
```

11.2. Random Forest 3 experiments

```python
# === RANDOM FOREST REGRESSOR - 3 EXPERIMENTS ===

# EXP 1
rf_exp1 = RandomForestRegressor(
    n_estimators=300,
    max_depth=8,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42,
    n_jobs=-1
)
rf_exp1, rf1_tr, rf1_val, rf1_te = run_tree_experiment(
    "RF_exp1_depth8", rf_exp1
)
rf_results.append(["RF_exp1_depth8",
                   *rf1_tr, *rf1_val, *rf1_te])

# EXP 2
rf_exp2 = RandomForestRegressor(
    n_estimators=500,
    max_depth=12,
    min_samples_split=5,
    min_samples_leaf=1,
    random_state=42,
    n_jobs=-1
)
rf_exp2, rf2_tr, rf2_val, rf2_te = run_tree_experiment(
    "RF_exp2_depth12", rf_exp2
)
rf_results.append(["RF_exp2_depth12",
                   *rf2_tr, *rf2_val, *rf2_te])

# EXP 3
rf_exp3 = RandomForestRegressor(
    n_estimators=300,
    max_depth=6,
    min_samples_split=10,
    min_samples_leaf=4,
    random_state=42,
    n_jobs=-1
)
rf_exp3, rf3_tr, rf3_val, rf3_te = run_tree_experiment(
    "RF_exp3_depth6_reg", rf_exp3
)
rf_results.append(["RF_exp3_depth6_reg",
                   *rf3_tr, *rf3_val, *rf3_te])

df_rf = pd.DataFrame(rf_results, columns=cols_tree)

print("\n>>> Random Forest - Baseline + 3 experiments:")
display(df_rf.sort_values("R2_test", ascending=False))
```

11.3. Random forest RandomizedSearchCV

```python
# === RANDOM FOREST REGRESSOR - RANDOMIZED SEARCH ===

rf_param_dist = {
    "n_estimators":    [100, 200, 300, 500],
    "max_depth":       [3, 5, 7, 9, None],
    "min_samples_split":[2, 5, 10],
    "min_samples_leaf":[1, 2, 4],
    "max_features":    ["auto", "sqrt", 0.5]
}

rf_base_rs = RandomForestRegressor(
    random_state=42,
    n_jobs=-1
)

rf_rand = RandomizedSearchCV(
    estimator=rf_base_rs,
    param_distributions=rf_param_dist,
    n_iter=30,
    scoring="neg_mean_squared_error",
    cv=5,
    n_jobs=-1,
    random_state=42,
    verbose=1
)

rf_rand.fit(Xtr_s, y_train)

print("\nBest params (Random Forest):", rf_rand.best_params_)

rf_best = rf_rand.best_estimator_

rf_best_tr  = eval_log_model(rf_best, Xtr_s,  y_train)
rf_best_val = eval_log_model(rf_best, Xval_s, y_val)
rf_best_te  = eval_log_model(rf_best, Xte_s,  y_test)

exp_name = "RF_exp4_randomSearch"
rf_results = [row for row in rf_results if row[0] != exp_name]

rf_results.append([exp_name,
                   *rf_best_tr, *rf_best_val, *rf_best_te])

df_rf = pd.DataFrame(rf_results, columns=cols_tree)

print("\n>>> Random Forest - All (baseline + 3 exp + randomSearch):")
display(df_rf.sort_values("R2_test", ascending=False))
```

11.4. Select Best RF + Feature importance

```python
# === RANDOM FOREST - CHỌN BEST MODEL & FEATURE IMPORTANCE ===

best_row_rf = df_rf.loc[df_rf["R2_test"].idxmax()]
best_rf_name = best_row_rf["Experiment"]

print(">>> Best Random Forest experiment based on Test R²:")
display(best_row_rf.to_frame().T)

rf_model_dict = {
    "RF_exp0_baseline":     rf_base,
    "RF_exp1_depth8":       rf_exp1,
    "RF_exp2_depth12":      rf_exp2,
    "RF_exp3_depth6_reg":   rf_exp3,
    "RF_exp4_randomSearch": rf_best
}

rf_best_model = rf_model_dict[best_rf_name]

plot_feature_importance(
    rf_best_model,
    feature_names,
    top_n=15,
    title=f"Random Forest - Feature importance ({best_rf_name})"
)
```

12. Model 5: XGBOOST

```
# === XGBOOST REGRESSOR - BASELINE ===

xgb_results = []

xgb_base = XGBRegressor(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)
xgb_base, xgb0_tr, xgb0_val, xgb0_te = run_tree_experiment(
    "XGB_exp0_baseline", xgb_base
)
xgb_results.append(["XGB_exp0_baseline",
                    *xgb0_tr, *xgb0_val, *xgb0_te])

df_xgb = pd.DataFrame(xgb_results, columns=cols_tree)

print("\n>>> XGBoost - Baseline only:")
display(df_xgb)
```

12.2. XGBoost 3 experiments

```
# === XGBOOST REGRESSOR - 3 EXPERIMENTS ===
# EXP1: deeper
xgb_exp1 = XGBRegressor(
    n_estimators=500,
    max_depth=6,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)
xgb_exp1, xgb1_tr, xgb1_val, xgb1_te = run_tree_experiment(
    "XGB_exp1_depth6", xgb_exp1
)
xgb_results.append(["XGB_exp1_depth6",
                    *xgb1_tr, *xgb1_val, *xgb1_te])
# EXP2: small lr, many estimators
xgb_exp2 = XGBRegressor(
    n_estimators=700,
    max_depth=5,
    learning_rate=0.03,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1)
xgb_exp2, xgb2_tr, xgb2_val, xgb2_te = run_tree_experiment(
    "XGB_exp2_lr0.03", xgb_exp2)
xgb_results.append(["XGB_exp2_lr0.03",
                    *xgb2_tr, *xgb2_val, *xgb2_te])
# EXP3: regularization
xgb_exp3 = XGBRegressor(
    n_estimators=500,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1)
xgb_exp3, xgb3_tr, xgb3_val, xgb3_te = run_tree_experiment(
    "XGB_exp3_reg", xgb_exp3)
xgb_results.append(["XGB_exp3_reg",
                    *xgb3_tr, *xgb3_val, *xgb3_te])
df_xgb = pd.DataFrame(xgb_results, columns=cols_tree)
print("\n>>> XGBoost - Baseline + 3 experiments:")
display(df_xgb.sort_values("R2_test", ascending=False))
```

12.3. XGBoost RandomizedSearchCV

```python
# === XGBOOST REGRESSOR - RANDOMIZED SEARCH (TUNED) ===

xgb_param_dist = {
    "n_estimators": [250, 300, 350, 400], # Focused around previous best 300
    "max_depth":    [2, 3, 4],           # Focused around previous best 3
    "learning_rate":[0.03, 0.05, 0.07], # Focused around previous best 0.05
    "subsample":    [0.5, 0.6, 0.7],     # Focused around previous best 0.6
    "colsample_bytree":[0.5, 0.6, 0.7] # Focused around previous best 0.6
}
xgb_base_rs = XGBRegressor(
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1,
    tree_method="hist")
xgb_rand = RandomizedSearchCV(
    estimator=xgb_base_rs,
    param_distributions=xgb_param_dist,
    n_iter=30,
    scoring="neg_mean_squared_error",
    cv=5,
    n_jobs=-1,
    random_state=42,
    verbose=1)
xgb_rand.fit(Xtr_s, y_train)
print("\nBest params (XGBoost TUNED):", xgb_rand.best_params_)

xgb_best = xgb_rand.best_estimator_

xgb_best_tr  = eval_log_model(xgb_best, Xtr_s,  y_train)
xgb_best_val = eval_log_model(xgb_best, Xval_s, y_val)
xgb_best_te  = eval_log_model(xgb_best, Xte_s,  y_test)
exp_name_tuned = "XGB_exp4_randomSearch_tuned"
prev_xgb_exp4_results = None
for row in xgb_results:
    if row[0] == old_exp_name:
        prev_xgb_exp4_results = row
        break
xgb_results = [row for row in xgb_results if row[0] != exp_name_tuned]
if prev_xgb_exp4_results is not None:
    if prev_xgb_exp4_results[0] == old_exp_name and prev_xgb_exp4_results not in xgb_results:
        xgb_results.append(prev_xgb_exp4_results)
xgb_results.append([exp_name_tuned,
                    *xgb_best_tr, *xgb_best_val, *xgb_best_te])
df_xgb = pd.DataFrame(xgb_results, columns=cols_tree)
print("\n>>> XGBoost - All (baseline + 3 exp + randomSearch):")
display(df_xgb.sort_values("R2_test", ascending=False))
```

12.4. Chọn Best XGBoost + Feature importance

```python
# === XGBOOST - CHỌN BEST MODEL & FEATURE IMPORTANCE ===

best_row_xgb = df_xgb.loc[df_xgb["R2_test"].idxmax()]
best_xgb_name = best_row_xgb["Experiment"]
print(">>> Best XGBoost experiment based on Test R²:")
display(best_row_xgb.to_frame().T)
xgb_model_dict = {
    "XGB_exp0_baseline":     xgb_base,
    "XGB_exp1_depth6":       xgb_exp1,
    "XGB_exp2_lr0.03":       xgb_exp2,
    "XGB_exp3_reg":          xgb_exp3,
    "XGB_exp4_randomSearch_tuned": xgb_best
}
xgb_best_model = None
if best_xgb_name == "XGB_exp4_randomSearch_tuned":
    xgb_best_model = xgb_best
elif best_xgb_name in xgb_model_dict:
    xgb_best_model = xgb_model_dict[best_xgb_name]
else:
    print(f"Warning: Model object for '{best_xgb_name}' not directly available. Falling back to the currently best-tuned model 'xgb_best' for feature importance.")
    xgb_best_model = xgb_best # Fallback to the latest tuned model
if xgb_best_model is not None:
    plot_feature_importance(
        xgb_best_model,
        feature_names,
        top_n=15,
        title=f"XGBoost - Feature importance ({best_xgb_name})"
    )
else:
    print("Error: No valid XGBoost model object could be retrieved for feature importance plotting.")
```

13. Evaluation function & drawing function (common for all 2 models - ANN, GMDH-like)

```
def eval_log_model(model, Xmat, y_true_log):
    y_pred_log = model.predict(Xmat).ravel()
    y_true = np.exp(y_true_log.values)
    y_pred = np.exp(y_pred_log)

    mse  = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae  = mean_absolute_error(y_true, y_pred)
    mape = np.mean(np.abs((y_true - y_pred) / np.clip(y_true, 1e-9, None))) * 100
    r2   = r2_score(y_true, y_pred)
    return rmse, mae, mape, r2

def plot_pred_vs_actual(y_true, y_pred, title):
    plt.figure(figsize=(4,4))
    plt.scatter(y_true, y_pred, alpha=0.5)
    lims = [min(y_true.min(), y_pred.min()), max(y_true.max(), y_pred.max())]
    plt.plot(lims, lims, 'r--')
    plt.xlabel("Actual price")
    plt.ylabel("Predicted price")
    plt.title(title)
    plt.tight_layout()
    plt.show()
```

14.1. Hàm build & run ANN (dùng cho mọi experiment)

```
from tensorflow.keras import backend as K
def build_ann_model(input_dim,
                    n_layers=2,
                    units_per_layer=[64, 32],
                    activation='relu',
                    dropout_rate=0.0,
                    learning_rate=1e-3,
                    optimizer_name='adam'):

    model = keras.Sequential()
    # Hidden layer 1
    model.add(
        layers.Dense(units_per_layer[0],
                     activation=activation,
                     input_shape=(input_dim,))
    )
    if dropout_rate > 0:
        model.add(layers.Dropout(dropout_rate))
    for i in range(1, n_layers):
        model.add(
            layers.Dense(units_per_layer[i],
                         activation=activation)
        )
        if dropout_rate > 0:
            model.add(layers.Dropout(dropout_rate))

    model.add(layers.Dense(1))
    if optimizer_name.lower() == 'adam':
        opt = keras.optimizers.Adam(learning_rate=learning_rate)
    elif optimizer_name.lower() == 'rmsprop':
        opt = keras.optimizers.RMSprop(learning_rate=learning_rate)
    else:
        opt = keras.optimizers.Adam(learning_rate=learning_rate)

    model.compile(
        optimizer=opt,
        loss='mse',
        metrics=['mae']
    )
    return model
```

```python
def run_ann_experiment(name,
                       n_layers,
                       units_per_layer,
                       activation,
                       dropout_rate,
                       batch_size,
                       learning_rate,
                       optimizer_name,
                       max_epochs=1000,
                       patience=30):
    print(f"\n================ {name} ================")
    print("Hyperparams:",
          f"n_layers={n_layers}, units={units_per_layer},",
          f"act={activation}, dropout={dropout_rate},",
          f"batch={batch_size}, lr={learning_rate}, opt={optimizer_name}")
    input_dim = Xtr_s.shape[1]
    # 1. Build model
    model = build_ann_model(
        input_dim=input_dim,
        n_layers=n_layers,
        units_per_layer=units_per_layer,
        activation=activation,
        dropout_rate=dropout_rate,
        learning_rate=learning_rate,
        optimizer_name=optimizer_name
    )
    model.summary()
    # 2. EarlyStopping
    es = keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=patience,
        restore_best_weights=True
    )
    # 3. Train
    history = model.fit(
        Xtr_s, y_train,
        validation_data=(Xval_s, y_val),
        epochs=max_epochs,
        batch_size=batch_size,
        callbacks=[es],
        verbose=0
    )
    print("Stopped at epoch:", len(history.history['loss']))
```

```python
    # 4. Evaluate on train / val / test (using common eval_log_model)
    ann_train = eval_log_model(model, Xtr_s,  y_train)
    ann_val   = eval_log_model(model, Xval_s, y_val)
    ann_test  = eval_log_model(model, Xte_s,  y_test)

    print("Train (RMSE, MAE, MAPE, R2):", ann_train)
    print("Val   (RMSE, MAE, MAPE, R2):", ann_val)
    print("Test  (RMSE, MAE, MAPE, R2):", ann_test)

    # 5. Combine Plot Actual vs Predicted - Test and Loss curve into one figure
    fig, axes = plt.subplots(1, 2, figsize=(14, 5)) # 1 row, 2 columns
    # Plot 1: Actual vs Predicted - Test
    y_test_true = np.exp(y_test.values)
    y_pred      = np.exp(model.predict(Xte_s).ravel())
    axes[0].scatter(y_test_true, y_pred, alpha=0.5)
    lims = [min(y_test_true.min(), y_pred.min()),
            max(y_test_true.max(), y_pred.max())]
    axes[0].plot(lims, lims, 'r--')
    axes[0].set_title(f"{name} - Actual vs Predicted (Test)")
    axes[0].set_xlabel("Actual price")
    axes[0].set_ylabel("Predicted price")

    # Plot 2: Loss curve
    axes[1].plot(history.history['loss'], label='Train loss')
    axes[1].plot(history.history['val_loss'], label='Val loss')
    axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('MSE loss')
    axes[1].set_title(f'{name} Training Curve')
    axes[1].legend()
    plt.tight_layout()
    plt.show()
    return model, ann_train, ann_val, ann_test, history
```

14.2. Models 6: ANN Baseline

```
# === ANN BASELINE (Exp0) ===

# ann_results = [] # Reset list if rerunning from scratch

ann_base, ann_base_tr, ann_base_val, ann_base_te, ann_base_history = run_ann_experiment(
    name           ="ANN_exp0_baseline",
    n_layers       =2,
    units_per_layer=[64, 32],
    activation     ="relu",
    dropout_rate   =0.0,
    batch_size     =32,
    learning_rate  =1e-3,
    optimizer_name ="adam",
    max_epochs     =1000,
    patience       =30
)
```

14.3. ANN Experiment 1–3 (manual tuning)

```
# === ANN EXPERIMENT 1 – mạng nhỏ, đơn giản, ít overfitting hơn ===
ann_exp1, ann1_tr, ann1_val, ann1_te, ann1_history = run_ann_experiment(
    name           ="ANN_exp1_small",
    n_layers       =1,
    units_per_layer=[32],
    activation     ="relu",
    dropout_rate   =0.0,
    batch_size     =32,
    learning_rate  =1e-3,
    optimizer_name ="adam",
    max_epochs     =500,
    patience       =30
)

# === ANN EXPERIMENT 2 – mạng lớn hơn, thêm dropout để regularize ===
ann_exp2, ann2_tr, ann2_val, ann2_te, ann2_history = run_ann_experiment(
    name           ="ANN_exp2_medium_dropout",
    n_layers       =2,
    units_per_layer=[64, 32],
    activation     ="relu",
    dropout_rate   =0.2,
    batch_size     =32,
    learning_rate  =5e-4,
    optimizer_name ="adam",
    max_epochs     =800,
    patience       =40
)

# === ANN EXPERIMENT 3 – thử activation 'tanh' + dropout mạnh hơn ===
ann_exp3, ann3_tr, ann3_val, ann3_te, ann3_history = run_ann_experiment(
    name           ="ANN_exp3_tanh_stronger_dropout",
    n_layers       =2,
    units_per_layer=[64, 32],
    activation     ="tanh",
    dropout_rate   =0.3,
    batch_size     =16,
    learning_rate  =5e-4,
    optimizer_name ="adam",
    max_epochs     =800,
    patience       =40
)
```

14.4. Choose the best ANN to include in the final comparison table

```python
# 1. Combine the results of 4 experiments into 1 dict
results_ann = {
    "ANN_exp0_baseline":              ann_base_te,
    "ANN_exp1_small":                 ann1_te,
    "ANN_exp2_medium_dropout":        ann2_te,
    "ANN_exp3_tanh_stronger_dropout": ann3_te,
}

# 2. Convert to DataFrame for readability
rows = []
for name, (rmse, mae, mape, r2) in results_ann.items():
    rows.append({
        "Experiment": name,
        "RMSE": float(rmse),
        "MAE":  float(mae),
        "MAPE": float(mape),
        "R2":   float(r2)
    })

df_ann_results = (
    pd.DataFrame(rows)
      .sort_values("R2", ascending=False)
      .reset_index(drop=True)
)

display(df_ann_results)

best_row = df_ann_results.loc[df_ann_results["R2"].idxmax()]
best_exp_name = best_row["Experiment"]

print(">>> Best ANN experiment based on Test R²:")
display(best_row.to_frame().T)
```

15.1.GMDH-liked build & run function

```python
def gmdh_diagnostics(model, poly,
                     Xtr_s, y_train,
                     Xte_s, y_test,
                     title="GMDH-like"):
    # 1. Poly transformation
    Xtr_g = poly.transform(Xtr_s)
    Xte_g = poly.transform(Xte_s)
    # 2. Predict on log scale, then back-transform
    y_tr_log_pred = model.predict(Xtr_g)
    y_te_log_pred = model.predict(Xte_g)
    y_tr_true = np.exp(y_train.values)
    y_te_true = np.exp(y_test.values)
    y_tr_pred = np.exp(y_tr_log_pred)
    y_te_pred = np.exp(y_te_log_pred)
    # 3. Residuals train
    resid_tr = y_tr_true - y_tr_pred
    # 4. Plot
    fig, axes = plt.subplots(1, 2, figsize=(12,4))
    # 4.1 Histogram residuals (Train)
    sns.histplot(resid_tr, kde=True, ax=axes[0])
    axes[0].set_title(f"{title} - Train residuals")
    axes[0].set_xlabel("Error (Actual - Predicted)")
    # 4.2 Actual vs Predicted (Test)
    axes[1].scatter(y_te_true, y_te_pred, alpha=0.5)
    lims = [min(y_te_true.min(), y_te_pred.min()),
            max(y_te_true.max(), y_te_pred.max())]
    axes[1].plot(lims, lims, 'r--')
    axes[1].set_title(f"{title} - Test vs Predicted")
    axes[1].set_xlabel("Actual (Test)")
    axes[1].set_ylabel("Predicted")
    plt.tight_layout()
    plt.show()
```

```python
    plt.show()
def run_gmdh_like_single(name,
                         degree=2,
                         alpha=0.001,
                         max_iter=20000):
    print(f"\n================= {name} =================")
    print(f"degree={degree}, alpha={alpha}")
    # 1. Polynomial transform
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    Xtr_gmdh  = poly.fit_transform(Xtr_s)
    Xval_gmdh = poly.transform(Xval_s)
    Xte_gmdh  = poly.transform(Xte_s)
    print("Poly feature shape (train):", Xtr_gmdh.shape)
    # 2. Lasso on poly features (self-organizing GMDH style)
    gmdh_model = Lasso(alpha=alpha,
                       max_iter=max_iter,
                       random_state=42)
    gmdh_model.fit(Xtr_gmdh, y_train)
    # 3.Evaluate on Train / Val / Test (original price scale)
    gmdh_train = eval_log_model(gmdh_model, Xtr_gmdh,  y_train)
    gmdh_val   = eval_log_model(gmdh_model, Xval_gmdh, y_val)
    gmdh_test  = eval_log_model(gmdh_model, Xte_gmdh,  y_test)
    print("Train (RMSE, MAE, MAPE, R2):", gmdh_train)
    print("Val   (RMSE, MAE, MAPE, R2):", gmdh_val)
    print("Test  (RMSE, MAE, MAPE, R2):", gmdh_test)
    # 4. Plot Actual vs Predicted - Test
    y_test_true = np.exp(y_test.values)
    y_pred_gmdh = np.exp(gmdh_model.predict(Xte_gmdh))
    plot_pred_vs_actual(y_test_true, y_pred_gmdh,
                        f"{name} - Actual vs Predicted (Test)")
    # 5. Residual diagnostics (Train residuals + Test vs Predicted)
    gmdh_diagnostics(gmdh_model, poly,
                     Xtr_s, y_train,
                     Xte_s, y_test,
                     title=name)

    return poly, gmdh_model, gmdh_train, gmdh_val, gmdh_test
```

15.2. GMDH-like Baseline

```python
# GMDH-like - BASELINE
poly_g_base, gmdh_base_model, gmdh_base_tr, gmdh_base_val, gmdh_base_te = run_gmdh_like_single(
    name   ="GMDH_like_exp0_baseline",
    degree =2,
    alpha  =0.001
)
```

```
================= GMDH_like_exp0_baseline =================
degree=2, alpha=0.001
Poly feature shape (train): (348, 104)
Train (RMSE, MAE, MAPE, R2): (np.float64(681542.84185427), 493136.5756379357, np.float64(11.172543452549121), 0.8429612918043006)
Val   (RMSE, MAE, MAPE, R2): (np.float64(1183252.9213250747), 873020.5443319354, np.float64(18.594592678372944), 0.5062310278378049)
Test  (RMSE, MAE, MAPE, R2): (np.float64(1408442.49054899), 1073255.9355208373, np.float64(22.946406109239238), 0.5583698861623536)
```

GMDH-like Experiment 1–3 (manual tuning)

```python
gmdh_results_list = [] # Initialize a list to store all GMDH results for a DataFrame later

# GMDH-like - EXPERIMENT 1 (alpha lớn hơn → ít term hơn, giảm overfitting)
poly_g1, gmdh1_model, gmdh1_tr, gmdh1_val, gmdh1_te = run_gmdh_like_single(
    name   ="GMDH_like_exp1_alpha_strong",
    degree =2,
    alpha  =0.01
)
gmdh_results_list.append(["GMDH_like_exp1_alpha_strong", *gmdh1_tr, *gmdh1_val, *gmdh1_te])

# GMDH-like - EXPERIMENT 2 (alpha nhỏ hơn → nhiều term đa thức hơn)
poly_g2, gmdh2_model, gmdh2_tr, gmdh2_val, gmdh2_te = run_gmdh_like_single(
    name   ="GMDH_like_exp2_alpha_small",
    degree =2,
    alpha  =0.0001
)
gmdh_results_list.append(["GMDH_like_exp2_alpha_small", *gmdh2_tr, *gmdh2_val, *gmdh2_te])

# GMDH-like - EXPERIMENT 3 (degree=3, alpha điều chỉnh)
poly_g3, gmdh3_model, gmdh3_tr, gmdh3_val, gmdh3_te = run_gmdh_like_single(
    name   ="GMDH_like_exp3_deg3",
    degree =3,
    alpha  =0.001
)
gmdh_results_list.append(["GMDH_like_exp3_deg3", *gmdh3_tr, *gmdh3_val, *gmdh3_te])

# Re-introducing the best manual tuned result
poly_g_manual_tuned, gmdh_manual_tuned_model, gmdh_manual_tuned_tr, gmdh_manual_tuned_val, gmdh_manual_tuned_te = run_gmdh_like_single(
    name   ="GMDH_like_manual_tuned_best",
    degree =2,
    alpha  =0.012 # This alpha value produced the R2 = 0.633924
)
```

15.4. Select the best GMDH-like to include in the final comparison table

```python
# COLLECT THE RESULTS OF GMDH-LIKE EXPERIMENTS (based on Test R²)
results_gmdh_test = {
    "GMDH_like_exp0_baseline":        gmdh_base_te,
    "GMDH_like_exp1_alpha_strong":    gmdh1_te,
    "GMDH_like_exp2_alpha_small":     gmdh2_te,
    "GMDH_like_exp3_deg3":            gmdh3_te,
    "GMDH_like_manual_tuned_best":    gmdh_manual_tuned_te, # Re-introduced the best tuned model
}
rows = []
for name, (rmse, mae, mape, r2) in results_gmdh_test.items():
    rows.append({
        "Experiment": name,
        "RMSE": float(rmse),
        "MAE":  float(mae),
        "MAPE": float(mape),
        "R2":   float(r2)
    })
df_gmdh_results = (pd.DataFrame(rows)
                   .sort_values("R2", ascending=False)
                   .reset_index(drop=True))
display(df_gmdh_results)

best_gmdh_row  = df_gmdh_results.loc[df_gmdh_results["R2"].idxmax()]
best_gmdh_name = best_gmdh_row["Experiment"]
# Update gmdh_best_train, gmdh_best_val, gmdh_best_test so LguHNCVElPHq can use the actual best GMDH
if best_gmdh_name == "GMDH_like_exp0_baseline":
    gmdh_best_train, gmdh_best_val, gmdh_best_test = gmdh_base_tr, gmdh_base_val, gmdh_base_te
elif best_gmdh_name == "GMDH_like_exp1_alpha_strong":
    gmdh_best_train, gmdh_best_val, gmdh_best_test = gmdh1_tr, gmdh1_val, gmdh1_te
elif best_gmdh_name == "GMDH_like_exp2_alpha_small":
    gmdh_best_train, gmdh_best_val, gmdh_best_test = gmdh2_tr, gmdh2_val, gmdh2_te
elif best_gmdh_name == "GMDH_like_exp3_deg3":
    gmdh_best_train, gmdh_best_val, gmdh_best_test = gmdh3_tr, gmdh3_val, gmdh3_te
elif best_gmdh_name == "GMDH_like_manual_tuned_best": # This is the re-introduced best model
    gmdh_best_train, gmdh_best_val, gmdh_best_test = gmdh_manual_tuned_tr, gmdh_manual_tuned_val, gmdh_manual_tuned_te
print(">>> Best GMDH-like test from manual refinement based on Test R²:") # Updated text
display(best_gmdh_row.to_frame())
```

16. Comparing the Evaluation Metics of the Models

Comparing the Evaluation Metics of the Models (OLS, XGBoost, GMDH-like)

```
# Define the models to be included in the final comparison
chosen_models = ['OLS', 'XGBoost', 'GMDH']
# Filter df_metrics to include only the chosen best models
df_best_models_metrics = df_metrics[df_metrics['Model'].isin(chosen_models)].copy()
# 1. Display Full Metrics Table for Best Models
print("=== FULL METRICS TABLE FOR BEST MODELS (OLS, XGBoost, GMDH-like) ===")
display(df_best_models_metrics)
# 2. Display Model Ranking by Test RMSE
df_best_test_ranking = df_best_models_metrics[df_best_models_metrics["Split"] == "Test"].copy()
df_best_test_ranking = df_best_test_ranking.sort_values("RMSE").reset_index(drop=True)
print("\n=== MODEL RANKING (BY TEST RMSE - thấp hơn là tốt hơn) FOR BEST MODELS ===")
display(df_best_test_ranking)
# 3. Display Generalization Gap Analysis for Best Models
df_best_train = df_best_models_metrics[df_best_models_metrics["Split"]=="Train"].set_index("Model")
df_best_test_idx = df_best_models_metrics[df_best_models_metrics["Split"]=="Test"].set_index("Model")
df_best_gap = pd.DataFrame({
    "RMSE_train": df_best_train["RMSE"],
    "RMSE_test":  df_best_test_idx["RMSE"],
    "RMSE_diff":  df_best_test_idx["RMSE"] - df_best_train["RMSE"],
    "R2_train":   df_best_train["R2"],
    "R2_test":    df_best_test_idx["R2"],
    "R2_diff":    df_best_train["R2"] - df_best_test_idx["R2"]
})
print("\n=== GENERALIZATION GAP (TRAIN vs TEST) FOR BEST MODELS ===")
display(df_best_gap)
# 4. Visualize Test RMSE Comparison
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_test_ranking, x="Model", y="RMSE")
plt.title("Test RMSE Comparison Across Best Models (OLS, XGBoost, GMDH-like)")
plt.ylabel("RMSE")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
# 5. Visualize Test R² Comparison
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_test_ranking, x="Model", y="R2")
plt.title("Test R² Comparison Across Best Models (OLS, XGBoost, GMDH-like)")
plt.ylabel("R²")
plt.xlabel("Model")
plt.ylim(0, 1) # R2 typically ranges from 0 to 1 for good models
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
# 6. Visualize Train vs Test RMSE
df_best_rmse = df_best_gap[["RMSE_train","RMSE_test"]].reset_index()
df_best_rmse_melt = df_best_rmse.melt(
    id_vars="Model",
    value_vars=["RMSE_train","RMSE_test"],
    var_name="Split",
    value_name="RMSE"
)
df_best_rmse_melt['Split'] = df_best_rmse_melt['Split'].replace({'RMSE_train': 'Train RMSE', 'RMSE_test': 'Test RMSE'}) # Improve
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_rmse_melt, x="Model", y="RMSE", hue="Split")
plt.title("Train vs Test RMSE by Best Model")
plt.ylabel("RMSE")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
# 7. Visualize Train vs Test R²
df_best_r2 = df_best_gap[["R2_train","R2_test"]].reset_index()
df_best_r2_melt = df_best_r2.melt(
    id_vars="Model",
    value_vars=["R2_train","R2_test"],
    var_name="Split",
    value_name="R2"
)
df_best_r2_melt['Split'] = df_best_r2_melt['Split'].replace({'R2_train': 'Train R²', 'R2_test': 'Test R²'}) # Improve labels
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_r2_melt, x="Model", y="R2", hue="Split")
plt.title("Train vs Test R² by Best Model")
plt.ylabel("R²")
plt.xlabel("Model")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
print("=== Full Metrics Table for Best Models (OLS, XGBoost, GMDH-like) ===")
display(df_best_models_metrics)
```

```
=== Full Metrics Table for Best Models (OLS, XGBoost, GMDH-like) ===
```

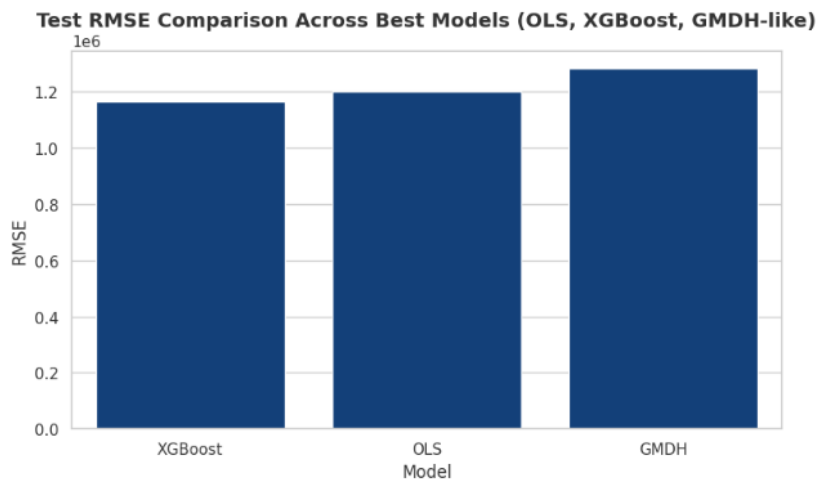
	Model	Split	RMSE	MAE	MAPE	R2
0	OLS	Train	9.308113e+05	657666.075627	14.459295	0.707084
1	OLS	Val	9.825708e+05	738291.910182	15.288038	0.659516
2	OLS	Test	1.200600e+06	931822.196594	20.027020	0.679095
12	XGBoost	Train	7.839473e+05	550387.690951	12.172478	0.792225
13	XGBoost	Val	1.014543e+06	774672.583984	16.101511	0.636998
14	XGBoost	Test	1.163692e+06	888072.930465	19.300390	0.698521
18	GMDH	Train	8.775716e+05	612177.764940	13.442188	0.739633
19	GMDH	Val	1.009727e+06	744447.466962	15.183333	0.640436
20	GMDH	Test	1.282316e+06	977545.956517	20.999222	0.633924

```
df_best_test_ranking = df_best_models_metrics[df_best_models_metrics["Split"] == "Test"].copy()
df_best_test_ranking = df_best_test_ranking.sort_values("RMSE").reset_index(drop=True)
print("\n=== MODEL RANKING (BY TEST RMSE - thấp hơn là tốt hơn) FOR BEST MODELS ===")
display(df_best_test_ranking)
```

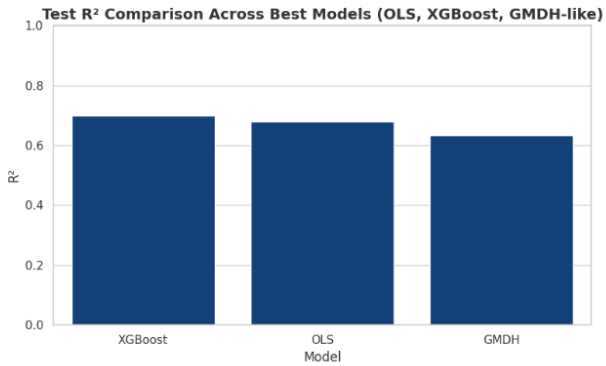
```
=== MODEL RANKING (BY TEST RMSE - thấp hơn là tốt hơn) FOR BEST MODELS ===
```

	Model	Split	RMSE	MAE	MAPE	R2
0	XGBoost	Test	1.163692e+06	888072.930465	19.300390	0.698521
1	OLS	Test	1.200600e+06	931822.196594	20.027020	0.679095
2	GMDH	Test	1.282316e+06	977545.956517	20.999222	0.633924

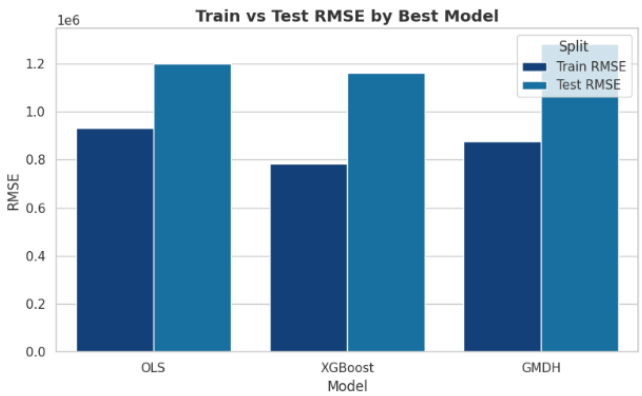
```
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_test_ranking, x="Model", y="RMSE")
plt.title("Test RMSE Comparison Across Best Models (OLS, XGBoost, GMDH-like)")
plt.ylabel("RMSE")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

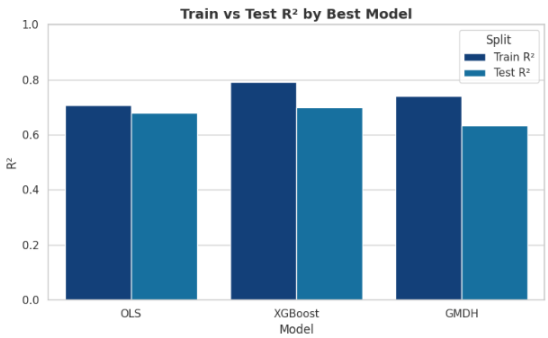


```python
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_test_ranking, x="Model", y="R2")
plt.title("Test R² Comparison Across Best Models (OLS, XGBoost, GMDH-like)")
plt.ylabel("R²")
plt.xlabel("Model")
plt.ylim(0, 1) # R2 typically ranges from 0 to 1 for good models
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```




```python
plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_rmse_melt, x="Model", y="RMSE", hue="Split")
plt.title("Train vs Test RMSE by Best Model")
plt.ylabel("RMSE")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```




```python
df_best_r2 = df_best_gap[["R2_train","R2_test"]].reset_index()
df_best_r2_melt = df_best_r2.melt(
    id_vars="Model",
    value_vars=["R2_train","R2_test"],
    var_name="Split",
    value_name="R2"
)
df_best_r2_melt['Split'] = df_best_r2_melt['Split'].replace({'R2_train': 'Train R²', 'R2_test': 'Test R²'}) # Improve labels

plt.figure(figsize=(8, 5))
sns.barplot(data=df_best_r2_melt, x="Model", y="R2", hue="Split")
plt.title("Train vs Test R² by Best Model")
plt.ylabel("R²")
plt.xlabel("Model")
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```